On Knowledge-Soundness of Plonk in ROM from Falsifiable Assumptions

July 10, 2025

Helger Lipmaa [1], Roberto Parisella [2], and Janno Siim [1,2]

[1] University of Tartu, Tartu, Estonia
[2] Simula UiB, Bergen, Norway

Abstract. Lipmaa, Parisella, and Siim [Eurocrypt, 2024] proved the extractability of the KZG polynomial commitment scheme under the falsifiable assumption ARSDH. They also proved that variants of fully optimized zk-SNARKs like Plonk can be made knowledge-sound in the random oracle model (ROM) under the ARSDH assumption. However, they did not consider various batching optimizations, resulting in their variant of Plonk having approximately 3.5 times longer argument. Our contributions are: (1) We prove that several batch-opening protocols for KZG, used in modern zk-SNARKs, have computational special soundness under the ARSDH assumption. (2) We prove that interactive Plonk has computational special soundness under the ARSDH assumption and a new falsifiable assumption SplitRSDH. We also prove that two minor modifications of the interactive Plonk have computational special soundness under only the ARSDH and a simpler variant of SplitRSDH. The Fiat-Shamir transform can be applied to obtain non-interactive versions, which are secure in the ROM under the same assumptions. We define a new type-safe oracle framework of the AGMOS (AGM with oblivious sampling) and prove SplitRSDH is secure in it.

Keywords: Batching · KZG · Plonk · special soundness · zk-SNARKs

1 Introduction

Zero-knowledge succinct non-interactive arguments of knowledge (zk-SNARKs) [Mic94,DL08,Gro10] are a cornerstone of modern cryptography, with widespread applications in blockchain scalability and privacy-preserving systems. Among existing schemes, Plonk [GWC19] stands out for its flexibility—particularly its support for lookup arguments [GW20a,EFG22,STW24,CFF+24] and custom gates [GW20b]—and has consequently gained significant traction.

Challenges and Approaches. Gentry and Wichs [GW11] showed that any sound SNARK in the standard model (i.e., without random oracles) must inherently rely on non-falsifiable assumptions. Consequently, the community has adopted one of three alternative strategies for securing zk-SNARKs:

- **Falsifiable Assumptions in the ROM.** Schemes such as hash-based SNARKs and STARKs [BBHR18,GLS+23] rely on falsifiable assumptions in the random oracle model (ROM). They benefit from transparent setups and efficient prover performance, though they often result in larger proofs and slower verification.
- **Idealized Group Models without ROM.**[3] Security proofs based solely on idealized group models (e.g., the Generic Group Model) avoid random oracles. However, such schemes either require circuit-dependent [Gro16] structured reference strings (SRSs) or have an impractically large SRS [GKM+18].

[3] Idealized group models include the Generic Group Model (GGM, [Sho97,Mau05]) and the Algebraic Group Model (AGM, [FKL18]). For simplicity (though doing this is debatable), we group knowledge assumptions under idealized group models.

– **Hybrid Models.** Combining the ROM with idealized group models yields efficient, constant-size SNARKs (including Plonk). Nonetheless, these proofs depend on the simultaneous validity of two idealized models.

Motivation. The reliance of efficient zk-SNARKs on hybrid models is problematic from both a theoretical and practical standpoint, motivating our central question: *Is it possible to prove that known practical zk-SNARKs, like Plonk, can be proven secure under a single idealized model?*

On Ideal Models. Every approach discussed so far relies on some idealized model—whether the ROM, an idealized group model (like the AGM or GGM), or a hybrid of both. Since these models are non-instantiable [CGH98,KRS25,Den02,Zha22], their use can introduce unforeseen issues when transitioning from theory to real-world applications. In particular, the recent preprint of Khovratovich, Rothblum, and Soukhanov [KRS25] shows that the ROM is non-instantiable in some real-life protocols. While the results are about the ROM, similar results can be expected for other ideal models. For this reason, if an ideal model must be employed, it is preferable to base the security proof on a single, well-understood model. Our work aims to minimize reliance on idealized group models in favor of the more familiar ROM. (Moreover, no efficient universal zk-SNARKs are known that avoid the ROM.)

More on AGM and AGMOS. Recent studies have further revealed subtle vulnerabilities in the AGM [Zha22,ZZK22,LPS23,BFHK24,JM24], underscoring the importance of giving proofs grounded solely on falsifiable assumptions. Following Zhandry [Zha22], Jaeger and Mohan proposed a coherent interpretation of the AGM, in particular, proving that under reasonable assumptions, AGM security implies GGM security, [JM24].

Separately, Lipmaa, Parisella, and Siim [LPS23] proposed AGM with Oblivious Sampling (AGMOS), a more realistic variant of the AGM where the adversary has an additional capacity to sample group elements obliviously, i.e., to sample group element without knowing the respective discrete logarithms. In elliptic curve groups, it is possible with hash-and-increment or admissible encodings [Ica09,LPS23]. They noted that a specific common use of the seminal KZG polynomial commitment scheme [KZG10], secure in the AGM, is not secure in the AGMOS and thus not in the standard model. [JM24]'s framework for AGM is rigorous, but it also does not consider oblivious sampling, which can significantly affect the security of KZG-based zk-SNARKs.

One also faces the tedious but necessary job of rewriting many existing AGM proofs. Only a few AGMOS proofs exist (e.g., for Plonk [FFR24] and Polymath [Lip24]); the security of other KZG-based zk-SNARKs in the AGMOS is still unproven. Because idealized group models frequently change, minimizing their use is desirable. For example, such models are a valuable tool for double-checking new standard-looking falsifiable assumptions, which has been their primary (and significantly less controversial) use case in the past.

The LPS Compiler. Lipmaa, Parisella, and Siim (LPS) made a breakthrough in 2024 [LPS24] by proving that the KZG commitment scheme has computational special soundness [CDS94] under the falsifiable ARSDH (Adaptive Rational Strong Diffie-Hellman) assumption. Combined with the Fiat-Shamir heuristic, this result implies knowledge-soundness for the resulting zk-SNARK. Exploiting this insight, the LPS compiler transforms an efficient polynomial IOP (PIOP)—such as the one used in Plonk—into a constant-size, knowledge-sound zk-SNARK. Given that many modern zk-SNARKs (including Plonk and those

in [CHM+20,RZ21,CFF+21,LSZ22]) integrate PIOPs with KZG commitments, the LPS compiler (when paired with Fiat-Shamir) delivers zk-SNARKs with constant-length proofs and linear-size SRS, all based on the ROM and ARSDH.

Nonetheless, the LPS compiler suffers from notable limitations. It only works with abstract PIOPs, preventing the manipulation of polynomial commitments without opening them. Moreover, it does not support batching techniques such as the popular "linearization trick" (LinTrick, [GWC19,CHM+20]), which are central to the efficiency of Plonk. One reason for the latter is that [LPS23] demonstrated that some instantiations of LinTrick can be insecure in settings that allow oblivious sampling, including the AGMOS and the standard model. While Faonio, Fiore, and Russo [FFR24] provided conditions (that avoid the [LPS23] attack) under which LinTrick is simulation-extractable in the AGM, their methods do not extend easily to the standard model. Consequently, whether the interactive version of Plonk is knowledge-sound in the standard model remains open.

The LPS compiler introduces even more overhead: compared to the original PIOP, the prover has to open each polynomial commitment at a new random point. Put together, [LPS24] proves that a *variant* of (interactive) Plonk is knowledge-sound under the ARSDH assumption. However, the variant is much less efficient than the fully optimized Plonk from [GWC19]. The most striking is the verifier's slowdown (who needs to implement 46 pairings instead of 2). See Table 1 for a comparison. We believe practitioners are unwilling to sacrifice efficiency by such a factor or change their battle-tested codebases.

LPS [LPS24] has an additional drawback: like [GWC19], it only establishes *knowledge-soundness* of the interactive Plonk. Knowledge-sound interactive protocols can suffer a significant security tightness loss after Fiat-Shamir compilation to zk-SNARKs [AFK22]. Practitioners often assume tight security for Fiat-Shamir zk-SNARKs in their implementations. Given that [GWC19,LPS24] prove that the interactive Plonk is knowledge-sound, this holds only heuristically. Achieving tight security with Fiat-Shamir requires more robust soundness notions, such as round-by-round soundness or special soundness.

To sum it up, Plonk is knowledge-sound, assuming both the ROM and the AGMOS [FFR24]. Lipmaa, Parisella, and Siim [LPS24] proved that a *variant* of Plonk is knowledge-sound in the ROM under falsifiable assumptions. However, their variant of Plonk is too inefficient to be used in practice. Given the importance of Plonk in industry, this situation is highly unsatisfactory. A proof that works in the ROM and the AGM(OS) but not in the ROM under falsifiable assumptions leaves it open to unknown problems of the AGM(OS).

Finally, Plonk's paper [GWC19] has been modified repeatedly to correct bugs and improve efficiency. A knowledge-soundness proof of Plonk under reasonable falsifiable assumptions in the ROM can be seen as an independent security audit, showing that the current variant of Plonk is secure and does not need to be changed anymore (except to improve efficiency). It also ascertains Plonk will stay secure even if more problems are found in the AGM(OS) or GGM, as long as one does not break the concrete falsifiable assumptions like ARSDH.

Main Research Question. In summary, our central research question is:

> *Can one prove the security of widely-used SNARKs—such as Plonk—solely under falsifiable assumptions within the ROM, thereby eliminating reliance on idealized group models?*

We answer this question in the affirmative.

Our Contributions. In this work, we make the following key contributions:

1. **Computational Special Soundness for Plonk:** We prove that the fully optimized interactive version of Plonk has computational special soundness relying solely on falsifiable assumptions (namely, ARSDH and SplitRSDH assumptions), thereby eliminating dependence on idealized group models. We also extend our proofs to two additional Plonk variants.
2. **Insecurity of the Linearization Trick:** We prove that the widely-used linearization trick does not have computational special soundness in the standard model, contrasting the fact that it has knowledge-soundness in the AGMOS under certain conditions.
3. **New Techniques—Sanitization and Rhino:** We introduce two novel techniques: *sanitization* (a design technique), which safeguards against oblivious sampling-based attacks, and *Rhino*, a reduction (proof) technique that leverages the new SplitRSDH assumption when standard extraction fails.
4. **Tight Fiat–Shamir Transformations:** By applying the Fiat-Shamir transformation, we obtain tightly knowledge-sound zk-SNARKs in the ROM. Our results lead to tighter security bounds than prior work without resorting to idealized group models.
5. **A Type-Safe AGMOS Framework:** We extend the oracle-based type-safe framework of Jaeger and Mohan [JM24] to incorporate oblivious sampling, yielding a type-safe AGMOS framework. This enables a clearer and more robust analysis of the underlying assumptions like SplitRSDH.

1.1 Technical Overview

In this section, we outline our technical contributions and proof strategies. Unlike prior work on Plonk and other zk-SNARKs (for instance, [LPS24] proved computational special soundness only for KZG, not for the interactive arguments), we establish computational special soundness for the arguments under consideration (see Section 2.2 for definitions). We decided to tackle several related problems, starting with the simplest (the BatchKZG protocol) and culminating in Plonk itself. Each subsequent problem requires a new approach: For example, SanLinTrick's analysis relies on sanitization, while Plonk's analysis relies on Rhino. Moreover, our analysis has broader implications since simpler protocols like BatchKZG and LinTrick are used in most other pairing-based zk-SNARKs.

Study 1: BatchKZG. In Section 3.1, we analyze the KZG batching protocol, BatchKZG, that is used in essentially all KZG-based zk-SNARKs. It is also the simplest protocol not analyzed in [LPS24]. We prove that BatchKZG satisfies computational $(n+1, m)$-special soundness, where m is the batching factor, assuming that KZG has computational $(n+1)$-special soundness. Our proof is structured in two parts. First, a tree extraction lemma constructs a tree extractor that, given an accepting transcript tree, outputs a tuple of accepting KZG transcripts. In Theorem 2, we use this tuple to build a computational special-soundness extractor $\mathsf{Ext}_{\mathsf{ss}}^{\mathsf{batch}}$ for BatchKZG and an adversary $\mathcal{B}_{\mathsf{ss}}^{\mathsf{kzg}}$ that breaks the computational special soundness of KZG if $\mathsf{Ext}_{\mathsf{ss}}^{\mathsf{batch}}$ fails.

Study 2: LinTrick. We next consider the linearization trick LinTrick [GWC19,CHM$^+$20] as formalized in [FFR24]. Assume that $\mathbf{a}(X) = (\mathsf{a}_\jmath(X))_{\jmath=1}^{n_a}$ and $\mathbf{d}(X) = (\mathsf{d}_t(X))_{t=1}^{n_b}$ are committed polynomials and $(\mathsf{g}_t(\boldsymbol{X}))_{t=1}^{n_b}$ are publicly known ("core") polynomials. The goal of LinTrick's prover is to prove that $\sum_{t=1}^{n_b} \mathsf{g}_t(\mathbf{a}(X))\mathsf{d}_t(X) = 0$. LinTrick and its variants are widely used in essentially all KZG-based zk-SNARKS, including Plonk. Moreover, as we will see shortly, LinTrick is a rather subtle protocol that does not lend itself to the same analysis as BatchKZG. We also introduce sanitization.

LinTrick is knowledge-sound in the AGM for any core polynomials $\mathsf{g}_t(\boldsymbol{X})$. Lipmaa, Parisella, and Siim [LPS23] proved that LinTrick is not always knowledge-sound in the AGMOS. This

result raised the question of the security of all zk-SNARKs that use LinTrick and have security proof in the AGM *without* oblivious sampling. Faonio, Fiore, and Russo [FFR24, Theorem 1] claim to solve this issue by proving that LinTrick is simulation-extractable in the AGM (under a non-falsifiable assumption) iff the public core polynomials are n-independent, which is a strengthening of linear independence. They proved that the corresponding polynomials in Plonk are n-independent, and thus, Plonk is simulation-extractable in the AGM. Moreover, they state, though they do not prove, that Plonk is knowledge-sound in the AGMOS. We fill this gap by giving a formal proof in the new type-safe AGMOS framework; see Section C.

Adapting an idea of [GKP22], we prove that if DL is hard, LinTrick does not have computational special soundness in the standard model. This does not contradict the result from [FFR24] as they use a different soundness definition, where the (non-black-box) extractor may depend on the adversary's code and random coins. The standard definition of (computational) special soundness does not allow these dependencies. In Section C, we provide a longer intuition of why LinTrick has knowledge-soundness in the AGMOS but does not have computational special soundness in the standard model. We also refer the interested reader to the discussion in the technical overview of [GKP22]. Thus, in our computational special-soundness proof for Plonk, we cannot rely on LinTrick's computational special soundness.

The attacks of [LPS23,FFR24] are possible since the adversary can obliviously sample the polynomials $\mathsf{d}_t(X)$ from some joint distribution. We introduce a new proof technique (*sanitization*) to bypass this issue. The prover of sanitized LinTrick (SanLinTrick, Section 3.3) batch opens all $\mathsf{d}_t(X)$ at a single location. We prove SanLinTrick has computational special soundness, assuming that KZG has computational special soundness and evaluation binding, that is, under ARSDH.

Study 3: Plonk And Variants. After analyzing more basic subroutines, proving that LinTrick does not have computational special soundness, and introducing one of our proof techniques (sanitization), we are now ready to focus on the interactive version of Plonk. At first glance, LinTrick's insecurity in the standard model might seem to contradict the claim that Plonk is sound. To resolve this, we study three distinct solutions, Plonk and its two variants. Table 1 compares these variants regarding efficiency and underlying assumptions.

Study 3a: Plonk. Recognizing that practitioners are unlikely to alter their well-established implementations, we first prove that the fully optimized version of Plonk (as presented in [GWC19]) has computationally special soundness. This proof employs a different proof technique—named *Rhino* (a shorthand for reduction to a hard assumption if not polynomial). To illustrate, consider that the Plonk prover sends $[a, b, c, z, t_{lo}, t_{mid}, t_{hi}]_1 \in \mathbb{G}_1^7$ to the verifier. At a random evaluation point $\mathfrak{z}$ chosen by the verifier, one obtains $[t_{lo} + \mathfrak{z}^n t_{mid} + \mathfrak{z}^{2n} t_{hi}]_1 = [t]_1$ (for some n), where $[t]_1$ encodes a high-degree polynomial. In the AGM, one can extract polynomials $\mathsf{t}_{\mathsf{lo}}(X)$, $\mathsf{t}_{\mathsf{mid}}(X)$, and $\mathsf{t}_{\mathsf{hi}}(X)$ from the commitments $[t_{lo}, t_{mid}, t_{hi}]_1$, given Plonk's accepting proof. However, a standard model extractor does not have enough information for this. In our proof (see Theorem 6), rather than extracting $\mathsf{t}_{\mathsf{lo}}(X)$, $\mathsf{t}_{\mathsf{mid}}(X)$, and $\mathsf{t}_{\mathsf{hi}}(X)$ directly, we compute a candidate *rational function* $\mathsf{t}(X)$ from previously extracted polynomials. If $\mathsf{t}(X)$ is a polynomial, it serves as a valid opening of $[t]_1$ and completes the proof. Otherwise, failing to be a polynomial implies breaking a new, standard-looking assumption, denoted SplitRSDH. This forms the essence of the Rhino technique.

Study 3b: SanPlonk. By replacing the use of LinTrick in Plonk with SanLinTrick, we obtain SanPlonk (see Table 1), which incurs the cost of one additional field element in communication.

Table 1. Comparison of different versions of Plonk, secure under falsifiable assumptions. The number of bits are given for the BLS381-12 pairing $e : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$. For convenience, $|\mathbb{G}_1|$ (resp. $|\mathbb{F}|$) denotes the number of bits needed to represent an element of $\mathbb{G}_1$ (resp. $|\mathbb{F}|$). Additionally, n denotes the number of gates, and ℓ is the number of elements in the public input of the circuit whose satisfiability is being proven.

Argument	**Proof size (bits)**	**Prover**	**Verifier**	**Assumptions**
Plonk's polynomial IOP compiled with [LPS24]	$23\|\mathbb{F}\| + 30\|\mathbb{G}_1\|$ (17408)	$30n$ $\mathbb{G}_1$ Exp., $\mathcal{O}(n \log n)$ $\mathbb{F}$ Ops.	46 Pair., 24 $\mathbb{G}_1$ Exp., $\mathcal{O}(\ell + \log n)$ $\mathbb{F}$ Ops.	ARSDH
Plonk (current work)	$6\|\mathbb{F}\| + 9\|\mathbb{G}_1\|$ (4992)	$9n$ $\mathbb{G}_1$ Exp., $\mathcal{O}(n \log n)$ $\mathbb{F}$ Ops.	2 Pair., 18 $\mathbb{G}_1$ Exp., $\mathcal{O}(\ell + \log n)$ $\mathbb{F}$ Ops.	ARSDH, SplitRSDH
SanPlonk (current work)	$7\|\mathbb{F}\| + 9\|\mathbb{G}_1\|$ (5248)	$9n$ $\mathbb{G}_1$ Exp., $\mathcal{O}(n \log n)$ $\mathbb{F}$ Ops.	2 Pair., 19 $\mathbb{G}_1$ Exp., $\mathcal{O}(\ell + \log n)$ $\mathbb{F}$ Ops.	ARSDH
SmallPlonk (current work)	$6\|\mathbb{F}\| + 7\|\mathbb{G}_1\|$ (4224)	$11n$ $\mathbb{G}_1$ Exp., $\mathcal{O}(n \log n)$ $\mathbb{F}$ Ops.	2 Pair., 16 $\mathbb{G}_1$ Exp., $\mathcal{O}(\ell + \log n)$ $\mathbb{F}$ Ops.	ARSDH, SplitRSDH

Since SanLinTrick has computational special soundness, so does SanPlonk. SanPlonk relies crucially on our sanitization technique. SanPlonk is mostly interesting from the security viewpoint since its computational special soundness relies solely on ARSDH.

Study 3c: SmallPlonk. We also analyze SmallPlonk, a variant where the commitment $[t]_1$ is not split into $[t_{lo}, t_{mid}, t_{hi}]_1$. While SmallPlonk yields a shorter argument, it has a slower prover. We use Rhino to prove that SmallPlonk has computational special soundness under ARSDH and a simpler variant of SplitRSDH.

On Sanitization And Rhino. Sanitization is a design technique that augments communication in SanLinTrick and SanPlonk by adding a single field element to ensure computational special soundness. This enhancement is particularly significant for SanLinTrick, given that its base protocol LinTrick lacks computational special soundness. In the case of SanPlonk, sanitization secures the protocol without needing an additional assumption (namely, SplitRSDH).

In contrast, Rhino is purely a proof technique. It does not require any extra communication; however, it applies only to protocols like Plonk that can be proven secure—albeit at the cost of introducing an extra assumption. Thus, sanitization and Rhino are complementary techniques; each offers distinct advantages. Sanitization is more broadly applicable because it can sometimes transform an insecure protocol into a secure one.

New Type-Safe Framework for AGMOS. SplitRSDH is a novel but falsifiable and standard-looking assumption (see Definition 3). SplitRSDH has a clear intuition about its security in the AGM(OS); see Section 4 (the first paragraph after Definition 3). We prove that SplitRSDH is secure in the AGMOS [LPS23], probably the most realistic known variant of AGM. For this, we describe AGMOS using the oracle framework of AGM from [JM24] but adding extra oracles for oblivious sampling as was done in [LPS23]. Thus, the lifting result of [JM24], stating that under certain conditions, a generic reduction implies an algebraic reduction (see Fact 1), holds for the AGMOS. Hence, SplitRSDH is secure in the GGM. See Section 4 and Appendix B for more analysis. The new AGMOS framework is simpler to use and understand than the one in [LPS23]. *This forms an important independent contribution to the current paper.*

We emphasize that Plonk does not rely on AGMOS or other idealized group models. The use of AGMOS to prove the security of SplitRSDH should be seen as an independent element of cryptanalysis for the new assumption.

Generalizations. Computational special soundness of Plonk with custom gates [GW20b] follows from our analysis (since such extensions only affect parameters like n or the shape of certain polynomials without changing the underlying falsifiable assumptions). However, lookup arguments like [EFG22,CFF+24] are complex protocols in their own right that need to be analyzed separately.

We believe that both sanitization and Rhino can deepen our understanding of the security of other KZG-based zk-SNARKs [CHM+20,RZ21,CFF+21,LSZ22]. We opted not to include proofs for other zk-SNARKs because each one is rather lengthy—and our paper is already long enough. A natural open question is: which zk-SNARKs that use LinTrick can be shown to have computational special soundness? Note that some aspects—for example, the precise assumptions needed for Rhino—will still differ even if they do.

Fiat-Shamir. We prove that the interactive versions of Plonk, SanPlonk, and SmallPlonk satisfy computational $\boldsymbol{\kappa}$-special soundness under falsifiable assumptions. By applying the Fiat-Shamir transformation (see Section E), we obtain zk-SNARKs that are knowledge-sound in the ROM with optimal tightness. Recent work [DG23,AFKR23] shows that such transformations are significantly tighter when applied to computationally special-sound protocols than knowledge-sound ones. This improvement yields tighter security bounds than prior proofs that relied on both the ROM and idealized group models.

Zero-Knowledge. Thus far, we have focused on soundness. In sanitized protocols, the requirement to batch-open certain polynomials at an extra point may necessitate adding a randomizer to maintain zero knowledge. For SanPlonk, we incorporate this new randomizer and prove that the protocol remains zero-knowledge. Zero-knowledge for Plonk and SmallPlonk was established in [Sef24].

Organization. The remainder of the paper is organized as follows. We review the necessary background in Section 2. In Section 3, we analyze the security of various subroutines—most notably the linearization trick and our sanitization approach. In Section 4, we introduce the new SplitRSDH assumption and analyze its security. Finally, in Section 5, we introduce Plonk and its variants and prove their security. In Section 6, we present our extended AGMOS framework.

2 Preliminaries

Let λ denote the security parameter. By $f(\lambda) \approx_\lambda 0$, we mean that f is a negligible function. PPT (resp. DPT) stands for probabilistic (resp. deterministic) polynomial time. We denote the concatenation of vectors $\boldsymbol{u}$ and $\boldsymbol{v}$ as $\boldsymbol{u}\|\boldsymbol{v}$. $\mathbb{F} = \mathbb{F}_p$ is a finite field of prime order p. $\mathbb{F}[X]$ is the polynomial ring in variable X over the field $\mathbb{F}$ and $\mathbb{F}_{\leq n}[X] \subset \mathbb{F}[X]$ is the set of polynomials of at most degree n. We denote $[a, b] := \{a, a+1, \ldots, b\}$, where $a \leq b$ are integers. Our notation is inspired by Plonk's paper [GWC19] (for example, we denote polynomials by using $\mathsf{SansSerif}$), but we do not follow it universally. For any set $\mathcal{S}$, $\mathsf{Z}_{\mathcal{S}}(X) = \prod_{s\in\mathcal{S}}(X - s)$ denotes the vanishing polynomial over $\mathcal{S}$.

Bilinear Groups. A bilinear group generator $\mathsf{Pgen}(1^\lambda)$ returns $\mathsf{p} = (p, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, \hat{e}, [1]_1, [1]_2)$, where $\mathbb{G}_1$, $\mathbb{G}_2$, and $\mathbb{G}_T$ are additive cyclic (thus, abelian) groups of prime order $p = |\mathbb{F}|$, $\hat{e} : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$ is a non-degenerate efficiently computable bilinear pairing, and $[1]_\iota$ is a fixed generator of $\mathbb{G}_\iota$. While $[1]_\iota$ is part of p, we often give it as an explicit input to different algorithms for clarity. The bilinear pairing is of Type-3, that is, there is no efficient isomorphism between $\mathbb{G}_1$ and $\mathbb{G}_2$. We use the common bracket notation, that is, for $\iota \in \{1, 2, T\}$ and $a \in \mathbb{F}$, we write $[a]_\iota$ to denote $a[1]_\iota$. For convenience, we write vectors as $([a_1]_\iota, \ldots, [a_n]_\iota) := [a_1, \ldots, a_n]_\iota$. We denote $\hat{e}([a]_1, [b]_2)$ by $[a]_1 \bullet [b]_2$ and assume $[1]_T = [1]_1 \bullet [1]_2$. Thus, $[a]_1 \bullet [b]_2 = [ab]_T$ for any $a, b \in \mathbb{F}$.

We recall the following falsifiable assumption ARSDH [LPS24]; however, they called this assumption $(n+1)$-ARSDH. Calling it n-ARSDH is more in line with standard naming conventions.

Definition 1. *The* n-ARSDH (Adaptive Rational Strong Diffie-Hellman) *assumption holds for* Pgen *in* $\mathbb{G}_1$ *if for any PPT* $\mathcal{A}$, $\mathsf{Adv}^{\mathrm{arsdh}}_{\mathsf{Pgen},n,\mathbb{G}_1,\mathcal{A}}(\lambda) :=$

$$\Pr\left[\begin{array}{l|l} \mathcal{S} \subset \mathbb{F} \wedge |\mathcal{S}| = n+1 \wedge & \mathsf{p} \leftarrow \mathsf{Pgen}(1^\lambda); x \leftarrow_\$ \mathbb{F}; \\ [g]_1 \neq [0]_1 \wedge [\varphi]_1 = [\frac{g}{\mathsf{Z}_\mathcal{S}(x)}]_1 & \mathsf{ck} \leftarrow ([(x^i)_{i=0}^n]_1, [1, x]_2); \\ & (\mathcal{S}, [g, \varphi]_1) \leftarrow \mathcal{A}(\mathsf{ck}) \end{array}\right] \approx_\lambda 0 \ ,$$

where $\mathsf{Z}_\mathcal{S}(X) := \prod_{s\in\mathcal{S}}(X - s)$. *The condition* $[\varphi]_1 = [\frac{g}{\mathsf{Z}_\mathcal{S}(x)}]_1$ *is equivalent to* $[g]_1 \bullet [1]_2 = [\varphi]_1 \bullet [\mathsf{Z}_\mathcal{S}(x)]_2$.

ARSDH is a minor generalization of the RSDH assumption [GR19], where the set $\mathcal{S}$ is fixed rather than chosen by the adversary. Lipmaa, Parisella, and Siim [LPS24] proved that ARSDH implies the well-known SDH assumption. Intuitively, ARSDH holds in the AGM since the explanations $g(X)$ and $\varphi(X)$ of $[g]_1$ and $[\varphi]_1$ must be degree-$\leq n$ polynomials. The verification equation implies that $g(X) = \varphi(X)\mathsf{Z}_\mathcal{S}(X)$. By comparing the degrees of the polynomials, we get that $g(X) = \varphi(X) = 0$, a contradiction with $[g]_1 \neq [0]_1$.

2.1 Polynomial Commitment Schemes

In a (univariate) polynomial commitment scheme (PCS, [KZG10]), the prover commits to a polynomial $\mathsf{f} \in \mathbb{F}_{\leq n}[X]$ and later opens it to $\mathsf{f}(\mathfrak{z})$ for $\mathfrak{z} \in \mathbb{F}$ chosen by the verifier. A *non-interactive* PCS consists of the following five algorithms. (1) Given 1^λ, $\mathsf{Pgen}(1^\lambda)$ returns system parameters p. (2) Given p and an upperbound n on the polynomial degree, $\mathsf{KGen}(\mathsf{p}, n)$ returns $(\mathsf{ck}, \mathsf{tk})$, where ck is the commitment key and tk is the trapdoor. We assume ck implicitly contains p. (3) Given ck and a polynomial $\mathsf{f} \in \mathbb{F}_{\leq n}[X]$, $\mathsf{Com}(\mathsf{ck}, \mathsf{f})$ returns a commitment C to f. (4) Given ck, C, an evaluation point $\mathfrak{z} \in \mathbb{F}$, and $\mathsf{f} \in \mathbb{F}_{\leq n}[X]$, $\mathsf{Open}(\mathsf{ck}, C, \mathfrak{z}, \mathsf{f})$ returns $(\bar{f}, \pi)$, where $\bar{f} \leftarrow \mathsf{f}(\mathfrak{z})$ is an evaluation, and π is an evaluation proof. (5) Given ck, C, $\mathfrak{z}$, $\bar{f}$, and π, $\mathsf{Vf}(\mathsf{ck}, C, \mathfrak{z}, \bar{f}, \pi)$ returns 1 (accept) or 0 (reject).

The KZG commitment scheme is a well-known non-interactive PCS [KZG10]; another such scheme is PST (multilinear KZG) [PST13]. Many PCSs have either an interactive opening or verification phase [BBHR18,BBB+18,BFS20].

A non-interactive PCS PC is *complete*, if for any λ, $\mathsf{p} \leftarrow \mathsf{Pgen}(1^\lambda)$, $n \in \mathsf{poly}(\lambda)$, $\mathfrak{z} \in \mathbb{F}$, $\mathsf{f} \in \mathbb{F}_{\leq n}[X]$,

$$\Pr\left[\mathsf{V}(\mathsf{ck}, C, \mathfrak{z}, \bar{f}, \pi) = 1 \,\middle|\, \begin{array}{l} (\mathsf{ck}, \mathsf{tk}) \leftarrow \mathsf{KGen}(\mathsf{p}, n); C \leftarrow \mathsf{Com}(\mathsf{ck}, \mathsf{f}); \\ (\bar{f}, \pi) \leftarrow \mathsf{Open}(\mathsf{ck}, C, \mathfrak{z}, \mathsf{f}) \end{array}\right] = 1.$$

A non-interactive PCS PC is *binding*, if for any PPT $\mathcal{A}$, $\mathsf{Adv}^{\mathrm{bind}}_{\mathsf{Pgen},\mathsf{PC},n,\mathcal{A}}(\lambda) :=$

$$\Pr\left[\begin{array}{l} C = \mathsf{Com}(\mathsf{ck},\mathsf{f}) = \mathsf{Com}(\mathsf{ck},\mathsf{g}) \wedge \\ \mathsf{f} \neq \mathsf{g} \wedge \deg(\mathsf{f}) \leq n, \deg(\mathsf{g}) \leq n \end{array} \middle| \begin{array}{l} \mathsf{p} \leftarrow \mathsf{Pgen}(1^\lambda); (\mathsf{ck},\mathsf{tk}) \leftarrow \mathsf{KGen}(\mathsf{p},n); \\ (C,\mathsf{f},\mathsf{g}) \leftarrow \mathcal{A}(\mathsf{ck}) \end{array}\right] \approx_\lambda 0 \ .$$

A PCS PC is *evaluation binding* [KZG10] for Pgen, if for any $n \in \mathsf{poly}(\lambda)$, and PPT adversary $\mathcal{A}$, $\mathsf{Adv}^{\mathrm{evb}}_{\mathsf{Pgen},\mathsf{PC},n,\mathcal{A}}(\lambda) :=$

$$\Pr\left[\begin{array}{l} \mathsf{V}(\mathsf{ck},C,\mathfrak{z},\bar{f},\pi) = 1 \wedge \\ \mathsf{V}(\mathsf{ck},C,\mathfrak{z},\bar{f}',\pi') = 1 \ \wedge \ \bar{f} \neq \bar{f}' \end{array} \middle| \begin{array}{l} \mathsf{p} \leftarrow \mathsf{Pgen}(1^\lambda); (\mathsf{ck},\mathsf{tk}) \leftarrow \mathsf{KGen}(\mathsf{p},n); \\ (C,\mathfrak{z},\bar{f},\pi,\bar{f}',\pi') \leftarrow \mathcal{A}(\mathsf{ck}) \end{array}\right] \approx_\lambda 0 \ .$$

Evaluation binding implies binding. Really, suppose $\mathcal{A}_{\mathsf{bind}}$ succeeded in breaking binding, outputting $([c]_1, \mathsf{f}(X), \mathsf{f}'(X))$ such that $c = \mathsf{f}(x) = \mathsf{f}'(x)$ and $\mathsf{f}(X) \neq \mathsf{f}'(X)$. Then, we can find a point $\mathfrak{z}$, such that $\mathsf{f}(\mathfrak{z}) \neq \mathsf{f}'(\mathfrak{z})$, open $[c]_1$ at $\mathsf{f}(\alpha)$, and $\mathsf{f}'(\alpha)$, and break evaluation binding.

We rely on the following terminology from [LPS24]. We call $\mathsf{tr} = (C, \mathfrak{z}, \bar{f}, \pi)$ a *transcript* of the PCS. We say that a commitment key ck and a transcript tr is *accepting* when $\mathsf{V}(\mathsf{ck},\mathsf{tr}) = 1$. For any $n \geq 1$ and any commitment key ck outputted by $\mathsf{KGen}(\mathsf{p},n)$, we define the following relations.

$$\begin{aligned} \mathcal{R}_{\mathsf{ck}} &:= \{(C,\mathsf{f}) : C = \mathsf{PC.Com}(\mathsf{ck},\mathsf{f}) \ \wedge \ \deg(\mathsf{f}) \leq n\} \ , \\ \mathcal{R}_{\mathsf{ck},\mathbf{tr}} &:= \{(C,\mathsf{f}) : (C,\mathsf{f}) \in \mathcal{R}_{\mathsf{ck}} \ \wedge \ \forall j \in [0,n].\mathsf{f}(\mathfrak{z}_j) = \bar{f}_j\} \ , \end{aligned} \tag{1}$$

where $\mathbf{tr} = (\mathsf{tr}_0, \ldots, \mathsf{tr}_n)$ contains $n+1$ accepting transcripts $\mathsf{tr}_j = (C, \mathfrak{z}_j, \bar{f}_j, \pi_j)$ such that C is the same in all transcripts, but $\mathfrak{z}_j$-s are pairwise distinct.

Let $n \in \mathsf{poly}(\lambda)$ with $n \geq 1$. A non-interactive PCS PC *has computational* $(n+1)$*-special soundness* [LPS24] for Pgen, if there exists a DPT extractor $\mathsf{Ext}_{\mathsf{ss}}$, such that for any PPT adversary $\mathcal{A}_{\mathsf{ss}}$, $\mathsf{Adv}^{\mathrm{ss}}_{\mathsf{Pgen},\mathsf{PC},\mathsf{Ext}_{\mathsf{ss}},n,\mathcal{A}_{\mathsf{ss}}}(\lambda) :=$

$$\Pr\left[\begin{array}{l} \mathbf{tr} = (\mathsf{tr}_j)_{j=0}^n \wedge \\ \forall j \in [0,n]. \begin{pmatrix} \mathsf{tr}_j = (C,\mathfrak{z}_j,\bar{f}_j,\pi_j) \\ \wedge \mathsf{V}(\mathsf{ck},\mathsf{tr}_j) = 1 \end{pmatrix} \\ \wedge \ (\forall i \neq j.\mathfrak{z}_i \neq \mathfrak{z}_j) \ \wedge \ (C,\mathsf{f}) \notin \mathcal{R}_{\mathsf{ck},\mathbf{tr}} \end{array} \middle| \begin{array}{l} \mathsf{p} \leftarrow \mathsf{Pgen}(1^\lambda); \\ (\mathsf{ck},\mathsf{tk}) \leftarrow \mathsf{KGen}(\mathsf{p},n); \\ \mathbf{tr} \leftarrow \mathcal{A}_{\mathsf{ss}}(\mathsf{ck}); \\ \mathsf{f} \leftarrow \mathsf{Ext}_{\mathsf{ss}}(\mathsf{ck},\mathbf{tr}) \end{array}\right] \approx_\lambda 0 \ .$$

The well-known KZG [KZG10] PCS is defined as follows:
$\mathsf{Pgen}(1^\lambda)$**:** is the bilinear group generator.
$\mathsf{KGen}(\mathsf{p},n)$**:** $\mathsf{tk} = x \leftarrow_\$ \mathbb{F}^*$; $\mathsf{ck} \leftarrow (\mathsf{p}, [(x^i)_{i=0}^n]_1, [1,x]_2)$; return $(\mathsf{ck},\mathsf{tk})$.
$\mathsf{Com}(\mathsf{ck},\mathsf{f})$**:** return $C \leftarrow [\mathsf{f}(x)]_1$.
$\mathsf{Open}(\mathsf{ck},C,\mathfrak{z},\mathsf{f})$**:** $\bar{f} \leftarrow \mathsf{f}(\mathfrak{z})$; $\varphi(X) \leftarrow (\mathsf{f}(X) - \bar{f})/(X - \mathfrak{z})$; $\pi \leftarrow [\varphi(x)]_1$; return $(\bar{f},\pi)$.
$\mathsf{V}(\mathsf{ck},C,\mathfrak{z},\bar{f},\pi)$**:** Return 1 iff $(C - \bar{f}[1]_1) \bullet [1]_2 = \pi \bullet [x - \mathfrak{z}]_2$.
KZG is evaluation binding under the n-SDH assumption [KZG10] and non-black-box extractable in the AGM [FKL18] under the PDL assumption [Lip12,CHM+20] and in AGMOS [LPS23] under the PDL and TOFR assumptions. All of these are falsifiable assumptions. We refer to the respective papers for their definition. The intuition behind the KZG's extractability in the AGM is straightforward. The commitment C to a degree $\leq n$ polynomial is the evaluation of the polynomial at a secret point x, where the commitment key contains $[(x^i)_{i=0}^n]_1, [1,x]_2$. The AGM explanation of C is a degree $\leq n$ polynomial $\mathsf{f}(X)$, such that $C = [\mathsf{f}(x)]_1$. The opening algorithm uses the fact that $(X - \mathfrak{z}) \mid (\mathsf{f}(X) - \bar{f}) \Leftrightarrow \mathsf{f}(\mathfrak{z}) = \bar{f}$.

Lipmaa, Parisella, and Siim [LPS24] proved the following.

Theorem 1. *If the n-ARSDH assumption holds, then KZG for degree $\leq n$ polynomials has computational $(n+1)$-special soundness: There exists a DPT KZG computational special-soundness extractor $\mathsf{Ext}^{\mathsf{kzg}}_{\mathsf{ss}}$, such that for any PPT $\mathcal{A}_{\mathsf{ss}}$, there exists a PPT $\mathcal{B}$, such that $\mathsf{Adv}^{\mathrm{ss}}_{\mathsf{Pgen},\mathsf{PC},\mathsf{Ext}^{\mathsf{kzg}}_{\mathsf{ss}},n,\mathcal{A}_{\mathsf{ss}}}(\lambda) \leq \mathsf{Adv}^{\mathrm{arsdh}}_{\mathsf{Pgen},n,\mathbb{G}_1,\mathcal{B}}(\lambda)$.*

We say that a non-interactive polynomial commitment scheme is *triply homomorphic*, if: if $(C_j, \mathfrak{z}, \bar{f}_j, \pi_j)$ is an accepting transcript for every j, then so is $(\sum s_j C_j, \mathfrak{z}, \sum s_j \bar{f}_j, \sum s_j \pi_j)$ for any $s_j \in \mathbb{F}$. Clearly, KZG is triply homomorphic.

2.2 Interactive Arguments

Let $\mathcal{R} \subseteq \{0,1\}^* \times \{0,1\}^* \times \{0,1\}^*$ be a ternary relation. $\mathcal{R}$ contains triples $(\mathtt{srs}, \mathbb{x}, \mathbb{w}) \in \mathcal{R}$ where $\mathtt{srs}$ is a public common reference string, $\mathbb{x}$ is a public statement, and $\mathbb{w}$ is a private witness. We only consider NP-*relations* $\mathcal{R}$, where the validity of a witness $\mathbb{w}$ can be verified in time polynomial in $|\mathbb{x}| + |\mathtt{srs}|$. Let $\mathsf{Pgen}(1^\lambda)$ generate system parameters p that are available to all algorithms. We do not always explicitly write p as an input.

An *interactive argument* $\Pi = (\mathsf{KGen}, \mathsf{P}, \mathsf{V})$ for a relation $\mathcal{R}$ is an interactive protocol between a probabilistic polynomial time prover P and verifier V. The key generator KGen generates a structured reference string $\mathtt{srs}$ and a trapdoor $\mathtt{td}_{\mathtt{srs}}$ at the beginning of the protocol. Both P and V take as public input $\mathtt{srs}$ and a statement $\mathbb{x}$ and, additionally, P takes as private input a witness $\mathbb{w}$. The verifier V either accepts or rejects the transcript (all messages exchanged in the protocol execution). Accordingly, we say the transcript is accepting or rejecting.

Let $\boldsymbol{\kappa} = (\kappa_1, \ldots, \kappa_\mu) \in \mathbb{N}^\mu$. A $\boldsymbol{\kappa}$-*tree of transcripts* for a $(2\mu+1)$-move public-coin interactive argument $\Pi = (\mathsf{KGen}, \mathsf{P}, \mathsf{V})$ is a set of $K = \prod_{i=1}^{\mu} \kappa_i$ transcripts arranged in the following tree structure. The nodes in this tree correspond to the prover's messages and the edges to the verifier's challenges. Every node at depth i has precisely κ_i children corresponding to κ_i pairwise distinct challenges. Every transcript corresponds to exactly one path from the root node to a leaf.

Definition 2. *Let $\boldsymbol{\kappa} = (\kappa_1, \ldots, \kappa_\mu)$. A $(2\mu+1)$-move public-coin interactive argument $\Pi = (\mathsf{KGen}, \mathsf{P}, \mathsf{V})$ for relation $\mathcal{R}$ has* computational $\boldsymbol{\kappa}$-special soundness *if there exists a DPT extractor $\mathsf{Ext}_{\mathsf{ss}}$, such that for any PPT $\mathcal{A}_{\mathsf{ss}}$, $\mathsf{Adv}^{\mathsf{ss}}_{\mathsf{Pgen},\Pi,\mathsf{Ext}_{\mathsf{ss}},\boldsymbol{\kappa},\mathcal{A}}(\lambda) :=$*

$$\Pr\left[\begin{array}{l|l} \mathcal{T} \text{ is a } \boldsymbol{\kappa}\text{-tree of} & \mathsf{p} \leftarrow \mathsf{Pgen}(1^\lambda); (\mathtt{srs}, \mathtt{td}_{\mathtt{srs}}) \leftarrow \mathsf{KGen}(\mathsf{p}); \\ \text{accepting transcripts} & (\mathbb{x}, \mathcal{T}) \leftarrow \mathcal{A}_{\mathsf{ss}}(\mathtt{srs}); \mathbb{w} \leftarrow \mathsf{Ext}_{\mathsf{ss}}(\mathtt{srs}, \mathbb{x}, \mathcal{T}) \\ \wedge\, (\mathtt{srs}, \mathbb{x}, \mathbb{w}) \notin \mathcal{R} & \end{array}\right] \approx_\lambda 0 \ .$$

3 Computational Special Soundness of Subroutines

It is a common practice to prove the security of KZG-based interactive arguments in idealized group models like AGM. Extending the techniques of [LPS24], we prove that some of these arguments have computational special soundness under falsifiable assumptions. We describe the standard batching argument BatchKZG and the linearization trick LinTrick. We prove that BatchKZG has computational special soundness. We show LinTrick does not have computational special soundness in the standard model. We apply a novel technique of sanitization to obtain a computationally special-sound version of LinTrick.

3.1 Computational Special Soundness of Batch-KZG

Consider the well-known argument BatchKZG from Fig. 1, where the prover has committed to m polynomials and then engages in a single batch proof that opens all m polynomials simultaneously at the common evaluation point $\mathfrak{z}$. Here, $[f_t]_1$ are polynomial commitments, $\bar{f}_t$ are purported evaluations of $[f_t]_1$ at $\mathfrak{z}$, and $[h]_1$ is the batched opening of all m commitments. Note that BatchKZG's verifier essentially checks that $\mathsf{k.tr} = (\sum_{t=1}^{m} v^{t-1}[f_t]_1, \mathfrak{z}, \sum_{t=1}^{m} v^{t-1}\bar{f}_t, [h]_1)$ is an accepting KZG transcript for a randomly sampled v.

$\mathsf{KGen}(\mathsf{p})$: $x \leftarrow\!\!\$\, \mathbb{F}^*$; return $\mathtt{srs} \leftarrow ([(x^s)_{s=0}^n]_1, [1, x]_2)$ and $\mathtt{td}_{\mathtt{srs}} \leftarrow x$;

$\mathsf{P}(\mathtt{srs}, \mathbb{w} = (\mathsf{f}_t(X))_{t=1}^m)$: for $t \in [1, m]$, $[f_t]_1 \leftarrow [\mathsf{f}_t(x)]_1$; return $[(f_t)_{t=1}^m]_1$;
V: return $\mathfrak{z} \leftarrow\!\!\$\, \mathbb{F}$; // Evaluation point
P: for $t \in [1, m]$, $\bar{f}_t \leftarrow \mathsf{f}_t(\mathfrak{z})$; return $(\bar{f}_t)_{t=1}^m$;
V: return $v \leftarrow\!\!\$\, \mathbb{F}$; // Batch coefficient
P: $\mathsf{h}(X) \leftarrow \left(\sum_{t=1}^m v^{t-1}(\mathsf{f}_t(X) - \bar{f}_t)\right)/(X - \mathfrak{z})$; return $[h]_1 \leftarrow [\mathsf{h}(x)]_1$;
V: check $[\sum_{t=1}^m v^{t-1}(f_t - \bar{f}_t)]_1 \bullet [1]_2 = [h]_1 \bullet [x - \mathfrak{z}]_2$;

Fig. 1. The protocol BatchKZG.

$\mathsf{TE}_{\mathsf{batch}}(\mathsf{ck}, \mathcal{T})$

Parse $\mathcal{T} = (\mathsf{tr}_{ij})_{i \in [1, n+1], j \in [1, m]}$; // $\mathsf{tr}_{ij} = ([(f_t)_{t=1}^m]_1, \mathfrak{z}_i, (\bar{f}_{ti})_{t=1}^m, v_{ij}, [h_{ij}]_1)$
$[h'_{i1}, \ldots, h'_{im}]_1^\intercal \leftarrow \boldsymbol{V}_i^{-1}[h_{i1}, \ldots, h_{im}]_1^\intercal$; (*) // See $\boldsymbol{V}_i$ in Eq. (2)
for $t \in [1, m]$ **do**
  for $i \in [1, n+1]$ **do** $\mathsf{k.tr}'_{it} \leftarrow ([f_t]_1, \mathfrak{z}_i, \bar{f}_{ti}, [h'_{ti}]_1)$; **endfor** (**)
  $\mathbf{k.tr}'_t \leftarrow (\mathsf{k.tr}'_{1t}, \ldots, \mathsf{k.tr}'_{n+1,t})$; **endfor**
return $(\mathbf{k.tr}'_t)_{t=1}^m$;

Fig. 2. The subroutine $\mathsf{TE}_{\mathsf{batch}}$.

In Lemma 1, we show how to extract from a transcript tree an admissible KZG transcript tuple. In Theorem 2, we use this to establish BatchKZG's computational special soundness. Thus, BatchKZG can be used without modification in proofs in without idealized group models. This is important since BatchKZG and its variants are ubiquitous in modern updatable zk-SNARKs like Plonk [GWC19], Marlin [CHM+20], and others [CFF+21,RZ21,LSZ22].

Lemma 1. *Let $\mathcal{T} = (\mathsf{tr}_{ij})$ be an $(n+1, m)$-tree of BatchKZG's accepting transcripts, where tr_{ij} are as in Fig. 2. The DPT extractor $\mathsf{TE}_{\mathsf{batch}}(\mathsf{ck}, \mathcal{T})$ in Fig. 2 computes a tuple of accepting KZG transcripts $(\mathbf{k.tr}'_t)_{t=1}^m$, such that $\mathsf{k.tr}'_{it} = ([f_t]_1, \mathfrak{z}_i, \ldots)$, with mutually different $\mathfrak{z}_i$ for $i \in [1, n+1]$.*

Proof. Let $\mathcal{T}$ be the given accepting tree of transcripts and

$$\boldsymbol{V}_i = \begin{pmatrix} 1 & v_{i1} & v_{i1}^2 & \cdots & v_{i1}^{m-1} \\ 1 & v_{i2} & v_{i2}^2 & \cdots & v_{i2}^{m-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & v_{im} & v_{im}^2 & \cdots & v_{im}^{m-1} \end{pmatrix} \tag{2}$$

be a Vandermonde matrix. Given $\mathcal{T}$'s structure, $\mathfrak{z}_i$-s are distinct and v_{ij}-s are distinct for each $\mathfrak{z}_i$, rendering $\boldsymbol{V}_i$ non-singular for every $i \in [1, n+1]$.

Define $[h'_{i1}, \ldots, h'_{im}]_1^\intercal$ as in (*) in Fig. 2. As noted above, by the definition of BatchKZG (see Fig. 1), for any i and j, since tr_{ij} is accepted by the BatchKZG verifier, $\mathsf{k.tr}_{ij} := ([\varphi_{ij}]_1, \mathfrak{z}_i, \Phi_{ij}, [h_{ij}]_1)$ is accepted by the KZG verifier, where $\varphi_{ij} := \sum_{t=1}^m v_{ij}^{t-1} f_t$, $\Phi_{ij} := \sum_{t=1}^m v_{ij}^{t-1} \bar{f}_{ti}$, and $h_{ij} = \sum_{t=1}^m v_{ij}^{t-1} h'_{it}$. Namely,

$$\begin{bmatrix} \varphi_{i1} - \Phi_{i1} \\ \vdots \\ \varphi_{im} - \Phi_{im} \end{bmatrix}_1 \bullet [1]_2 = \boldsymbol{V}_i \begin{bmatrix} f_1 - \bar{f}_{i1} \\ \vdots \\ f_m - \bar{f}_{im} \end{bmatrix}_1 \bullet [1]_2 = \begin{bmatrix} h_{i1} \\ \vdots \\ h_{im} \end{bmatrix}_1 \bullet [x - \mathfrak{z}_i]_2 \ ,$$

for any $i \in [1, n+1]$. But then

$$\begin{bmatrix} f_1 \\ \vdots \\ f_m \end{bmatrix}_1 = \boldsymbol{V}_i^{-1} \begin{bmatrix} \varphi_{i1} \\ \vdots \\ \varphi_{im} \end{bmatrix}_1, \quad \begin{pmatrix} \bar{f}_{i1} \\ \vdots \\ \bar{f}_{im} \end{pmatrix} = \boldsymbol{V}_i^{-1} \begin{pmatrix} \Phi_{i1} \\ \vdots \\ \Phi_{im} \end{pmatrix}, \quad \begin{bmatrix} h'_{i1} \\ \vdots \\ h'_{im} \end{bmatrix}_1 := \boldsymbol{V}_i^{-1} \begin{bmatrix} h_{i1} \\ \vdots \\ h_{im} \end{bmatrix}_1 .$$

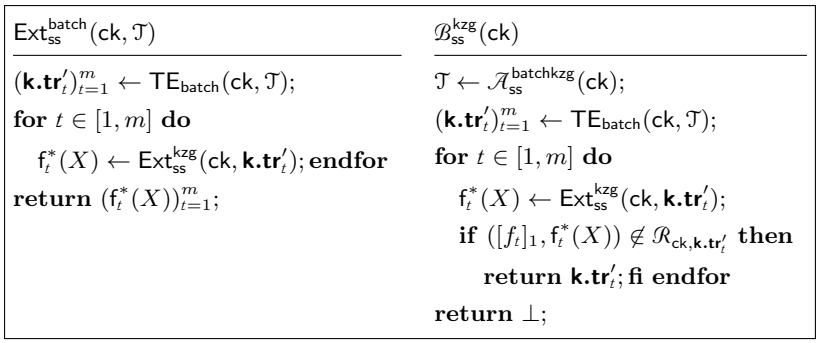

Fig. 3. The computational $\boldsymbol{\kappa}$-special-soundness extractor $\mathsf{Ext}^{\mathsf{batch}}_{\mathsf{ss}}$ and the KZG computational $(n+1)$-special-soundness adversary $\mathcal{B}^{\mathsf{kzg}}_{\mathsf{ss}}$ in Theorem 2.

KZG's triple homomorphism ensures $\mathsf{k.tr}'_{it}$ (see (**) in Fig. 2) is an accepting KZG transcript. Thus, $\mathsf{TE}_{\mathsf{batch}}$ returns accepting KZG transcripts $\mathbf{k.tr}'_t = (\mathsf{k.tr}'_{1t}, \ldots, \mathsf{k.tr}'_{n+1,t})$, for $t \in [1,m]$, of the claimed form. □

Theorem 2. *Let $n, m \in \mathsf{poly}(\lambda)$ and $\boldsymbol{\kappa} = (n+1, m)$. If KZG has computational $(n+1)$-special soundness, then BatchKZG has computational $\boldsymbol{\kappa}$-special soundness.*

Proof. Let $\mathsf{Ext}^{\mathsf{kzg}}_{\mathsf{ss}}$ be the promised computational $(n+1)$-special-soundness extractor of KZG. Let $\mathcal{A}^{\mathsf{batchkzg}}_{\mathsf{ss}}$ be any BatchKZG computational $\boldsymbol{\kappa}$-special-soundness adversary with a non-negligible success probability against any special-soundness BatchKZG extractor. We construct a computational $\boldsymbol{\kappa}$-special-soundness extractor $\mathsf{Ext}^{\mathsf{batch}}_{\mathsf{ss}}$ for BatchKZG and a computational $(n+1)$-special-soundness adversary $\mathcal{B}^{\mathsf{kzg}}_{\mathsf{ss}}$ for KZG. (Both are depicted in Fig. 3.) Here, $\mathsf{Ext}^{\mathsf{batch}}_{\mathsf{ss}}$ has an oracle access to $\mathsf{Ext}^{\mathsf{kzg}}_{\mathsf{ss}}$ and $\mathcal{B}^{\mathsf{kzg}}_{\mathsf{ss}}$ has an oracle access to $\mathcal{A}^{\mathsf{batchkzg}}_{\mathsf{ss}}$ and $\mathsf{Ext}^{\mathsf{kzg}}_{\mathsf{ss}}$. We show that the advantage of $\mathcal{B}^{\mathsf{kzg}}_{\mathsf{ss}}$ against $\mathsf{Ext}^{\mathsf{kzg}}_{\mathsf{ss}}$ is exactly equal to $\mathcal{A}^{\mathsf{batchkzg}}_{\mathsf{ss}}$'s (non-negligible) advantage against $\mathsf{Ext}^{\mathsf{batch}}_{\mathsf{ss}}$.

$\mathsf{Ext}^{\mathsf{batch}}_{\mathsf{ss}}$ inputs ck and a $\boldsymbol{\kappa}$-tree $\mathcal{T} = (\mathsf{tr}_{ij})_{i\in[1,n+1],j\in[1,m]}$ of accepting BatchKZG transcripts, where tr_{ij} is defined as in Fig. 2. $\mathsf{Ext}^{\mathsf{batch}}_{\mathsf{ss}}$ calls the deterministic extractor $\mathsf{TE}_{\mathsf{batch}}$ (see Fig. 2) to compute $n+1$ valid transcripts $\mathsf{k.tr}'_{it} = ([f_t]_1, \mathfrak{z}_i, \ldots)$ for each polynomial that has to be extracted. $\mathsf{Ext}^{\mathsf{batch}}_{\mathsf{ss}}$ calls m times the computational $(n+1)$-special-soundness extractor $\mathsf{Ext}^{\mathsf{kzg}}_{\mathsf{ss}}$ to compute the witness. Let us bound the advantage of BatchKZG's adversary $\mathcal{A}^{\mathsf{batchkzg}}_{\mathsf{ss}}$ against the extractor $\mathsf{Ext}^{\mathsf{batch}}_{\mathsf{ss}}$. Assume that $\mathcal{T}$ is an $(n+1,m)$-tree of BatchKZG transcripts, each transcript accepted by the BatchKZG verifier. Let bad be the event that for at least one t, $([f_t]_1, \mathsf{f}^*_t(X)) \notin \mathcal{R}_{\mathsf{ck},\mathbf{k.tr}'_t}$. If bad does not occur, then $\mathsf{Ext}^{\mathsf{batch}}_{\mathsf{ss}}$ has computed a valid witness for BatchKZG. Since $\mathfrak{z}_i$ are all distinct, $\mathcal{B}^{\mathsf{kzg}}_{\mathsf{ss}}$ wins the computational $(n+1)$-special-soundness game if and only if bad happened. Thus, $\mathsf{Adv}^{\mathsf{ss}}_{\mathsf{Pgen},\mathsf{KZG},\mathsf{Ext}^{\mathsf{kzg}}_{\mathsf{ss}},n+1,\mathcal{B}^{\mathsf{kzg}}_{\mathsf{ss}}}(\lambda) = \Pr[\mathsf{bad}] = \mathsf{Adv}^{\mathsf{ss}}_{\mathsf{Pgen},\mathsf{BatchKZG},\mathsf{Ext}^{\mathsf{batch}}_{\mathsf{ss}},n+1,m,\mathcal{A}^{\mathsf{batchkzg}}_{\mathsf{ss}}}(\lambda)$. □

Thus, BatchKZG has computational $\boldsymbol{\kappa}$-special soundness under the n-ARSDH assumption (see Theorem 1).

3.2 LinTrick: Linearization Trick

Definition. In many zk-SNARKs, a natural subtask[4] is for the prover to prove that $\sum_{t=1}^{n_b} \mathsf{g}_t(\mathbf{a}(X))\mathsf{d}_t(X) = 0$, where $\mathbf{g}(\boldsymbol{X}) = (\mathsf{g}_1(\boldsymbol{X}), \ldots, \mathsf{g}_{n_b}(\boldsymbol{X}))$ is a tuple of publicly known

[4] Here, we follow closely the formalism of [FFR24]. One can easily generalize this framework to allow for more general expressions.

Table 2. Comparison of different protocols for the relation $\mathcal{R}_{\mathsf{ck},\mathbf{g}}^{\mathsf{LinTrick}}$. The communication does not include the commitment costs. KS means knowledge-soundness, and CSS means computational special soundness. For convenience, $|\mathbb{G}_t|$ (resp. $|\mathbb{F}|$) denotes the numbers of bits needed to represent an element of $\mathbb{G}$ (resp. $|\mathbb{F}|$). The number of bits is given for BLS381-12.

Method	$\lvert\pi\rvert$ (bits)	KS in the AGM	KS and CSS in the standard model
Opening $\mathsf{a}_\imath, \mathsf{d}_t$ separately	$(n_a+n_b)(\lvert\mathbb{F}\rvert+\lvert\mathbb{G}_t\rvert)$ $(640(n_a+n_b))$	✓	✓
Batch-opening $a_\imath, d_t$	$(n_a+n_b)\lvert\mathbb{F}\rvert+\lvert\mathbb{G}_t\rvert$ $(256(n_a+n_b)+384)$	✓	✓
LinTrick	$n_a\lvert\mathbb{F}\rvert+\lvert\mathbb{G}_t\rvert$ $(256n_a+384)$	✓	✗
SanLinTrick (our work)	$(n_a+1)\lvert\mathbb{F}\rvert+\lvert\mathbb{G}_t\rvert$ $(256n_a+640)$	✓	✓

KGen: $x \leftarrow\$ \mathbb{F}^*$; return $\mathtt{srs} \leftarrow ([(x^t)_{t=0}^n]_1, [1,x]_2)$ and $\mathtt{td}_{\mathtt{srs}} \leftarrow x$;

$\mathsf{P}(\mathtt{srs}, \mathbb{w} = ((\mathsf{a}_\imath(X))_{\imath=1}^{n_a}, (\mathsf{d}_t(X))_{t=1}^{n_b})$:
  for $\imath \in [1,n_a]$: $[a_\imath]_1 \leftarrow [\mathsf{a}_\imath(x)]_1$; for $t \in [1,n_b]$: $[d_t]_1 \leftarrow [\mathsf{d}_t(x)]_1$;
  return $[(a_\imath)_{\imath=1}^{n_a}, (d_t)_{t=1}^{n_b}]_1$;
V: return $\mathfrak{z} \leftarrow\$ \mathbb{F}$;
P: for $\imath \in [1,n_a]$: $\bar{a}_\imath \leftarrow \mathsf{a}_\imath(\mathfrak{z})$; return $(\bar{a}_\imath)_{\imath=1}^{n_a}$;
V: return $\gamma \leftarrow\$ \mathbb{F}$;
P: return $\bar{d} \leftarrow \sum_{t=1}^{n_b} \gamma^{t-1}\mathsf{d}_t(\mathfrak{z})$;
V: return $\beta \leftarrow\$ \mathbb{F}$;
P: $\mathsf{H}(X) \leftarrow \sum_{\imath=1}^{n_a} \beta^{\imath-1}(\mathsf{a}_\imath(X) - \bar{a}_\imath) + \beta^{n_a}\sum_{t=1}^{n_b} \mathsf{g}_t(\bar{\boldsymbol{a}})\mathsf{d}_t(X) + \beta^{n_a+1}\left(\sum_{t=1}^{n_b}\gamma^{t-1}\mathsf{d}_t(X) - \bar{d}\right)$;
  $\mathsf{h}(X) \leftarrow \mathsf{H}(X)/(X-\mathfrak{z})$; return $[h]_1 \leftarrow [\mathsf{h}(x)]_1$;
V: check
  $\left[\sum_{\imath=1}^{n_a}\beta^{\imath-1}(a_\imath - \bar{a}_\imath) + \beta^{n_a}\sum_{t=1}^{n_b}\mathsf{g}_t(\bar{\boldsymbol{a}})d_t + \beta^{n_a+1}\left(\sum_{t=1}^{n_b}\gamma^{t-1}d_t - \bar{d}\right)\right]_1 \bullet [1]_2 = [h]_1 \bullet [x-\mathfrak{z}]_2$;

Fig. 4. LinTrick (without highlighted parts) and SanLinTrick (with highlighted parts).

multivariate polynomials, $\mathbf{a}(X) = (\mathsf{a}_1(X), \ldots, \mathsf{a}_{n_a}(X))$, and $\mathsf{a}_\imath(X)$ (for $\imath \in [1,n_a]$) and $\mathsf{d}_t(X)$ (for $t \in [1,n_b]$) are committed polynomials.

Example 1. In R1CS's quadratic check, one has $n_a = 1$, $n_b = 2$, $\mathsf{g}_1(\mathbf{a}(X)) = \mathsf{a}_1(X)$, and $\mathsf{g}_2(\mathbf{a}(X)) = -1$. One checks that $\mathsf{a}_1(X)\mathsf{d}_1(X) - \mathsf{d}_2(X) = 0$.

We define the underlying relation as

$$\mathcal{R}_{\mathsf{ck},\mathbf{g}}^{\mathsf{LinTrick}} = \left(\begin{array}{c} [(a_\imath)_{\imath=1}^{n_a}, (d_t)_{t=1}^{n_b}]_1, \\ ((\mathsf{a}_\imath(X))_{\imath=1}^{n_a}, (\mathsf{d}_t(X))_{t=1}^{n_b}) \end{array} \middle| \begin{array}{l} \forall \imath.([a_\imath]_1, \mathsf{a}_\imath(X)) \in \mathcal{R}_{\mathsf{ck}} \wedge \\ \forall t.([d_t]_1, \mathsf{d}_t(X)) \in \mathcal{R}_{\mathsf{ck}} \wedge \\ \sum_{t=1}^{n_b} \mathsf{g}_t(\mathbf{a}(X))\mathsf{d}_t(X) = 0 \end{array} \right) .$$

We define $\mathcal{R}_{\mathsf{ck},\mathbf{g},\mathbf{tr}}^{\mathsf{LinTrick}}$ as usual, requiring the committed polynomials to be consistent with the provided transcripts. Following [FFR24], we say $\mathsf{a}_\imath(X)$ are *left* polynomials, $\mathsf{g}_t(\mathbf{a}(X))$ are *core* polynomials, and $\mathsf{d}_t(X)$ are *right* polynomials.

For this task, several natural solutions exist. First, one can open all $n_a + n_b$ polynomials separately at a random point $\mathfrak{z}$ and test that $\sum_{t=1}^{n_b} \mathsf{g}_t(\mathbf{a}(\mathfrak{z}))\mathsf{d}_t(\mathfrak{z}) = 0$. Second, one can do the same but batch the opening proofs. Third, one can use the linearization trick, known at least from [GWC19,CHM+20]. Using the linearization trick LinTrick, one performs a single batch-opening at a random point $\mathfrak{z}$, opening (1) all $\mathsf{a}_\imath(X)$, $\imath \in [1,n_a]$, to $\bar{a}_\imath := \mathsf{a}_\imath(\mathfrak{z})$, and (2) the *linearization polynomial* $\Lambda(X) := \sum_{t=1}^{n_b} \mathsf{g}_t(\bar{\boldsymbol{a}})\mathsf{d}_t(X)$ to 0. (See Fig. 4 for a description of LinTrick.)

Insecurity of LinTrick. LinTrick is knowledge-sound in the AGM, [CHM+20] since in the AGM, one can extract all polynomials $\mathsf{a}_\imath(X)$ and $\mathsf{d}_t(X)$ from their commitments. However,

Lipmaa, Parisella, and Siim [LPS23] showed that (a simple case of) LinTrick is not knowledge-sound in the AGMOS. Faonio, Fiore, and Russo [FFR24] generalized the attack of [LPS23], which we recall in Section A.

Faonio, Fiore, and Russo [FFR24] also proved that LinTrick is (policy-based) simulation-extractable in the AGM iff $\mathsf{g}_t(\mathbf{a}(X))$ are n-independent.[5] Furthermore, they pointed out that in the case of Plonk, the core polynomials are n-independent, thus justifying the use of LinTrick in Plonk. However, this still leaves it open whether LinTrick has computational special soundness in the standard model, assuming n-independence of core polynomials.

Theorem 3. *Let $\kappa_1, \kappa_2 \in \mathsf{poly}(\lambda)$. Assuming DL holds in $\mathbb{G}_1$, LinTrick does not have computational (κ_1, κ_2)-special soundness in the standard model even when the core polynomials $\mathsf{g}_t(\mathbf{a}(X))$ are n-independent.*

Proof. Fix arbitrary polynomials $\mathsf{a}_\imath, \mathsf{d}'_t, \in \mathbb{F}_{\le n}[X]$ and $\mathsf{g}_t \in \mathbb{F}[\boldsymbol{X}]$, where $\imath \in [1, n_a]$ and $t \in [1, n_b]$, satisfying the equation $\sum_{t=1}^{n_b} \mathsf{g}_t(\mathbf{a}(X))\mathsf{d}'_t(X) = 0$. In particular, $\{\mathsf{g}_t(\mathbf{a}(X))\}$ can be n-independent, as was required in [FFR24]. Define $\mathsf{d}_t(X) \leftarrow y\mathsf{d}'_t(X)$ for a randomly sampled $y \leftarrow\!\!\$\, \mathbb{F}$; y will later act as a DL challenge. Clearly, $\sum_{t=1}^{n_b} \mathsf{g}_t(\mathbf{a}(X))\mathsf{d}_t(X) = 0$.

Assume LinTrick has computational (κ_1, κ_2)-special soundness (for *any* $\kappa_1, \kappa_2 \in \mathsf{poly}(\lambda)$) for $\mathsf{a}_\imath, \mathsf{d}_t$ and g_t as defined above. We next break DL in $\mathbb{G}_1$. Let $\mathsf{Ext}_{\mathsf{ss}}$ be a PPT extractor, existing due to LinTrick's computational special soundness. $\mathsf{Ext}_{\mathsf{ss}}$ inputs $\mathsf{ck} = ([(x^i)_{i=0}^n]_1, [1, x]_2)$ and a tree of accepting LinTrick transcripts

$$\mathsf{tr}_{ij} = ([(a_\imath)_{\imath=1}^{n_a}, (d_t)_{t=1}^{n_b}]_1, \mathfrak{z}_i, (\bar{a}_{\imath i})_{\imath=1}^{n_a}, \beta_{ij}, [h_{ij}]_1)$$

for $(i, j) \in [1, \kappa_1] \times [1, \kappa_2]$. Thus, for all i and j, $[\sum_{\imath=1}^{n_a} \beta_{ij}^{\imath-1}(a_\imath - \bar{a}_{\imath i}) + \beta_{ij}^{n_a} \sum_{t=1}^{n_b} \mathsf{g}_t(\bar{\boldsymbol{a}}_i)d_t]_1 \bullet [1]_2 = [h_{ij}]_1 \bullet [x - \mathfrak{z}_i]_2$. For any computational special-soundness adversary $\mathcal{A}_{\mathsf{ss}}^{\mathsf{lintrick}}$, $\mathsf{Ext}_{\mathsf{ss}}$ outputs $((\mathsf{a}_\imath(X))_{\imath=1}^{n_a}, (\mathsf{d}_t(X))_{t=1}^{n_b}) \notin \mathcal{R}_{\mathsf{ck},\mathbf{g},\mathbf{tr}}^{\mathsf{LinTrick}}$ (i.e., $\mathsf{Ext}_{\mathsf{ss}}$ does not succeed) with a probability $\le \varepsilon$. Next, we construct a DL adversary $\mathcal{B}_{\mathsf{dl}}$, see Fig. 5, that internally uses $\mathsf{Ext}_{\mathsf{ss}}$ to break DL with probability $\ge 1 - \varepsilon$. Thus, ε cannot be negligible, and thus LinTrick cannot have computational special soundness.

$\mathcal{B}_{\mathsf{dl}}$ gets as an input a DL challenge $[y]_1$ for $y \leftarrow\!\!\$\, \mathbb{F}$. $\mathcal{B}_{\mathsf{dl}}$ samples x and generates ck from x. Using the knowledge of x, $\mathcal{B}_{\mathsf{dl}}$ computes commitments $[a_\imath]_1 \leftarrow [\mathsf{a}_\imath(x)]_1$ and $[d_t]_1 \leftarrow \mathsf{d}'_t(x)[y]_1$. $\mathcal{B}_{\mathsf{dl}}$ then generates a transcript tree $\mathbf{tr} = (\mathsf{tr}_{ij})$, expected by $\mathsf{Ext}_{\mathsf{ss}}$. More precisely, $\mathcal{B}_{\mathsf{dl}}$ sets $\mathfrak{z}_i = i$ and $\beta_{ij} = j$ (it suffices that $\{\mathfrak{z}_i\}_i$ are pair-wise distinct and for any i, $\{\beta_{ij}\}_j$ are pair-wise distinct), with the opening proofs $[h_{ij}]_1$ as on line 8. After that, $\mathcal{B}_{\mathsf{dl}}$ runs $\mathsf{Ext}_{\mathsf{ss}}(\mathsf{p}, \mathsf{ck}, \mathbf{tr})$ to obtain (some) polynomials $((\widetilde{\mathsf{a}}_\imath(X))_{\imath=1}^{n_a}, (\widetilde{\mathsf{d}}_t(X))_{t=1}^{n_b})$ and outputs $y^* \leftarrow \widetilde{\mathsf{d}}_1(x)/\mathsf{d}'_1(x)$ as a candidate for the DL y.

Suppose $\mathcal{B}_{\mathsf{dl}}$ breaks DL with probability $\varepsilon_{\mathcal{B}_{\mathsf{dl}}}$. We present two games to show that $\varepsilon_{\mathcal{B}_{\mathsf{dl}}} \ge 1 - \varepsilon$, which implies that ε cannot be negligible if the DL assumption holds. The two games, Game_1 and Game_2, use $\mathcal{B}_{\mathsf{dl}}$ and $\mathcal{A}_{\mathsf{ss}}^{\mathsf{lintrick}}$ from Fig. 5. Intuitively, there are two trapdoors, x and y, and we use a trapdoor switching trick. The games differ in which trapdoor they provide to the internal adversaries, who use the corresponding trapdoor to perform otherwise similar computations. Given one trapdoor, each adversary's goal is to compute something related to the other trapdoor, thereby either breaking the DL assumption or the computational special soundness of LinTrick.

Game_1: The first game in Fig. 6 samples a discrete logarithm challenge $[y]_1$ and then calls $\mathcal{B}_{\mathsf{dl}}(\mathsf{p}, [y]_1)$ who returns a claimed discrete logarithm y^*. Game_1 outputs 1 when $y^*[1]_1 = [y]_1$ ($\mathcal{B}_{\mathsf{dl}}$ succeeded). Thus, the probability that Game_1 outputs 1 is $\varepsilon_{\mathcal{B}_{\mathsf{dl}}}$, the probability that $\mathcal{B}_{\mathsf{dl}}$ succeeds in the DL game.

[5] Polynomials $b_t(X)$ are n-independent [FFR24], if from $\sum_t b_t(X)c_t(X) = 0$, where $c_t(X) \in \mathbb{F}_{\le n}[X]$, it follows that $c_t(X) = 0$ for all t.

$\mathcal{B}_{\mathsf{dl}}(\mathsf{p}, [y]_1)$:

1 : $x \leftarrow\!\$\, \mathbb{F}; \mathsf{ck} \leftarrow ([(x^i)_{i=0}^n]_1, [1, x]_2);$
2 : **for** $\jmath \in [1, n_a]$ **do** $[a_\jmath]_1 \leftarrow [\mathsf{a}_\jmath(x)]_1$; **endfor**
3 : **for** $t \in [1, n_b]$ **do** $[d_t]_1 \leftarrow [\mathsf{d}'_t(x)][y]_1$; **endfor**
4 : **for** $i \in [1, \kappa_1]$ **do**
5 : $\mathfrak{z}_i \leftarrow i$; **for** $\jmath \in [1, n_a]$ **do** $\bar{a}_{\jmath i} \leftarrow \mathsf{a}_\jmath(\mathfrak{z}_i)$; **endfor**
6 : **for** $j \in [1, \kappa_2]$ **do**
7 : $\beta_{ij} \leftarrow j$;
8 : $[h_{ij}]_1 \leftarrow (\sum_{\jmath=1}^{n_a} \beta_{ij}^{\jmath-1} ([a_\jmath]_1 - \bar{a}_{\jmath i}[1]_1) + \beta_{ij}^{n_a} \sum_{t=1}^{n_b} \mathsf{g}_t(\bar{\mathbf{a}}_i)[d_t]_1)/(x - \mathfrak{z}_i)$;
9 : $\mathsf{tr}_{ij} \leftarrow ([(a_\jmath)_{\jmath=1}^{n_a}, (d_t)_{t=1}^{n_b}]_1, \mathfrak{z}_i, (\bar{a}_{\jmath i})_{\jmath=1}^{n_a}, \beta_{ij}, [h_{ij}]_1)$; **endfor endfor**
10 : $((\widetilde{\mathsf{a}}_\jmath(X))_{\jmath=1}^{n_a}, (\widetilde{\mathsf{d}}_t(X))_{t=1}^{n_b}) \leftarrow \mathsf{Ext}_{\mathsf{ss}}(\mathsf{p}, \mathsf{ck}, \mathbf{tr})$;
11 : **return** $y^* \leftarrow \widetilde{\mathsf{d}}_1(x)/\mathsf{d}'_1(x)$;

$\mathcal{A}_{\mathsf{ss}}^{\mathsf{lintrick}}(\mathsf{p}, \mathsf{ck})$:

1 : $y \leftarrow\!\$\, \mathbb{F}$;
2 : **for** $\jmath \in [1, n_a]$ **do** $[a_\jmath]_1 \leftarrow [\mathsf{a}_\jmath(x)]_1$; **endfor**
3 : **for** $t \in [1, n_b]$ **do** $[d_t]_1 \leftarrow y[\mathsf{d}'_t(x)]_1$; **endfor**
4 : **for** $i \in [1, \kappa_1]$ **do**
5 : $\mathfrak{z}_i \leftarrow i$; **for** $\jmath \in [1, n_a]$ **do** $\bar{a}_{\jmath i} \leftarrow \mathsf{a}_\jmath(\mathfrak{z}_i)$; **endfor**
6 : **for** $j \in [1, \kappa_2]$ **do**
7 : $\beta_{ij} \leftarrow j$;
8 : $\mathsf{h}_{ij}(X) \leftarrow (\sum_{\jmath=1}^{n_a} \beta_{ij}^{\jmath-1} (\mathsf{a}_\jmath(X) - \bar{a}_{\jmath i}) + \beta_{ij}^{n_a} \sum_{t=1}^{n_b} \mathsf{g}_t(\bar{\mathbf{a}}_i) y \mathsf{d}'_t(X))/(X - \mathfrak{z}_i)$;
9 : $[h_{ij}]_1 \leftarrow [\mathsf{h}_{ij}(x)]_1$;
10 : $\mathsf{tr}_{ij} \leftarrow ([((a_\jmath)_{\jmath=1}^{n_a}, (d_t)_{t=1}^{n_b})]_1, \mathfrak{z}_i, (\bar{a}_{\jmath i})_{\jmath=1}^{n_a}, \beta_{ij}, [h_{ij}]_1)$; **endfor endfor**
11 : **return** tr;

Fig. 5. DL adversary $\mathcal{B}_{\mathsf{dl}}$ and computational special-soundness adversary $\mathcal{A}_{\mathsf{ss}}^{\mathsf{lintrick}}$. We highlight the differences between two adversaries by using colorful, boxes.

Game_1 :	Game_2 :
$y \leftarrow\!\$\, \mathbb{F}$;	$x \leftarrow\!\$\, \mathbb{F}; \mathsf{ck} \leftarrow ([(x^i)_{i=0}^n]_1, [1, x]_2)$;
$y^* \leftarrow \mathcal{B}_{\mathsf{dl}}(\mathsf{p}, [y]_1)$;	$\mathbf{tr} \leftarrow \mathcal{A}_{\mathsf{ss}}^{\mathsf{lintrick}}(\mathsf{p}, \mathsf{ck})$; // $\mathcal{A}_{\mathsf{ss}}^{\mathsf{lintrick}}$ from Fig. 5
return $y^*[1]_1 \stackrel{?}{=} [y]_1$;	$((\widetilde{\mathsf{a}}_\jmath(X))_{\jmath=1}^{n_a}, (\widetilde{\mathsf{d}}_t(X))_{t=1}^{n_b}) \leftarrow \mathsf{Ext}_{\mathsf{ss}}(\mathsf{p}, \mathsf{ck}, \mathbf{tr})$;
	Let $[y]_1$ be as defined inside $\mathcal{A}_{\mathsf{ss}}^{\mathsf{lintrick}}$;
	return $(\widetilde{\mathsf{d}}_1(x)/\mathsf{d}'_1(x))[1]_1 \stackrel{?}{=} [y]_1$;

Fig. 6. Games in the proof of Theorem 3.

Game_2: The second game in Fig. 6 starts by sampling x and ck. After that, it calls $\mathcal{A}_{\mathsf{ss}}^{\mathsf{lintrick}}$ from Fig. 5. $\mathcal{A}_{\mathsf{ss}}^{\mathsf{lintrick}}$ uses the knowledge of y (instead of x, as done by Game_1) to simulate transcripts, distributed identically to Game_1. On step 3, $\mathcal{A}_{\mathsf{ss}}^{\mathsf{lintrick}}$ computes $[d_t]_1 \leftarrow y[\mathsf{d}'_t(x)]_1$. Since $\sum_{t=1}^{n_b} \mathsf{g}_t(\mathbf{a}(\mathfrak{z}_i))\mathsf{d}_t(\mathfrak{z}_i) = 0$ for any $\mathfrak{z}_i$, $(X - \mathfrak{z}_i) \mid \sum_{t=1}^{n_b} \mathsf{g}_t(\mathbf{a}(\mathfrak{z}_i))\mathsf{d}_t(X)$. Thus, $\mathsf{h}_{ij}(X)$ is a polynomial and $\mathcal{A}_{\mathsf{ss}}^{\mathsf{lintrick}}$ can compute $[\mathsf{h}_{ij}(x)]_1$ using only ck (and not x). In fact, Game_2 only

uses the knowledge of x on the first and last steps. Since inputs to $\mathsf{Ext}_{\mathsf{ss}}$ are identically distributed to Game_1, the probability that Game_2 outputs 1 is also $\varepsilon_{\mathcal{B}_{\mathsf{dl}}}$.

Next, we use $\mathcal{A}_{\mathsf{ss}}^{\mathsf{lintrick}}$ from Fig. 5 as a computational special-soundness adversary. Notably, $\mathcal{A}_{\mathsf{ss}}^{\mathsf{lintrick}}$ does not require the knowledge of x. Clearly, $\Pr[\mathsf{Game}_2 \neq 1]$ is bounded by the advantage of $\mathcal{A}_{\mathsf{ss}}^{\mathsf{lintrick}}$ in the computational special-soundness game. Namely, $\mathsf{Game}_2 \neq 1$ happens only if $\widetilde{\mathsf{d}}_1(x) \neq d_1$. If this is the case, then $((\widetilde{\mathsf{a}}_\imath(X))_{\imath=1}^{n_a}, (\widetilde{\mathsf{d}}_t(X))_{t=1}^{n_b}) \notin \mathcal{R}_{\mathsf{ck},\mathbf{g}}^{\mathsf{LinTrick}}$, meaning that $\mathcal{A}_{\mathsf{ss}}^{\mathsf{lintrick}}$ wins the computational special-soundness game. This happens at most with probability ε as we assumed $\mathsf{Ext}_{\mathsf{ss}}$ to fail at most with probability ε. Thus, the probability that $\widetilde{\mathsf{d}}_1(x)/\mathsf{d}'_1(x) = y$ in Game_2 must be at least $1-\varepsilon$. We conclude that $\varepsilon_{\mathcal{B}_{\mathsf{dl}}} \geq 1-\varepsilon$, which implies that if ε is negligible (and thus the computational special soundness holds), then we break the DL assumption. □

3.3 SanLinTrick: Sanitized LinTrick

The attack of [LPS23,FFR24] is possible since LinTrick's prover never opens any polynomials $\mathsf{d}_t(X)$, allowing it to "obliviously sample" $[d_t]_1$. However, $[a_\imath]_1$ (that *are* opened) cannot be sampled obliviously. Next, we counter this attack introducing a technique called *sanitization* that involves batch-opening all polynomial commitments that were not batch-opened otherwise. In SanLinTrick (*sanitized LinTrick*), this means batch-opening all polynomials $\mathsf{d}_t(X)$ at a random point. Since we do not need the actual evaluations $\mathsf{d}_t(\mathfrak{z})$, it suffices for the prover to send $\bar{d} \leftarrow \sum_{t=1}^{n_b} \gamma^{t-1}\mathsf{d}_t(\mathfrak{z})$, for a new batching coefficient γ, adding a single field element $\bar{d}$ (and one round) to LinTrick's communication. We depict SanLinTrick in Fig. 4 and compare it to (less efficient or secure) protocols in Table 2.

Theorem 4. *Let $n, n_a, n_b \in \mathsf{poly}(\lambda)$, $n_{\mathsf{g}} := \max_t \deg(\mathsf{g}_t(\mathbf{a}(X))) \in \mathsf{poly}(\lambda)$, $\kappa_1 := \max(n_{\mathsf{g}}, n)+1$, and $\boldsymbol{\kappa} = (\kappa_1, n_b, n_a+2)$. If KZG for degree $\leq n$ polynomials has computational $(n+1)$-special soundness and evaluation binding, then SanLinTrick has computational $\boldsymbol{\kappa}$-special soundness.*

Before proving Theorem 4, we state the following lemma.

Lemma 2. *Assume that $\mathcal{T} = (\mathsf{tr}_{ijk})$ is a $\boldsymbol{\kappa}$-tree of SanLinTrick's accepting transcripts, where tr_{ijk} are as in step 2 in Fig. 7. The PPT algorithm $\mathsf{TE}_{\mathsf{lin}}(\mathsf{ck}, \mathcal{T})$ in Fig. 7 computes accepting KZG transcripts $(\mathbf{tr}_{a_\imath})_{\imath=1}^{n_a}$ and $(\mathbf{tr}_{d_t})_{t=1}^{n_b}$, s.t. $\mathsf{tr}_{a_\imath,i} = ([a_\imath]_1, \mathfrak{z}_i, \ldots)$ and $\mathsf{tr}_{d_t,i} = ([d_t]_1, \mathfrak{z}_i, \ldots)$ for $i \in [1, n+1]$, and $\mathfrak{z}_i$ are mutually distinct.*

Proof. Let $\mathcal{T}$ be an accepting transcript tree. For $i \in [1, \kappa_1]$ and $j \in [1, n_b]$, let

$$\boldsymbol{C}_i = \begin{pmatrix} 1 & \gamma_{i1} & \gamma_{i1}^2 & \vdots & \gamma_{i1}^{n_b-1} \\ 1 & \gamma_{i2} & \gamma_{i2}^2 & \vdots & \gamma_{i2}^{n_b-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \gamma_{in_b} & \gamma_{in_b}^2 & \vdots & \gamma_{in_b}^{n_b-1} \end{pmatrix} \quad \text{and} \quad \boldsymbol{B}_{ij} = \begin{pmatrix} 1 & \beta_{ij1} & \cdots & \beta_{ij1}^{n_a+1} \\ 1 & \beta_{ij2} & \cdots & \beta_{ij2}^{n_a+1} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & \beta_{ij,n_a+2} & \cdots & \beta_{ij,n_a+2}^{n_a+1} \end{pmatrix} \tag{3}$$

be Vandemonde matrices. Since $\mathcal{T}$ is a tree of transcripts, $\mathfrak{z}_i$ are all distinct, and for each $\mathfrak{z}_i$, all γ_{ij} are disctinct, and for each γ_{ij}, all β_{ijk} are distinct. Thus, for each $i \in [1, \kappa_1]$ and $j \in [1, n_b]$, $\boldsymbol{C}_i$ and $\boldsymbol{B}_{ij}$ are non-singular.

By the construction of SanLinTrick, tr_{ijk} is an accepting SanLinTrick transcript iff

$$\mathsf{k}.\mathsf{tr}_{ijk} = ([\varphi_{ijk}]_1, \mathfrak{z}_i, \Phi_{ijk}, [h_{ijk}]_1)$$

is an accepting KZG transcript, where

$$\varphi_{ijk} := \sum_{\imath=1}^{n_a} \beta_{ijk}^{\imath-1} a_\imath + \beta_{ijk}^{n_a} \sum_{t=1}^{n_b} \mathsf{g}_t(\bar{\mathbf{a}}_i) d_t + \beta_{ijk}^{n_a+1} \sum_{t=1}^{n_b} \gamma_{ij}^{t-1} d_t$$

$\mathsf{TE}_{\mathsf{lin}}(\mathsf{ck}, \mathcal{T})$

1 : Parse $\mathcal{T} = (\mathsf{tr}_{ijk})_{i\in[1,\kappa_1],j\in[1,n_b],k\in[1,n_a+2]}$;
2 : Parse $\mathsf{tr}_{ijk} = ([(a_\imath)_{\imath=1}^{n_a}, (d_t)_{t=1}^{n_b}]_1, \mathfrak{z}_i, (\bar{a}_{it})_{t=1}^{n_b}, \gamma_{ij}, \bar{d}_{ij}, \beta_{ijk}, [h_{ijk}]_1)$;
3 : **for** $i \in [1, \kappa_1]$ **do for** $j \in [1, n_b]$ **do**
4 : $\quad [h^*_{ij1}, \ldots, h^*_{ij,n_a+2}]_1^\intercal \leftarrow \boldsymbol{B}_{ij}^{-1}[h_{ij1}, \ldots, h_{ij,n_a+2}]_1^\intercal$;
5 : $\quad$ **for** $\imath \in [1, n_a]$ **do** $\mathsf{k.tr}^*_{ij\imath} \leftarrow ([a_\imath]_1, \mathfrak{z}_i, \bar{a}_{i\imath}, [h^*_{ij\imath}]_1)$;
6 : $\quad \mathsf{k.tr}^*_{ij,n_a+1} \leftarrow ([\sum_{t=1}^{n_b} \mathsf{g}_t(\bar{\boldsymbol{a}}_i)d_t]_1, \mathfrak{z}_i, 0, [h^*_{ij,n_a+1}]_1)$;
7 : $\quad \mathsf{k.tr}^*_{ij,n_a+2} \leftarrow ([\sum_{t=1}^{n_b} \gamma_{ij}^{t-1} d_t]_1, \mathfrak{z}_i, \bar{d}_{ij}, [h^*_{ij,n_a+2}]_1)$;
8 : **for** $\imath \in [1, n_a]$ **do**
9 : $\quad$ **for** $i \in [1, \kappa_1]$ **do** $\mathsf{k.tr}_{a_\imath,i} \leftarrow \mathsf{k.tr}^*_{i1\imath}$;
10 : $\quad \mathbf{tr}_{a_\imath} \leftarrow (\mathsf{k.tr}_{a_\imath,1}, \ldots, \mathsf{k.tr}_{a_\imath,n+1})$;
11 : **for** $i \in [1, \kappa_1]$ **do**
12 : $\quad \begin{pmatrix} \bar{d}'_{i1} \\ \cdots \\ \bar{d}'_{in_b} \end{pmatrix} \leftarrow \boldsymbol{C}_i^{-1} \begin{pmatrix} \bar{d}_{i1} \\ \cdots \\ \bar{d}_{in_b} \end{pmatrix}; \begin{bmatrix} h'_{i1} \\ \cdots \\ h'_{in_b} \end{bmatrix}_1 \leftarrow \boldsymbol{C}_i^{-1} \begin{bmatrix} h^*_{i1,n_a+2} \\ \cdots \\ h^*_{in_b,n_a+2} \end{bmatrix}_1$;
13 : $\quad$ **for** $t \in [1, n_b]$ **do**
14 : $\quad\quad \mathsf{k.tr}_{d_t,i} \leftarrow ([d_t]_1, \mathfrak{z}_i, \bar{d}'_{it}, [h'_{it}]_1)$;
15 : $\quad \mathbf{tr}_{d_t} \leftarrow (\mathsf{k.tr}_{d_t,1}, \ldots, \mathsf{k.tr}_{d_t,n+1})$;
16 : **return** $((\mathbf{tr}_{a_\imath})_{\imath=1}^{n_a}, (\mathbf{tr}_{d_t})_{t=1}^{n_b})$;

Fig. 7. The $\mathsf{TE}_{\mathsf{lin}}$ subroutine.

and

$$\Phi_{ijk} := \sum_{\imath=1}^{n_a} \beta_{ijk}^{\imath-1} \bar{a}_{i\imath} + \beta_{ijk}^{n_a+1} \bar{d}_{ij} \ .$$

Thus,

$$\left(a_1, \ldots, a_{n_a}, \sum_{t=1}^{n_b} \mathsf{g}_t(\bar{\boldsymbol{a}}_i)d_t, \sum_{t=1}^{n_b} \gamma_{ij}^{t-1} d_t\right)^\intercal = \boldsymbol{B}_{ij}^{-1} \cdot (\varphi_{ij1}, \ldots, \varphi_{ij,n_a+2})^\intercal \ ,$$
$$(\bar{a}_{i1}, \ldots, \bar{a}_{in_a}, 0, \bar{d}_{ij})^\intercal = \boldsymbol{B}_{ij}^{-1} \cdot (\Phi_{ij1}, \ldots, \Phi_{ij,n_a+2})^\intercal \ .$$

Since KZG is triply homomorphic, $\mathsf{k.tr}^*_{ij\imath}$ (see Steps 5, 6, 7 in Fig. 7) are accepting KZG transcripts for all $i, j, \imath$. Thus, one can define $\mathsf{tr}_{a_\imath,i} = \mathsf{k.tr}^*_{ij\imath}$ for $j = 1$ (one can choose any value of j).

Let $h^*_{i1,n_a+2}, \ldots, h^*_{in_b,n_a+2}$ be as in Step 4 of Fig. 7. Define $(\bar{d}'_{i1}, \ldots, \bar{d}'_{in_b})^\intercal \leftarrow \boldsymbol{C}_i^{-1} \cdot (\bar{d}_{i1}, \ldots, \bar{d}_{in_b})^\intercal$ and $[h'_{i1}, \ldots, h'_{in_b}]_1^\intercal \leftarrow \boldsymbol{C}_i^{-1} \cdot [h^*_{i1,n_a+2}, \ldots, h^*_{in_b,n_a+2}]_1^\intercal$. Clearly, the vector of first elements $[\sum_{t=1}^{n_b} \gamma_{ij}^{t-1} d_t]_1$ of $\mathsf{k.tr}^*_{ij,n_a+2}$ is equal to $\boldsymbol{C}_i \cdot [d_1, \ldots, d_{n_b}]_1^\intercal$. Since KZG is triple homomorphic and $\mathsf{k.tr}^*_{ij,n_a+2}$ are accepting, $\mathsf{k.tr}_{d_t i} \leftarrow ([d_t]_1, \mathfrak{z}_i, \bar{d}'_{it}, [h'_{it}]_1)$ is an accepting KZG transcript. □

Next, we prove Theorem 4.

Proof (Theorem 4). Recall that a valid witness contains $n_a + n_b$ polynomials $(\mathsf{a}^*_\imath(X))_{\imath=1}^{n_a}$ and $(\mathsf{d}^*_t(X))_{t=1}^{n_b}$ of degree at most n, such that $\sum_{t=1}^{n_b} \mathsf{g}_t(\mathbf{a}^*(X))\mathsf{d}^*_t(X) = 0$ and $\mathsf{a}^*_\imath(x) = a_\imath$, $\mathsf{d}^*_t(x) = d_t$ for $\imath \in [1, n_a]$, $t \in [1, n_b]$.

$\mathsf{Ext}_{\mathsf{ss}}^{\mathsf{sanlintrick}}(\mathsf{ck}, \mathcal{T})$

$((\mathbf{tr}_{a_\imath})_{\imath=1}^{n_a}, (\mathbf{tr}_{d_t})_{t=1}^{n_b}) \leftarrow \mathsf{TE}_{\mathsf{lin}}(\mathsf{ck}, \mathcal{T})$;
for $\imath \in [1, n_a]$ **do**
  $\mathsf{a}_\imath^*(X) \leftarrow \mathsf{Ext}_{\mathsf{ss}}^{\mathsf{kzg}}(\mathsf{ck}, \mathbf{tr}_{a_\imath})$;
for $t \in [1, n_b]$ **do**
  $\mathsf{d}_t^*(X) \leftarrow \mathsf{Ext}_{\mathsf{ss}}^{\mathsf{kzg}}(\mathsf{ck}, \mathbf{tr}_{d_t})$;
return $((\mathsf{a}_\imath^*(X))_{\imath=1}^{n_a}, (\mathsf{d}_t^*(X))_{t=1}^{n_b})$;

$\mathcal{B}_{\mathsf{ss}}^{\mathsf{kzg}}(\mathsf{ck})$

$\mathcal{T} \leftarrow \mathcal{A}(\mathsf{ck})$;
$((\mathbf{tr}_{a_\imath})_{\imath=1}^{n_a}, (\mathbf{tr}_{d_t})_{t=1}^{n_b}) \leftarrow \mathsf{TE}_{\mathsf{lin}}(\mathsf{ck}, \mathcal{T})$;
for $\imath \in [1, n_a]$ **do**
  $\mathsf{a}_\imath^*(X) \leftarrow \mathsf{Ext}_{\mathsf{ss}}^{\mathsf{kzg}}(\mathsf{ck}, \mathbf{tr}_{a_\imath})$;
  if $([a_\imath]_1, \mathsf{a}_\imath^*(X)) \notin \mathcal{R}_{\mathsf{ck}, \mathbf{tr}_{a_\imath}}$ **then return** $\mathbf{tr}_{a_\imath}$;
for $t \in [1, n_b]$ **do**
  $\mathsf{d}_t^*(X) \leftarrow \mathsf{Ext}_{\mathsf{ss}}^{\mathsf{kzg}}(\mathsf{ck}, \mathbf{tr}_{d_t})$;
  if $([d_t]_1, \mathsf{d}_t^*(X)) \notin \mathcal{R}_{\mathsf{ck}, \mathbf{tr}_{d_t}}$ **then return** $\mathbf{tr}_{d_t}$;
return $\bot$;

$\mathcal{C}_{\mathsf{evb}}(\mathsf{ck})$

$\mathcal{T} \leftarrow \mathcal{A}(\mathsf{ck})$; $((\mathbf{tr}_{a_\imath})_{\imath=1}^{n_a}, (\mathbf{tr}_{d_t})_{t=1}^{n_b}) \leftarrow \mathsf{TE}_{\mathsf{lin}}(\mathsf{ck}, \mathcal{T})$;
for $\imath \in [1, n_a]$ **do** $\mathsf{a}_\imath^*(X) \leftarrow \mathsf{Ext}_{\mathsf{ss}}^{\mathsf{kzg}}(\mathsf{ck}, \mathbf{tr}_{a_\imath})$;
for $t \in [1, n_b]$ **do** $\mathsf{d}_t^*(X) \leftarrow \mathsf{Ext}_{\mathsf{ss}}^{\mathsf{kzg}}(\mathsf{ck}, \mathbf{tr}_{d_t})$;
Parse $((\mathbf{tr}_{a_\imath})_{\imath=1}^{n_a}, (\mathbf{tr}_{d_t})_{t=1}^{n_b})$ as in Fig. 7;
for $i \in [n+2, \kappa_1]$ **do**
  for $\imath \in [1, n_a]$ **do**
    if $\bar{a}_{i\imath} \neq \mathsf{a}_\imath^*(\mathfrak{z}_i)$ **then return** $\left([a_\imath]_1, \mathfrak{z}_i, \bar{a}_{i\imath}, [h_{i1\imath}^*]_1, \mathsf{a}_\imath^*(\mathfrak{z}_i), \left[\frac{\mathsf{a}_\imath^*(x) - \mathsf{a}_\imath^*(\mathfrak{z}_i)}{x - \mathfrak{z}_i}\right]_1\right)$;
  for $t \in [1, n_b]$ **do**
    if $\bar{d}'_{it} \neq \mathsf{d}_t^*(\mathfrak{z}_i)$ **then return** $\left([d_t]_1, \mathfrak{z}_i, \bar{d}'_{it}, [h'_{it}]_1, \mathsf{d}_t^*(\mathfrak{z}_i), \left[\frac{\mathsf{d}_t^*(x) - \mathsf{d}_t^*(\mathfrak{z}_i)}{x - \mathfrak{z}_i}\right]_1\right)$;
for $i \in [1, \kappa_1]$ **do**
  if $\sum_{t=1}^{n_b} \mathsf{g}_t(\bar{\boldsymbol{a}}_{\boldsymbol{i}}) \bar{d}'_{it} \neq 0$ **then**
    return $\left(\left[\sum_{t=1}^{n_b} \mathsf{g}_t(\bar{\boldsymbol{a}}_{\boldsymbol{i}}) d_t\right]_1, \mathfrak{z}_i, \sum_{t=1}^{n_b} \mathsf{g}_t(\bar{\boldsymbol{a}}_{\boldsymbol{i}}) \bar{d}'_{it}, \left[\sum_{t=1}^{n_b} \mathsf{g}_t(\bar{\boldsymbol{a}}_{\boldsymbol{i}}) h'_{it}\right]_1, 0, [h_{i1,n_a+1}^*]_1\right)$;
return $\bot$;

Fig. 8. The extractor $\mathsf{Ext}_{\mathsf{ss}}^{\mathsf{sanlintrick}}$, the KZG computational special-soundness adversary $\mathcal{B}_{\mathsf{ss}}^{\mathsf{kzg}}$, and the KZG evaluation-binding adversary $\mathcal{C}_{\mathsf{evb}}$.

Recall $\boldsymbol{\kappa} = (\kappa_1, n_b, n_a + 2)$. Let $\mathsf{Ext}_{\mathsf{ss}}^{\mathsf{kzg}}$ be a computational $(n+1)$-special-soundness extractor $\mathsf{Ext}_{\mathsf{ss}}^{\mathsf{kzg}}$ of KZG. We depict the computational $\boldsymbol{\kappa}$-special-soundness extractor $\mathsf{Ext}_{\mathsf{ss}}^{\mathsf{sanlintrick}}$ for SanLinTrick in Fig. 8. It has blackbox access to $\mathsf{Ext}_{\mathsf{ss}}^{\mathsf{kzg}}$. On input a $\boldsymbol{\kappa}$-tree of SanLinTrick accepting transcripts, $\mathsf{Ext}_{\mathsf{ss}}^{\mathsf{sanlintrick}}$ calls $\mathsf{TE}_{\mathsf{lin}}$ from Fig. 7 that returns accepting KZG transcripts $\mathbf{tr}_{a_\imath}, \mathbf{tr}_{d_t}$ for $\imath \in [1, n_a]$, $t \in [1, n_b]$. As stated in Lemma 2, each $\mathsf{tr}_{a_\imath i}$ (resp., $\mathsf{tr}_{d_t i}$) contains a transcript for the commitment $[a_\imath]_1$ (resp., $[d_t]_1$) with a distinct evaluation point $\mathfrak{z}_i$. We feed them separately into KZG's computational $(n+1)$-special-soundness extractor $\mathsf{Ext}_{\mathsf{ss}}^{\mathsf{kzg}}$ to extract all $\mathsf{a}_\imath^*(X)$ and $\mathsf{d}_t^*(X)$.

Next, we show that $\varepsilon = \mathsf{Adv}_{\mathsf{Pgen}, \mathsf{SanLinTrick}, \mathsf{Ext}_{\mathsf{ss}}^{\mathsf{sanlintrick}}, \boldsymbol{\kappa}, \mathcal{A}}^{\mathsf{ss}}(\lambda)$ (see Definition 2) is negligible for any PPT adversary $\mathcal{A}$. That is, if $\mathcal{A}$ outputs an accepting tree of transcripts, then $\mathsf{Ext}_{\mathsf{ss}}^{\mathsf{sanlintrick}}$ fails to extract $(\mathsf{a}_\imath^*(X))_{\imath=1}^{n_a}, (\mathsf{d}_t^*(X))_{t=1}^{n_b}$, such that

$$\left(\left(([a_\imath]_1, \bar{a}_\imath)_{\imath=1}^{n_a}, ([d_t]_1, \bar{d}_t)_{t=1}^{n_b}\right), ((\mathsf{a}_\imath^*(X))_{\imath=1}^{n_a}, (\mathsf{d}_t^*(X))_{t=1}^{n_b})\right) \in \mathcal{R}_{\mathsf{ck}, \mathbf{g}, \mathbf{tr}}^{\mathsf{LinTrick}} \quad ,$$

(for naturally defined $\mathbf{tr}$) with probability $\varepsilon = \mathsf{negl}(\lambda)$.

We consider the following two failure events for $\mathsf{Ext}_{\mathsf{ss}}^{\mathsf{sanlintrick}}$:

1. The event $\mathsf{bad}_{\mathsf{ext}}$ happens when $([a_\imath]_1, \mathsf{a}^*_\imath(X)) \notin \mathcal{R}_{\mathsf{ck},\mathbf{tr}_{a_\imath}}$ or $([d_t]_1, \mathsf{d}^*_t(X)) \notin \mathcal{R}_{\mathsf{ck},\mathbf{tr}_{d_t}}$ for some $\imath \in [1, n_a]$, $t \in [1, n_b]$.
2. The event $\mathsf{bad}_{\mathsf{evb}}$ happens when $\mathsf{bad}_{\mathsf{ext}}$ did not happen, but one of the following conditions hold:
   (a) For any $\imath \in [1, n_a]$, $t \in [1, n_b]$, either $\mathsf{a}^*_\imath(\mathfrak{z}_i) \neq \bar{a}_{i\imath}$ or $\mathsf{d}^*_t(\mathfrak{z}_i) \neq \bar{d}'_{it}$ for some $i \in [n+2, \kappa_1]$.
   (b) $\sum_{t=1}^{n_b} \mathsf{g}_t(\bar{\boldsymbol{a}}_i)\bar{d}'_{it} \neq 0$ for any $i \in [1, \kappa_1]$.

Consider the scenario where neither $\mathsf{bad}_{\mathsf{ext}}$ nor $\mathsf{bad}_{\mathsf{evb}}$ occurs. Then, we have extracted polynomials $\mathsf{a}^*_\imath(X)$ and $\mathsf{d}^*_t(X)$ for $\imath \in [1, n_a]$, $t \in [1, n_b]$ of degree at most n which are consistent with the commitments. Denote $\mathsf{g}^*(X) := \sum_{t=1}^{n_b} \mathsf{g}_t(\mathbf{a}^*(X))\mathsf{d}^*_t(X)$. For any $i \in [1, \kappa_1]$,

$$\mathsf{g}^*(\mathfrak{z}_i) = \sum_{t=1}^{n_b} \mathsf{g}_t(\mathbf{a}^*(\mathfrak{z}_i))\mathsf{d}^*_t(\mathfrak{z}_i) = \sum_{t=1}^{n_b} \mathsf{g}_t(\bar{\boldsymbol{a}}_i)\bar{d}'_{it} = 0 \ .$$

Since $\mathsf{g}^*(X)$ is at most of degree $n_{\mathsf{g}} < \kappa_1$, it follows that $\mathsf{g}^*(X) = 0$ and consequently $\sum_{t=1}^{n_b} \mathsf{g}_t(\mathbf{a}^*(X))\mathsf{d}^*_t(X) = 0$. Hence, $\mathsf{Ext}_{\mathsf{ss}}^{\mathsf{sanlintrick}}$ extracts a valid witness.

Next, we bound the probabilities $\Pr[\mathsf{bad}_{\mathsf{ext}}]$ and $\Pr[\mathsf{bad}_{\mathsf{evb}}]$. To bound $\Pr[\mathsf{bad}_{\mathsf{ext}}]$, we construct a PPT adversary $\mathcal{B}_{\mathsf{ss}}^{\mathsf{kzg}}$ (see Fig. 8) that breaks the computational special soundness of KZG when the event $\mathsf{bad}_{\mathsf{ext}}$ happens. $\mathcal{B}_{\mathsf{ss}}^{\mathsf{kzg}}$ runs $\mathcal{A}(\mathsf{ck})$ to recover the tree $\mathcal{T}$. Then, it obtains transcript vectors $((\mathbf{tr}_{a_\imath})_{\imath=1}^{n_a}, (\mathbf{tr}_{d_t})_{t=1}^{n_b}) \leftarrow \mathsf{TE}_{\mathsf{lin}}(\mathsf{ck}, \mathcal{T})$. For each transcript vector $\mathbf{tr}_f$, where $f \in \{a_\imath\}_{\imath=1}^{n_a} \cup \{d_t\}_{t=1}^{n_b}$, $\mathcal{B}_{\mathsf{ss}}^{\mathsf{kzg}}$ runs the deterministic extractor $\mathsf{Ext}_{\mathsf{ss}}^{\mathsf{kzg}}(\mathsf{ck}, \mathbf{tr}_f)$ to extract the polynomial $\mathsf{f}^*(X) \in \{\mathsf{a}^*_\imath\}_{\imath=1}^{n_a} \cup \{\mathsf{d}^*_t\}_{t=1}^{n_b}$. $\mathcal{B}_{\mathsf{ss}}^{\mathsf{kzg}}$ returns one transcript vector $\mathbf{tr}_f$, which satisfies $([f]_1, \mathsf{f}^*(X)) \notin \mathcal{R}_{\mathsf{ck},\mathbf{tr}_f}$. When the event $\mathsf{bad}_{\mathsf{ext}}$ happens, $([f]_1, \mathsf{f}^*(X)) \notin \mathcal{R}_{\mathsf{ck},\mathbf{tr}_f}$ for some f, and hence $\mathcal{B}_{\mathsf{ss}}^{\mathsf{kzg}}$ breaks the computational special soundness of KZG. Thus,

$$\Pr[\mathsf{bad}_{\mathsf{ext}}] = \mathsf{Adv}^{\mathrm{ss}}_{\mathsf{Pgen},\mathsf{KZG},\mathsf{Ext}_{\mathsf{ss}}^{\mathsf{kzg}},n+1,\mathcal{B}_{\mathsf{ss}}^{\mathsf{kzg}}}(\lambda) \ .$$

Second, we bind the probability $\Pr[\mathsf{bad}_{\mathsf{evb}}]$. We construct a PPT KZG's evaluation-binding adversary $\mathcal{C}_{\mathsf{evb}}$, depicted in Fig. 8. $\mathcal{C}_{\mathsf{evb}}$ runs $\mathcal{A}$, $\mathsf{TE}_{\mathsf{lin}}$ and $\mathsf{Ext}_{\mathsf{ss}}^{\mathsf{sanlintrick}}$ to obtain $\mathcal{T}$ and corresponding $((\mathsf{a}^*_\imath(X))_{\imath=1}^{n_a}, (\mathsf{d}^*_t(X))_{t=1}^{n_b})$. If the event $\mathsf{bad}_{\mathsf{evb}}$ happens, then for all $\imath$ and t, $([a_\imath]_1, \mathsf{a}^*_\imath(X)) \in \mathcal{R}_{\mathsf{ck},\mathbf{tr}_{a_\imath}}$ and $([d_t]_1, \mathsf{d}^*_t(X)) \in \mathcal{R}_{\mathsf{ck},\mathbf{tr}_{d_t}}$. Thus, $\mathsf{a}^*_\imath(x) = a_\imath$, and $\mathsf{d}^*_t(x) = d_t$. Note that

$$h_{a_\imath}(X) := (\mathsf{a}^*_\imath(X) - \mathsf{a}^*_\imath(\mathfrak{z}_i))/(X - \mathfrak{z}_i)$$

is always a polynomial. Hence, $\mathcal{C}_{\mathsf{evb}}$ can compute $[h_{a_\imath}(x)]_1$ that satisfies

$$[h_{a_\imath}(x)]_1 \bullet [x - \mathfrak{z}_i]_2 = [\mathsf{a}^*_\imath(x) - \mathsf{a}^*_\imath(\mathfrak{z}_i)]_T = [a_\imath - \mathsf{a}^*_\imath(\mathfrak{z}_i)]_T \ .$$

Thus,

$$([a_\imath]_1, \mathfrak{z}_i, \mathsf{a}^*_\imath(\mathfrak{z}_i), [h_{a_\imath}(x)]_1)$$

is an accepting transcript. However, $\mathsf{k.tr}^*_{ij\imath} = ([a_\imath]_1, \mathfrak{z}_i, \bar{a}_{i\imath}, [h^*_{i1\imath}]_1)$ is also an accepting transcript. If $\mathsf{a}^*_\imath(\mathfrak{z}_i) \neq \bar{a}_{i\imath}$, $\mathcal{C}_{\mathsf{evb}}$ has broken KZG's evaluation binding by finding a collision. Analogously, one can show that $\mathsf{d}^*_t(\mathfrak{z}_i) \neq \bar{d}'_{it}$ implies that $\mathcal{C}_{\mathsf{evb}}$ broke KZG's evaluation binding.

We also need that $\mathsf{g}^*(\mathfrak{z}_i) = \sum_{t=1}^{n_b} \mathsf{g}_t(\mathbf{a}^*(\mathfrak{z}_i))\mathsf{d}^*_t(\mathfrak{z}_i) = 0$. From Lemma 2,

$$\mathsf{k.tr}_{d_t,i} = ([d_t]_1, \mathfrak{z}_i, \bar{d}'_{it}, [h'_{it}]_1)$$

and

$$\left(\left[\sum_{t=1}^{n_b} \mathsf{g}_t(\bar{\boldsymbol{a}}_i)d_t\right]_1, \mathfrak{z}_i, 0, \left[h^*_{ij,n_a+1}\right]_1\right)$$

are accepting transcripts for all $i \in [1,\kappa_1]$. Since KZG is triply homomorphic, from all $\mathsf{k.tr}_{d_t,i}$ being accepting, we get

$$\left(\left[\sum_{t=1}^{n_b} \mathsf{g}_t(\bar{\boldsymbol{a}}_i)d_t\right]_1, \mathfrak{z}_i, \sum_{t=1}^{n_b} \mathsf{g}_t(\bar{\boldsymbol{a}}_i)\bar{d}'_{it}, \left[\sum_{t=1}^{n_b} \mathsf{g}_t(\bar{\boldsymbol{a}}_i)h'_{it}\right]_1\right)$$

is an accepting transcript. If $\sum_{t=1}^{n_b} \mathsf{g}_t(\bar{\boldsymbol{a}}_i)\bar{d}'_{it} \neq 0$, $\mathcal{C}_{\mathsf{evb}}$ has found a collision and thus broken KZG's evaluation binding. Therefore,

$$\Pr[\mathsf{bad}_{\mathsf{evb}}] = \mathsf{Adv}^{\mathrm{evb}}_{\mathsf{Pgen},\mathsf{KZG},n,\mathcal{C}_{\mathsf{evb}}}(\lambda)$$

and we can conclude that

$$\mathsf{Adv}^{\mathrm{ss}}_{\mathsf{Pgen},\mathsf{SanLinTrick},\mathsf{Ext}^{\mathsf{sanlintrick}}_{\mathsf{ss}},\boldsymbol{\kappa},\mathcal{A}}(\lambda) = \mathsf{Adv}^{\mathrm{ss}}_{\mathsf{Pgen},\mathsf{KZG},\mathsf{Ext}^{\mathsf{kzg}}_{\mathsf{ss}},n+1,\mathcal{B}^{\mathsf{kzg}}_{\mathsf{ss}}}(\lambda) + \mathsf{Adv}^{\mathrm{evb}}_{\mathsf{Pgen},\mathsf{KZG},n,\mathcal{C}_{\mathsf{evb}}}(\lambda) \ .$$

□

4 New Assumption: SplitRSDH

Our proofs for Plonk and SmallPlonk rely on the following novel falsifiable assumption SplitRSDH.

Definition 3 (par-SplitRSDH). *Let* $\mathsf{par} = (m, n_\psi, n_1, n_{\mathcal{S}}, (\psi_k(X))_{k=1}^m)$. *Assume* $m, n_\psi, n_1, n_{\mathcal{S}} \in \mathsf{poly}(\lambda)$ *are positive integers so that* $n_{\mathcal{S}} > n_1+n_\psi+1$. *Assume* $\psi_k(X) \in \mathbb{F}_{\leq n_\psi}[X]$. *For any PPT* $\mathcal{A}$, $\mathsf{Adv}^{\mathrm{splitrsdh}}_{\mathsf{Pgen},\mathsf{par},\mathcal{A}}(\lambda) :=$

$$\Pr\left[\begin{array}{l|l} \mathcal{S} \subset \mathbb{F} \wedge |\mathcal{S}| = n_{\mathcal{S}} \wedge L(X) \in \mathbb{F}[X] \wedge & x \leftarrow\!\!\$\, \mathbb{F}; \mathsf{p} \leftarrow \mathsf{Pgen}(1^\lambda); \\ \deg L(X) \in [n_1+n_\psi+1, n_{\mathcal{S}}-1] \wedge & \mathsf{ck} \leftarrow ([1,x,\ldots,x^{n_1}]_1, [1,x]_2); \\ \left(\sum_{k=1}^m [\tau_k\psi_k(x)]_1 = [L(x)]_1 + [\varphi \mathsf{Z}_{\mathcal{S}}(x)]_1\right) & \left(\mathcal{S}, L(X), [(\tau_k)_{k=1}^m, \varphi]_1\right) \leftarrow \mathcal{A}(\mathsf{p}, \mathsf{ck}) \end{array}\right]$$

is negligible. Here, $\mathsf{Z}_{\mathcal{S}}(X) := \prod_i (X - \mathfrak{z}_i)$ *is the vanishing polynomial. (The condition* $\sum_{k=1}^m [\tau_k\psi_k(x)]_1 = [L(x)]_1 + [\varphi \mathsf{Z}_{\mathcal{S}}(x)]_1$ *is equivalent to* $\sum_{k=1}^m ([\tau_k]_1 \bullet [\psi_k(x)]_2) = [1]_1 \bullet [L(x)]_2 + [\varphi]_1 \bullet [\mathsf{Z}_{\mathcal{S}}(x)]_2$.*)*

SplitRSDH is falsifiable since the challenger, knowing x, can verify the adversary's success. In Section B, we prove SplitRSDH's security in the new type-safe framework of AGMOS. We will present this framework in Section 6, which we recommend to read first. The intuition behind SplitRSDH and its AGMOS proof is simple (the same intuition is valid for the GGM and AGM, [FKL18]). Let $\tau_k(X)$ be the explanations of $[\tau_k]_1$ provided by the AGMOS adversary. (Recall that in the AGM, the output group element has to be in the span of input group elements.) Let $T(X) := \sum_{k=1}^m \tau_k(X)\psi_k(X) \in \mathbb{F}_{\leq n_1+n_\psi}[X]$. If the challenger accepts, then $T(X) - L(X)$ has a degree smaller than $n_{\mathcal{S}}$ and thus $\varphi(X) = 0$. But then $T(X) = L(X)$, which means that $\deg L(X) \leq n_1 + n_\psi$, a contradiction.

For $m \geq 1$, we can reduce SplitRSDH to the following nicer-looking but purportedly stronger assumption TIDRSDH (target intermediate degree RSDH), where we allow $\mathcal{A}$ to output $[g]_T \leftarrow \sum_{k=1}^m ([\tau_k]_1 \bullet [\psi_k(x)]_2)$. Then, it suffices for the following probability to be negligible (under the conditions of Definition 3):

$$\Pr\left[\begin{array}{l|l} \mathcal{S} \subset \mathbb{F} \wedge |\mathcal{S}| = n_{\mathcal{S}} \wedge L(X) \in \mathbb{F}[X] \wedge & x \leftarrow\!\!\$\, \mathbb{F}; \\ \deg L(X) \in [n_1+n_\psi+1, n_{\mathcal{S}}-1] \wedge & \mathsf{ck} \leftarrow ([1,x,\ldots,x^{n_1}]_1, [1,x,\ldots,x^{n_\psi}]_2); \\ [g - L(x)]_T = [\varphi \mathsf{Z}_{\mathcal{S}}(x)]_T & (\mathcal{S}, L(X), [\varphi]_1, [g]_T) \leftarrow \mathcal{A}(\mathsf{p}, \mathsf{ck}); \end{array}\right] .$$

The AGM(OS) proof of TIDRSDH follows from noticing that if the adversary succeeds, then $g(X) - L(X)$ has degree $< n_{\mathcal{S}}$ and thus $\varphi(X) = 0$. But then $g(X) = L(X)$, which means $\deg L(X) \leq n_1 + n_\psi$ and the adversary failed. TIDRSDH is not weaker and (purportedly) stronger than SplitRSDH for the same par. First, it allows the adversary to output $\mathbb{G}_T$ elements[6]. Second, a SplitRSDH adversary has less freedom to choose $[g]_T$. Third, ck has more elements so the AGMOS reduction is to a stronger PDL. Due to this, we will explicitly use SplitRSDH within the current paper but one can instead use TIDRSDH.

SplitRSDH can also be reduced to two somewhat more standard assumptions. Assume a par-SplitRSDH adversary succeeds. Consider two cases:

- $\varphi \neq 0$. We construct a reduction to a "target ARSDH" assumption. Given $([1, \ldots, x^{n_{\mathcal{S}}-1-n_\psi}]_1, [1, \ldots, x^{n_\psi}]_2)$, it outputs $(\mathcal{S}, [g]_T, [\varphi]_1)$, where $[g]_T = \sum_{k=1}^{m}([\tau_k]_1 \bullet [\psi_k(x)]_2) - [L(x)]_T$, and checks $[g]_T = [\varphi]_1 \bullet [\mathsf{Z}_{\mathcal{S}}(x)]_2$ and $\varphi \neq 0$.
- $\varphi = 0$. We construct a reduction to a "target DHE assumption". Given $([1, \ldots, x^{n_1}]_1, [1, \ldots, x^{n_\psi}]_2)$, it outputs $(L(X), [g]_T)$, where $[g]_T = \sum_{k=1}^{m}([\tau_k]_1 \bullet [\psi_k(x)]_2)$, and checks $\deg L(X) > n_1 + n_\psi$ and $[g]_T = [L(x)]_T$.

5 Special Soundness of Plonk And Variants

In this section, we prove that interactive Plonk [GWC19] has computational special soundness, assuming that KZG has evaluation binding and computational special soundness, and SplitRSDH holds. In addition, we prove the computational special soundness of two Plonk variants.

5.1 Preliminaries For Plonk

We recall Plonk [GWC19], a popular zk-SNARK for proving satisfiability of arbitrary arithmetic circuits. We closely follow the notation of [GWC19].

Let $\mathbb{H}$ be a multiplicative subgroup of $\mathbb{F}$ containing the nth roots of unity. Let ω be a primitive nth root of unity and a generator of $\mathbb{H} = \{1, \omega, \ldots, \omega^{n-1}\}$. Let $\mathsf{Z}_{\mathbb{H}}(X) := X^n - 1$ be the vanishing polynomial on $\mathbb{H}$. For $i \in [1, n]$, $\mathsf{L}_i(X)$ denotes the ith Lagrange polynomial on $\mathbb{H}$. I.e., $\mathsf{L}_i(X)$ is the unique monic polynomial of at most degree $n-1$ such that $\mathsf{L}_i(\omega^i) = 1$ and $\mathsf{L}_i(\omega^j) = 0$ for all $j \in [1, n] \setminus \{i\}$. We assume that the number of constraints is upper bounded by n.

The SNARK proof relation. Let $\mathscr{P} = \{\mathsf{q}_\mathsf{M}(X), \mathsf{q}_\mathsf{L}(X), \mathsf{q}_\mathsf{R}(X), \mathsf{q}_\mathsf{O}(X), \mathsf{q}_\mathsf{C}(X), \mathsf{S}_{\sigma 1}(X), \mathsf{S}_{\sigma 2}(X), \mathsf{S}_{\sigma 3}(X)\}$ be a set of polynomials that defines a given circuit. See Section D.1 for the exact conditions they satisfy. (They are exactly the same as in [GWC19].) We use the notation $\mathsf{q}_{\mathsf{X}i} := \mathsf{q}_\mathsf{X}(\omega^i)$.

Given $\ell \leq n$ and $\mathscr{P}$, we wish to prove statements of knowledge for the relation $\mathcal{R}_\mathscr{P} \subset \mathbb{F}^\ell \times \mathbb{F}^{3n-\ell}$ containing all pairs $\mathbb{x} = (w_i)_{i=1}^{\ell}, \mathbb{w} = (w_i)_{i=\ell+1}^{3n}$ such that

1. For $i \in [1, \ell]$: $\mathsf{q}_{\mathsf{M}i} = \mathsf{q}_{\mathsf{R}i} = \mathsf{q}_{\mathsf{O}i} = \mathsf{q}_{Ci} = 0$ and $\mathsf{q}_{\mathsf{L}i} = -1$, which guarantees
$$\mathsf{q}_{\mathsf{M}i} w_i w_{n+i} + \mathsf{q}_{\mathsf{L}i} w_i + \mathsf{q}_{\mathsf{R}i} w_{n+i} + \mathsf{q}_{\mathsf{O}i} w_{2n+i} + \mathsf{q}_{Ci} = -w_i \ . \tag{4}$$
(This is needed to force the prover to use the correct $\mathbb{x}$.)
2. For all $i \in [\ell + 1, n]$:
$$\mathsf{q}_{\mathsf{M}i} w_i w_{n+i} + \mathsf{q}_{\mathsf{L}i} w_i + \mathsf{q}_{\mathsf{R}i} w_{n+i} + \mathsf{q}_{\mathsf{O}i} w_{2n+i} + \mathsf{q}_{Ci} = 0 \ , \tag{5}$$

[6] Since $\mathbb{G}_T$ (a subgroup of the multiplicative group of a finite field) is not a generic group [JR10], we prefer not to handle adversaries who output $\mathbb{G}_T$ elements. See [JR10] for a discussion.

3. For all $i \in [1, 3n]$:

$$w_i = w_{\sigma(i)} \ . \tag{6}$$

We refer to [GWC19] for the explanation how these constraints are related to arithmetic circuits.

5.2 Plonk And Variants

We present Plonk, SmallPlonk, and SanPlonk. While we describe their interactive versions, to save space, we will omit the adjective "interactive". We describe them in parallel, highlighting changes in SanPlonk and SmallPlonk compared to Plonk. The description of Plonk follows [GWC19] with minor differences. Compared to Plonk, SanPlonk batch opens $[t_{lo}, t_{mid}, t_{hi}]_1$ (three group elements sent in Plonk), applying the sanitization technique from Section 3.2. This results in an interactive argument with two additional rounds and one more field element sent by P, compared to Plonk. On the other hand, SmallPlonk sends just a single group element $[t]_1$ instead of three elements $[t_{lo}, t_{mid}, t_{hi}]_1$.

Common preprocessed input: n, $[x, \ldots, x^{n+5}]_1$ plus additional elements $[x^{n+6}, \ldots, x^{3n+5}]_1$ in SmallPlonk), $(q_{\mathsf{M}i}, q_{\mathsf{L}i}, q_{\mathsf{R}i}, q_{\mathsf{O}i}, q_{\mathsf{C}i})_{i=1}^n$, σ^*, $\mathsf{q}_\mathsf{M}(X) = \sum_{i=1}^n q_{\mathsf{M}i}\mathsf{L}_i(X)$, $\mathsf{q}_\mathsf{L}(X) = \sum_{i=1}^n q_{\mathsf{L}i}\mathsf{L}_i(X)$, $\mathsf{q}_\mathsf{R}(X) = \sum_{i=1}^n q_{\mathsf{R}i}\mathsf{L}_i(X)$, $\mathsf{q}_\mathsf{O}(X) = \sum_{i=1}^n q_{O i}\mathsf{L}_i(X)$, $\mathsf{q}_\mathsf{C}(X) = \sum_{i=1}^n q_{C i}\mathsf{L}_i(X)$, $\mathsf{S}_{\sigma 1}(X) = \sum_{i=1}^n \sigma^*(i)\mathsf{L}_i(X)$, $\mathsf{S}_{\sigma 2}(X) = \sum_{i=1}^n \sigma^*(n+i)\mathsf{L}_i(X)$, $\mathsf{S}_{\sigma 3}(X) = \sum_{i=1}^n \sigma^*(2n+i)\mathsf{L}_i(X)$.

Verifier preprocessed input: $[\mathsf{q}_\mathsf{M}]_1 := [\mathsf{q}_\mathsf{M}(x)]_1$, $[\mathsf{q}_\mathsf{L}]_1 := [\mathsf{q}_\mathsf{L}(x)]_1$, $[\mathsf{q}_\mathsf{R}]_1 := [\mathsf{q}_\mathsf{R}(x)]_1$, $[\mathsf{q}_\mathsf{O}]_1 := [\mathsf{q}_\mathsf{O}(x)]_1$, $[\mathsf{q}_\mathsf{C}]_1 := [\mathsf{q}_\mathsf{C}(x)]_1$, $[s_{\sigma 1}]_1 := [\mathsf{S}_{\sigma 1}(x)]_1$, $[s_{\sigma 2}]_1 := [\mathsf{S}_{\sigma 2}(x)]_1$, $[s_{\sigma 3}]_1 := [\mathsf{S}_{\sigma 3}(x)]_1$, $[x]_1$,

Public input: $(\ell, (w_i)_{i=1}^\ell)$.

First round. On input $(w_i)_{i=1}^{3n}$, the prover samples $(b_1, \ldots, b_9) \leftarrow_\$ \mathbb{F}$, sets $\mathsf{a}(X) \leftarrow \sum_{i=1}^n w_i\mathsf{L}_i(X) + (b_1 X + b_2)\mathsf{Z}_\mathbb{H}(X) \in \mathbb{F}_{\leq n+1}[X]$, $\mathsf{b}(X) \leftarrow \sum_{i=1}^n w_{n+i}\mathsf{L}_i(X) + (b_3 X + b_4)\mathsf{Z}_\mathbb{H}(X) \in \mathbb{F}_{\leq n+1}[X]$, $\mathsf{c}(X) \leftarrow \sum_{i=1}^n w_{2n+i}\mathsf{L}_i(X) + (b_5 X + b_6)\mathsf{Z}_\mathbb{H}(X) \in \mathbb{F}_{\leq n+1}[X]$, and sends $[\mathsf{a}(x), \mathsf{b}(x), \mathsf{c}(x)]_1$ to V. V replies with $\beta, \gamma \leftarrow_\$ \mathbb{F}$.

Second round. The prover sends $[\mathsf{z}(x)]_1$ to V, where $\mathsf{z}(X) \leftarrow \mathsf{L}_1(X) + \sum_{i=1}^{n-1}\left(\prod_{j=1}^i \frac{(w_j+\beta\omega^j+\gamma)(w_{n+j}+\beta k_1\omega^j+\gamma)(w_{2n+j}+\beta k_2\omega^j+\gamma)}{(w_j+\sigma^*(j)\omega^j+\gamma)(w_{n+j}+\sigma^*(n+j)\omega^j+\gamma)(w_{2n+j}+\sigma^*(2n+j)\omega^j+\gamma)}\right)\mathsf{L}_{i+1}(X) + (b_7 X^2 + b_8 X + b_9)\mathsf{Z}_\mathbb{H}(X) \in \mathbb{F}_{\leq n+2}[X]$. V replies with $\alpha \leftarrow_\$ \mathbb{F}$.

Third round. The prover does the following. Let $\mathsf{PI}(X) = \sum_{i=0}^\ell -w_i\mathsf{L}_i(X)$ be the public input polynomial. Compute

$$\begin{aligned}
\mathsf{F}_0(X) &:= \mathsf{a}(X)\mathsf{b}(X)\mathsf{q}_\mathsf{M}(X) + \mathsf{a}(X)\mathsf{q}_\mathsf{L}(X) + \mathsf{b}(X)\mathsf{q}_\mathsf{R}(X) + \mathsf{c}(X)\mathsf{q}_\mathsf{O}(X) + \mathsf{PI}(X) + \mathsf{q}_\mathsf{C}(X) \\
\mathsf{F}_1(X) &:= (\mathsf{a}(X) + \beta X + \gamma)\,(\mathsf{b}(X) + \beta k_1 X + \gamma)\,(\mathsf{c}(X) + \beta k_2 X + \gamma)\,\mathsf{z}(X) \\
&\quad - (\mathsf{a}(X) + \beta\mathsf{S}_{\sigma 1}(X) + \gamma)(\mathsf{b}(X) + \beta\mathsf{S}_{\sigma 2}(X) + \gamma)(\mathsf{c}(X) + \beta\mathsf{S}_{\sigma 3}(X) + \gamma)\mathsf{z}(X\omega) \\
\mathsf{F}_2(X) &:= (\mathsf{z}(X) - 1)\mathsf{L}_1(X) \\
\mathsf{F}(X) &:= \mathsf{F}_0(X) + \alpha\mathsf{F}_1(X) + \alpha^2\mathsf{F}_2(X) \ , \\
\mathsf{t}(X) &:= \frac{\mathsf{F}(X)}{\mathsf{Z}_\mathbb{H}(X)} \in \mathbb{F}_{\leq 3n+5}[X] \ .
\end{aligned} \tag{7}$$

In SmallPlonk, the prover just sends $[\mathsf{t}(x)]_1$ to the verifier. In Plonk and SanPlonk, we split $\mathsf{t}(X)$ into polynomials $\mathsf{t}'_{\mathsf{lo}}(X), \mathsf{t}'_{\mathsf{mid}}(X)$ (both of degree less than n) and $\mathsf{t}'_{\mathsf{hi}}(X)$ (of degree at most $n+5$), such that $\mathsf{t}(X) = \mathsf{t}'_{\mathsf{lo}}(X) + X^n\mathsf{t}'_{\mathsf{mid}}(X) + X^{2n}\mathsf{t}'_{\mathsf{hi}}(X)$. Sample $b_{10}, b_{11}, b_{12} \leftarrow_\$ \mathbb{F}$ and define $\mathsf{t}_{\mathsf{lo}}(X) := \mathsf{t}'_{\mathsf{lo}}(X) + b_{10}X^n + b_{12}X^{n+1}$, $\mathsf{t}_{\mathsf{mid}}(X) := \mathsf{t}'_{\mathsf{mid}}(X) - b_{10} - b_{12}X + b_{11}X^n$, and $\mathsf{t}_{\mathsf{hi}}(X) := \mathsf{t}'_{\mathsf{hi}}(X) - b_{11}$. ($b_{12}$ is required for the zero-knowledge proof of SanPlonk.) Note that $\mathsf{t}(X) = \mathsf{t}_{\mathsf{lo}}(X) + X^n\mathsf{t}_{\mathsf{mid}}(X) + X^{2n}\mathsf{t}_{\mathsf{hi}}(X)$. Send $[\mathsf{t}_{\mathsf{lo}}(x), \mathsf{t}_{\mathsf{mid}}(x), \mathsf{t}_{\mathsf{hi}}(x)]_1$ to the verifier. The verifier replies with the evaluation randomness $\mathfrak{z} \leftarrow_\$ \mathbb{F}$.

Fourth round. The prover does the following. Set $\bar{a} \leftarrow \mathsf{a}(\mathfrak{z})$, $\bar{b} \leftarrow \mathsf{b}(\mathfrak{z})$, $\bar{c} \leftarrow \mathsf{c}(\mathfrak{z})$, $\bar{s}_{\sigma 1} \leftarrow \mathsf{S}_{\sigma 1}(\mathfrak{z})$, $\bar{s}_{\sigma 2} \leftarrow \mathsf{S}_{\sigma 2}(\mathfrak{z})$, $\bar{z}_\omega \leftarrow \mathsf{z}(\omega\mathfrak{z})$. Send $(\bar{a}, \bar{b}, \bar{c}, \bar{s}_{\sigma 1}, \bar{s}_{\sigma 2}, \bar{z}_\omega)$ to the verifier. The verifier replies with the sanitization randomness $\delta \leftarrow\!\!\$\ \mathbb{F}$.

Fourth (Plonk/SmallPlonk) or fifth (SanPlonk) round. The prover sends $\bar{t}_{\mathfrak{z}} \leftarrow \mathsf{t}_{\mathsf{lo}}(\mathfrak{z}) + \delta\mathsf{t}_{\mathsf{mid}}(\mathfrak{z}) + \delta^2\mathsf{t}_{\mathsf{hi}}(\mathfrak{z})$. The verifier replies with $v \leftarrow\!\!\$\ \mathbb{F}$.

Fifth (Plonk/SmallPlonk) or sixth (SanPlonk) round. Prover does the following. Compute the linearization polynomial $\mathsf{r}(X)$:

$$
\begin{aligned}
\Lambda_0(X) &= \bar{a}\bar{b} \cdot \mathsf{q}_{\mathsf{M}}(X) + \bar{a} \cdot \mathsf{q}_{\mathsf{L}}(X) + \bar{b} \cdot \mathsf{q}_{\mathsf{R}}(X) + \bar{c} \cdot \mathsf{q}_{\mathsf{O}}(X) + \mathsf{PI}(\mathfrak{z}) + \mathsf{q}_{\mathsf{C}}(X) \ , \\
\Lambda_1(X) &= (\bar{a} + \beta\mathfrak{z} + \gamma)(\bar{b} + \beta k_1\mathfrak{z} + \gamma)(\bar{c} + \beta k_2\mathfrak{z} + \gamma) \cdot \mathsf{z}(X) \\
&\quad - (\bar{a} + \beta\bar{s}_{\sigma 1} + \gamma)(\bar{b} + \beta\bar{s}_{\sigma 2} + \gamma)(\bar{c} + \beta \cdot \mathsf{S}_{\sigma 3}(X) + \gamma)\bar{z}_\omega \ , \\
\Lambda_2(X) &= (\mathsf{z}(X) - 1)\mathsf{L}_1(\mathfrak{z}) \ , \\
\mathsf{r}(X) &= \Lambda_0(X) + \alpha\Lambda_1(X) + \alpha^2\Lambda_2(X) \\
&\quad - \begin{cases} \mathsf{Z}_{\mathbb{H}}(\mathfrak{z}) \cdot (\mathsf{t}_{\mathsf{lo}}(X) + \mathfrak{z}^n\mathsf{t}_{\mathsf{mid}}(X) + \mathfrak{z}^{2n}\mathsf{t}_{\mathsf{hi}}(X)) & \text{(in Plonk and SanPlonk)} \\ \mathsf{Z}_{\mathbb{H}}(\mathfrak{z}) \cdot \mathsf{t}(X) & \text{(in SmallPlonk)} \end{cases} .
\end{aligned} \tag{8}
$$

Let

$$
\begin{aligned}
\mathsf{H}(X) &:= \mathsf{r}(X) + v\mathsf{a}(X) + v^2\mathsf{b}(X) + v^3\mathsf{c}(X) + v^4\mathsf{S}_{\sigma 1}(X) + v^5\mathsf{S}_{\sigma 2}(X) \\
&\quad + v^6(\mathsf{t}_{\mathsf{lo}}(X) + \delta\mathsf{t}_{\mathsf{mid}}(X) + \delta^2\mathsf{t}_{\mathsf{hi}}(X)) \ , \\
\bar{H} &:= v\bar{a} + v^2\bar{b} + v^3\bar{c} + v^4\bar{s}_{\sigma 1} + v^5\bar{s}_{\sigma 2} + v^6\bar{t}_{\mathfrak{z}} \ .
\end{aligned} \tag{9}
$$

Compute the opening proof polynomials $\mathsf{W}_{\mathfrak{z}}(X) := (\mathsf{H}(X) - \bar{H})/(X - \mathfrak{z})$ and $\mathsf{W}_{\mathfrak{z}\omega}(X) = \frac{\mathsf{z}(X) - \bar{z}_\omega}{X - \mathfrak{z}\omega}$, $[\mathsf{W}_{\mathfrak{z}}]_1 := [\mathsf{W}_{\mathfrak{z}}(x)]_1$; $[\mathsf{W}_{\mathfrak{z}\omega}]_1 := [\mathsf{W}_{\mathfrak{z}\omega}(x)]_1$; Send $[\mathsf{W}_{\mathfrak{z}}, \mathsf{W}_{\mathfrak{z}\omega}]_1$;

Verification algorithm.

1. Compute $\mathsf{Z}_{\mathbb{H}}(\mathfrak{z}) = \mathfrak{z}^n - 1$, $\mathsf{L}_1(\mathfrak{z}) = \frac{\omega(\mathfrak{z}^n - 1)}{n(\mathfrak{z} - \omega)}$, and $\mathsf{PI}(\mathfrak{z}) = \sum_{i \in [\ell]} w_i\mathsf{L}_i(\mathfrak{z})$.
2. Split $\mathsf{r}(X)$ into its constant and non-constant terms. Compute $\mathsf{r}(X)$'s constant term: $r_0 := \mathsf{PI}(\mathfrak{z}) - \mathsf{L}_1(\mathfrak{z})\alpha^2 - \alpha(\bar{a} + \beta\bar{s}_{\sigma 1} + \gamma)(\bar{b} + \beta\bar{s}_{\sigma 2} + \gamma)(\bar{c} + \gamma)\bar{z}_\omega$, and let $\mathsf{r}'(X) := \mathsf{r}(X) - r_0$.
3. Sample $u \leftarrow\!\!\$\ \mathbb{F}$ and compute the first part of the batched polynomial commitment $[D]_1 := [\mathsf{r}'(x)]_1 + u \cdot [z]_1$:

$$
\begin{aligned}
[D]_1 &:= \bar{a}\bar{b} \cdot [\mathsf{q}_{\mathsf{M}}]_1 + \bar{a} \cdot [\mathsf{q}_{\mathsf{L}}]_1 + \bar{b} \cdot [\mathsf{q}_{\mathsf{R}}]_1 + \bar{c} \cdot [\mathsf{q}_{\mathsf{O}}]_1 + [\mathsf{q}_{\mathsf{C}}]_1 \\
&\quad + \big((\bar{a} + \beta\mathfrak{z} + \gamma)(\bar{b} + \beta k_1\mathfrak{z} + \gamma)(\bar{c} + \beta k_2\mathfrak{z} + \gamma)\alpha + \mathsf{L}_1(\mathfrak{z})\alpha^2 + u\big) \cdot [z]_1 \\
&\quad - (\bar{a} + \beta\bar{s}_{\sigma 1} + \gamma)\ (\bar{b} + \beta\bar{s}_{\sigma 2} + \gamma)\alpha\beta\bar{z}_\omega \cdot [s_{\sigma 3}]_1 \\
&\quad - \begin{cases} \mathsf{Z}_{\mathbb{H}}(\mathfrak{z}) \cdot ([t_{lo}]_1 + \mathfrak{z}^n \cdot [t_{mid}]_1 + \mathfrak{z}^{2n} \cdot [t_{hi}]_1) & \text{(in Plonk and SanPlonk)} \\ \mathsf{Z}_{\mathbb{H}}(\mathfrak{z}) \cdot [t]_1 & \text{(in SmallPlonk)} \end{cases} .
\end{aligned}
$$

4. Compute full batched polynomial commitment $[F]_1 := [D]_1 + v \cdot [a]_1 + v^2 \cdot [b]_1 + v^3 \cdot [c]_1 + v^4 \cdot [s_{\sigma 1}]_1 + v^5 \cdot [s_{\sigma 2}]_1 + v^6[t_{lo} + \delta t_{mid} + \delta^2 t_{hi}]_1$.
5. Compute the batch evaluation $E_0 := -r_0 + v\bar{a} + v^2\bar{b} + v^3\bar{c} + v^4\bar{s}_{\sigma 1} + v^5\bar{s}_{\sigma 2} + v^6\bar{t}_{\mathfrak{z}}$, $E_1 := \bar{z}_\omega$, and $E := E_0 + uE_1$.
6. Batch validate all evaluations:

$$
([\mathsf{W}_{\mathfrak{z}}]_1 + u \cdot [\mathsf{W}_{\mathfrak{z}\omega}]_1) \bullet [x]_2 \stackrel{?}{=} (\mathfrak{z} \cdot [\mathsf{W}_{\mathfrak{z}}]_1 + u\mathfrak{z}\omega \cdot [\mathsf{W}_{\mathfrak{z}\omega}]_1 + [F]_1 - [E]_1) \bullet [1]_2 \ . \tag{10}
$$

Clearly, SanPlonk remains complete after the highlighted changes. In Section F, we prove that SanPlonk has zero knowledge.

5.3 Computational Special-Soundness Proof of (San)Plonk's IP

We prove the computational special soundness of Plonk and SmallPlonk assuming that (1) KZG has evaluation binding and computational special soundness, and (2) a new falsifiable assumption SplitRSDH holds. We prove the computational special soundness of SanPlonk solely under (1). By applying the Fiat-Shamir transform to any of the three constructions, one obtains a zk-SNARKs secure in the ROM under the same assumptions.

Before going on, we note that the Plonk verifier performs batch verification, using a batching coefficient u created after the prover's last message. That is, after unrolling all optimizations, Eq. (10) can be replaced by two checks,

$$\begin{aligned} \begin{bmatrix} r+v(a-\bar{a})+v^2(b-\bar{b})+v^3(c-\bar{c})+v^4(s_{\sigma 1}-\bar{s}_{\sigma 1}) \\ +v^5(s_{\sigma 2}-\bar{s}_{\sigma 2})+v^6(t_{lo}+\delta t_{mid}+\delta^2 t_{hi}-\bar{t}_{\mathfrak{z}}) \end{bmatrix}_1 \bullet [1]_2 &= [\mathsf{W}_{\mathfrak{z}}]_1 \bullet [x-\mathfrak{z}]_2 \ , \\ [z-\bar{z}_{\omega}]_1 \bullet [1]_2 &= [\mathsf{W}_{\mathfrak{z}\omega}]_1 \bullet [x-\mathfrak{z}\omega]_2 \ , \end{aligned} \tag{11}$$

where $[r]_1 = [D]_1 - u[z]_1 + r_0[1]_1$. Batching with u introduces a soundness error $1/|\mathbb{F}|$. We say that a Plonk transcript is *accepting* when Eq. (11) is satisfied.

We divide the proof into several smaller lemmas. In Section 5.4, we analyze a subtree of accepting transcripts for fixed β, γ, and α, showing that from it, one can extract certain polynomials. In Section 5.5, we use this to prove the computational special soundness of three Plonk variants.

In the following, $n \in \mathsf{poly}(\lambda)$ and,

$$\begin{aligned} \kappa_{\mathsf{kzg}} &= \begin{cases} n+5 & \text{(in Plonk and SanPlonk)} \ , \\ 3n+5 & \text{(in SmallPlonk)} \ , \end{cases} \\ \boldsymbol{\kappa}_{\mathsf{Plonk}} &= (\kappa_\beta = 3n+1, \kappa_\gamma = 3n+1, \kappa_\alpha = 3, \kappa_{\mathfrak{z}} = 4\kappa_{\mathsf{kzg}}+1, \kappa_v^{\mathsf{Plonk}} = 6) \ , \\ \boldsymbol{\kappa}_{\mathsf{san}} &= (\kappa_\beta = 3n+1, \kappa_\gamma = 3n+1, \kappa_\alpha = 3, \kappa_{\mathfrak{z}} = 4\kappa_{\mathsf{kzg}}+1, \kappa_\delta = 3, \kappa_v^{\mathsf{san}} = 7) \end{aligned} \tag{12}$$

corresponding to the branching factors of KZG, Plonk, SanPlonk, and SmallPlonk. Moreover, SmallPlonk uses $\boldsymbol{\kappa}_{\mathsf{small}} = \boldsymbol{\kappa}_{\mathsf{Plonk}}$, except for a different value of $\kappa_{\mathsf{kzg}} = 3n+5$. Let $\kappa_v = \kappa_v^{\mathsf{Plonk}}$ in Plonk and $\kappa_v = \kappa_v^{\mathsf{san}}$ in SanPlonk.

We define the SplitRSDH parameters (see Section 4)

$$\mathsf{par}_n^{\mathsf{plonk}} = (m, n_\psi, n_1, n_{\mathcal{S}}, (\psi_k(X))_{k=1}^m) \ ,$$

with $m = 3$, $n_\psi = 2n$, $n_1 = \kappa_{\mathsf{kzg}} = n+5$, $n_{\mathcal{S}} = \kappa_{\mathfrak{z}} = 4\kappa_{\mathsf{kzg}}+1$, and $\psi_k(X) = X^{(k-1)n}$, and

$$\mathsf{par}_n^{\mathsf{smallplonk}} = (m, n_\psi, n_1, n_{\mathcal{S}}, (\psi_k(X))_{k=1}^m) \ ,$$

with $m = 1$, $n_\psi = 0$, $n_1 = \kappa_{\mathsf{kzg}} = 3n+5$, $n_{\mathcal{S}} = \kappa_{\mathfrak{z}} = 4\kappa_{\mathsf{kzg}}+1$, and $\psi_1(X) = 1$. In both cases, $n_{\mathcal{S}} > n_1 + n_\psi + n$ (we need the latter in Theorem 9). Here, $\mathsf{par}_n^{\mathsf{plonk}}$ is only useful for Plonk and $\mathsf{par}_n^{\mathsf{smallplonk}}$ is only useful for SmallPlonk.

5.4 Subtree Analysis

In this subsection, we analyze a subtree of Plonk's transcripts for fixed β, γ, and α. As usual, we start with a tree extractor lemma that gets a tree of accepting Plonk transcripts as input and outputs many accepting KZG transcripts that open relevant polynomial commitments at many different locations.

$\mathsf{TE}_{*\mathsf{plonk}}(\mathsf{ck}, \hat{\mathfrak{T}}_{\beta\gamma\alpha})$

1 : Parse $\hat{\mathfrak{T}}_{\beta\gamma\alpha} = (\mathsf{tr}_{ijk})_{i\in[1,\kappa_{\mathfrak{z}}],j\in[1,\kappa_\delta],k\in[1,\kappa_v]}$; // tr_{ijk} as in Eq. (13); $\kappa_\delta = 1$ in Plonk

2 : **for** $i \in [1, \kappa_{\mathfrak{z}}]$ **do**

3 : $[\Lambda_{0i}]_1 \leftarrow \bar{a}_i\bar{b}_i[\mathsf{q}_\mathsf{M}]_1 + \bar{a}_i[\mathsf{q}_\mathsf{L}]_1 + \bar{b}_i[\mathsf{q}_\mathsf{R}]_1 + \bar{c}_i[\mathsf{q}_\mathsf{O}]_1 + \mathsf{PI}(\mathfrak{z})[1]_1 + [\mathsf{q}_\mathsf{C}]_1$;

4 : $[\Lambda_{1i}]_1 \leftarrow (\bar{a}_i + \beta\mathfrak{z}_i + \gamma)(\bar{b}_i + \beta k_1\mathfrak{z}_i + \gamma)(\bar{c}_i + \beta k_2\mathfrak{z}_i + \gamma)[z]_1$

5 : $\quad - (\bar{a}_i + \beta\bar{s}_{\sigma 1} + \gamma)(\bar{b}_i + \beta\bar{s}_{\sigma 2} + \gamma)(\bar{c}_i[1]_1 + \beta[\mathsf{S}_{\sigma 3}(x)]_1 + \gamma[1]_1)\bar{z}_{\omega,i}$;

6 : $[\Lambda_{2i}]_1 \leftarrow [z-1]_1\mathsf{L}_1(\mathfrak{z}_i)$;

7 : $[r_i]_1 \leftarrow [\Lambda_{0i}]_1 + \alpha[\Lambda_{1i}]_1 + \alpha^2[\Lambda_{2i}]_1 - \begin{cases} \mathsf{Z}_{\mathbb{H}}(\mathfrak{z}_i)\cdot([t_{lo}]_1 + \mathfrak{z}_i^n[t_{mid}]_1 + \mathfrak{z}_i^{2n}[t_{hi}]_1) \\ \mathsf{Z}_{\mathbb{H}}(\mathfrak{z}_i)\cdot[t]_1 \ \text{(SmallPlonk)} \end{cases}$;

8 : $[\mathsf{W}'_{\mathfrak{z}i\mathbf{1}1}, \ldots, \mathsf{W}'_{\mathfrak{z}i\mathbf{1}\kappa_v}]_1^\intercal \leftarrow \boldsymbol{V}_{i\mathbf{1}}^{-1}[\mathsf{W}_{\mathfrak{z}i\mathbf{1}1}, \ldots, \mathsf{W}_{\mathfrak{z}i\mathbf{1}\kappa_v}]_1^\intercal$; // $\boldsymbol{V}_{i\mathbf{1}}$ as in 15, 16

9 : $\mathsf{k.tr}_{1i} \leftarrow ([r_i]_1, \mathfrak{z}_i, 0, [\mathsf{W}'_{\mathfrak{z}i\mathbf{1}1}]_1)$; $\mathsf{k.tr}_{2i} \leftarrow ([a]_1, \mathfrak{z}_i, \bar{a}_i, [\mathsf{W}'_{\mathfrak{z}i\mathbf{1}2}]_1)$;

10 : $\mathsf{k.tr}_{3i} \leftarrow ([b]_1, \mathfrak{z}_i, \bar{b}_i, [\mathsf{W}'_{\mathfrak{z}i\mathbf{1}3}]_1)$; $\mathsf{k.tr}_{4i} \leftarrow ([c]_1, \mathfrak{z}_i, \bar{c}_i, [\mathsf{W}'_{\mathfrak{z}i\mathbf{1}4}]_1)$;

11 : $\mathsf{k.tr}_{5i} \leftarrow ([s_{\sigma 1}]_1, \mathfrak{z}_i, \bar{s}_{\sigma 1 i}, [\mathsf{W}'_{\mathfrak{z}i\mathbf{1}5}]_1)$; $\mathsf{k.tr}_{6i} \leftarrow ([s_{\sigma 2}]_1, \mathfrak{z}_i, \bar{s}_{\sigma 2 i}, [\mathsf{W}'_{\mathfrak{z}i\mathbf{1}6}]_1)$;

12 : **for** $j \in \{1,2,3\}$ **do** $\bar{t}_{\mathfrak{z}ij} \leftarrow (\boldsymbol{V}_{ij}^{-1})_7(\bar{H}_{ij1}, \ldots, \bar{H}_{ij7})^\intercal$; **endfor**

13 : $(\bar{t}_{\mathfrak{z}lo,i}, \bar{t}_{\mathfrak{z}mid,i}, \bar{t}_{\mathfrak{z}hi,i})^\intercal := \boldsymbol{C}_i^{-1}(\bar{t}_{\mathfrak{z}i1}, \bar{t}_{\mathfrak{z}i2}, \bar{t}_{\mathfrak{z}i3})^\intercal$;

14 : $[W_{lo,i}, W_{mid,i}, W_{hi,i}]_1 \leftarrow \boldsymbol{C}_i^{-1}[\mathsf{W}'_{\mathfrak{z}i17}, \mathsf{W}'_{\mathfrak{z}i27}, \mathsf{W}'_{\mathfrak{z}i37}]_1^\intercal$;

15 : $\mathsf{k.tr}_{7i} \leftarrow ([t_{lo}]_1, \mathfrak{z}_i, \bar{t}_{\mathfrak{z}lo,i}, [\mathsf{W}_{lo,i}]_1)$; $\mathsf{k.tr}_{8i} \leftarrow ([t_{mid}]_1, \mathfrak{z}_i, \bar{t}_{\mathfrak{z}mid,i}, [\mathsf{W}_{mid,i}]_1)$;

16 : $\mathsf{k.tr}_{9i} \leftarrow ([t_{hi}]_1, \mathfrak{z}_i, \bar{t}_{\mathfrak{z}hi,i}, [\mathsf{W}_{hi,i}]_1)$;

17 : $\mathsf{k.tr}_i^\omega \leftarrow ([z]_1, \mathfrak{z}_i\omega, \bar{z}_{\omega i}, [\mathsf{W}_{\mathfrak{z}\omega i\mathbf{1}1}]_1)$; **endfor**

18 : $\mathbf{k.tr}^{\boldsymbol{\omega}} \leftarrow (\mathsf{k.tr}_i^\omega)_{i\in[1,\kappa_{\mathfrak{z}}]}$; **for** $k \in [1, \kappa_v + 2]$ **do** $\mathbf{k.tr}_k \leftarrow (\mathsf{k.tr}_{ki})_{i\in[1,\kappa_{\mathfrak{z}}]}$; **endfor**

19 : **return** $((\mathbf{k.tr}_k)_{k\in[1,\kappa_v+2]}, \mathbf{k.tr}^{\boldsymbol{\omega}})$;

Fig. 9. The subroutine $\mathsf{TE}_{*\mathsf{plonk}}$.

Lemma 3 (Transcript tree to KZG transcripts). *Let $\mathcal{T}$ be a $\boldsymbol{\kappa}_{\mathsf{Plonk}}$-tree of Plonk's (resp., $\boldsymbol{\kappa}_{\mathsf{san}}$-tree of SanPlonk's and $\boldsymbol{\kappa}_{\mathsf{small}}$-tree of SmallPlonk's) accepting transcripts. Let $\hat{\mathfrak{T}}_{\beta\gamma\alpha}$ be a $(1, 1, 1, \kappa_{\mathfrak{z}}, \kappa_\delta, \kappa_v)$-subtree of $\mathcal{T}$ for any fixed β, γ, and α, with the transcripts in this subtree denoted as*

$$\mathsf{tr}_{ijk} = \begin{pmatrix} [a,b,c]_1, \beta, \gamma, [z]_1, \alpha, [t_{lo}, t_{mid}, t_{hi}]_1 \ \textit{([t]}_1 \textit{ in SmallPlonk)}, \\ \mathfrak{z}_i, \bar{a}_i, \bar{b}_i, \bar{c}_i, \bar{s}_{\sigma 1 i}, \bar{s}_{\sigma 2 i}, \bar{z}_{\omega i}, \delta_{ij}, \bar{t}_{\mathfrak{z}ij}, v_{ijk}, [\mathsf{W}_{\mathfrak{z}ij\mathbf{1}k}, \mathsf{W}_{\mathfrak{z}\omega ij\mathbf{1}k}]_1 \end{pmatrix} . \tag{13}$$

The DPT extractor $\mathsf{TE}_{*\mathsf{plonk}}(\mathsf{ck}, \hat{\mathfrak{T}}_{\beta\gamma\alpha})$ in Fig. 9 computes $((\mathbf{k.tr}_k)_k, \mathbf{k.tr}^{\boldsymbol{\omega}})$, where $k \in [1, \kappa_v^{\mathsf{Plonk}}] = [1, 6]$ in Plonk and SmallPlonk and $k \in [1, \kappa_v^{\mathsf{san}} + 2] = [1, 9]$ in SanPlonk, of KZG accepting transcripts, such that (1) $\mathsf{k.tr}_{1i}$, $\mathsf{k.tr}_{2i}$, $\mathsf{k.tr}_{3i}$, $\mathsf{k.tr}_{4i}$, $\mathsf{k.tr}_{5i}$, and $\mathsf{k.tr}_{6i}$ open (respectively) $[r_i]_1$, $[a]_1$, $[b]_1$, $[c]_1$, $[s_{\sigma 1}]_1$ and $[s_{\sigma 2}]_1$ to 0, $\bar{a}_i$, $\bar{b}_i$, $\bar{c}_i$, $\bar{s}_{\sigma 1 i}$, and $\bar{s}_{\sigma 2 i}$ at $\mathfrak{z}_i$, and (2) $\mathsf{k.tr}_i^\omega$ opens $[z]_1$ to $\bar{z}_{\omega i}$ at $\mathfrak{z}_i\omega$. In SanPlonk, $\mathsf{k.tr}_{7i}$, $\mathsf{k.tr}_{8i}$, and $\mathsf{k.tr}_{9i}$ open (respectively) $[t_{lo}, t_{mid}, t_{hi}]_1$ to some $\bar{t}_{\mathfrak{z}lo,i}$, $\bar{t}_{\mathfrak{z}mid,i}$, and $\bar{t}_{\mathfrak{z}hi,i}$ at $\mathfrak{z}_i$. Moreover, $\mathfrak{z}_i$ are mutually different.

Proof. Let $\mathcal{T}$ be a $(\kappa_\beta, \kappa_\gamma, \kappa_\alpha, \kappa_{\mathfrak{z}}, \ldots)$-tree of accepting transcripts. We fix a $(1, 1, 1, \kappa_{\mathfrak{z}}, \kappa_\delta, \kappa_v)$-subtree $\hat{\mathfrak{T}}_{\beta\gamma\alpha}$ of $\mathcal{T}$ for some β, γ, and α. Then, $\hat{\mathfrak{T}}_{\beta\gamma\alpha} = \{\mathsf{tr}_{ijk}\}$ contains accepting transcripts given in Eq. (13) with mutually different $\mathfrak{z}_i$.

Let us unload some of the formulas in Fig. 9. First, $[\Lambda_{0i}]_1$, $[\Lambda_{1i}]_1$, $[\Lambda_{2i}]_1$, $[r_i]_1$ (lines 3, 4, 6, and 7 in Fig. 9) are commitments to ($\mathfrak{z}_i$-dependent) polynomials Λ_{0i}, Λ_{1i}, Λ_{2i}, and r, defined as in Eq. (8). Recall that we analyze the case the verifier individually tests the two verification

equations in Eq. (11), ignoring the optimization induced by using the batching variable u. Let

$$\begin{aligned}
\bar{H}_{ijk} &\leftarrow v^0_{ijk}\cdot 0 + v^1_{ijk}\bar{a}_i + v^2_{ijk}\bar{b}_i + v^3_{ijk}\bar{c}_i + v^4_{ijk}\bar{s}_{\sigma 1 i} + v^5_{ijk}\bar{s}_{\sigma 2 i} + v^6_{ijk}\bar{t}_{\mathfrak{z} ij} \ , \\
[\mathsf{t}_{\delta_{ij}}]_1 &\leftarrow [t_{lo} + \delta_{ij} t_{mid} + \delta^2_{ij} t_{hi}]_1 \ , \\
[\mathsf{H}_{ijk}]_1 &\leftarrow v^0_{ijk}[r_i]_1 + v^1_{ijk}[a]_1 + v^2_{ijk}[b]_1 + v^3_{ijk}[c]_1 + v^4_{ijk}[s_{\sigma 1}]_1 + v^5_{ijk}[s_{\sigma 2}]_1 + v^6_{ijk}[\mathsf{t}_{\delta_{ij}}]_1 .
\end{aligned} \tag{14}$$

Since KZG is triply homomorphic, tr_{ijk} is an accepting Plonk transcript iff $([\mathsf{H}_{ijk}]_1, \mathfrak{z}_i, \bar{H}_{ijk}, [\mathsf{W}_{\mathfrak{z} ijk}]_1)$ and $\mathsf{k.tr}^{\omega}_i$ (line 17 in Fig. 9) are accepting KZG transcripts. We get this by slight rewriting of Plonk's verification equation. In particular, $\mathsf{k.tr}^{\omega}_i$ are accepting transcripts, with different values of $\mathfrak{z}_i$.

We will separate the rest of the proof to the case of (1) Plonk and SmallPlonk, and (2) SanPlonk. However, the subroutine on Fig. 9 corresponds to both.

Plonk And SmallPlonk. Here,

$$\boldsymbol{V}_i = \begin{pmatrix} 1 & v_{i1} & \dots & v^5_{i1} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & v_{i6} & \dots & v^5_{i6} \end{pmatrix} \ , \tag{15}$$

is an invertible Vandermonde matrix. According to Eq. (14), $(\bar{H}_{i1}, \dots, \bar{H}_{i6})^\intercal = \boldsymbol{V}_i \cdot (0, \bar{a}_i, \bar{b}_i, \bar{c}_i, \bar{s}_{\sigma 1 i}, \bar{s}_{\sigma 2 i})^\intercal$ and $[\mathsf{H}_{i1}, \dots, \mathsf{H}_{i6}]^\intercal_1 = \boldsymbol{V}_i \cdot [r_i, a, b, c, s_{\sigma 1}, s_{\sigma 2}]^\intercal_1$. Let $[\mathsf{W}'_{\mathfrak{z} i1}, \dots, \mathsf{W}'_{\mathfrak{z} i6}]^\intercal_1$ be as on line 8 of Fig. 9. By the triple homomorphism of KZG, $\mathsf{k.tr}_{ki}$ are accepting KZG transcripts for every i and k.

SanPlonk. Here,

$$\boldsymbol{V}_{ij} = \begin{pmatrix} 1 & v_{ij1} & \dots & v^6_{ij1} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & v_{ij7} & \dots & v^6_{ij7} \end{pmatrix} \quad \text{and} \quad \boldsymbol{C}_i := \begin{pmatrix} 1 & \delta_{i1} & \delta^2_{i1} \\ 1 & \delta_{i2} & \delta^2_{i2} \\ 1 & \delta_{i3} & \delta^2_{i3} \end{pmatrix} \tag{16}$$

are invertible Vandermonde matrices. According to Eq. (14), $[\mathsf{H}_{ij1}, \dots, \mathsf{H}_{ij7}]_1 = \boldsymbol{V}_{ij} \cdot [r_i, a, b, c, s_{\sigma 1}, s_{\sigma 2}, \mathsf{t}_{\delta_{ij}}]^\intercal_1$ and $(\bar{H}_{ij1}, \dots, \bar{H}_{ij7})^\intercal = \boldsymbol{V}_{ij} \cdot (0, \bar{a}_i, \bar{b}_i, \bar{c}_i, \bar{s}_{\sigma 1 i}, \bar{s}_{\sigma 2 i}, \bar{t}_{\mathfrak{z} ij})^\intercal$. Let $[\mathsf{W}'_{\mathfrak{z} ij1}, \dots, \mathsf{W}'_{\mathfrak{z} ij7}]^\intercal_1$ be defined as on line 8 of Fig. 9. By triple homomorphism, $\mathsf{k.tr}_{1i}, \dots, \mathsf{k.tr}_{6i}$ and $\mathsf{k.tr}^*_{7i}$ are accepting KZG transcripts, where $\mathsf{k.tr}_{1i}$ to $\mathsf{k.tr}_{6i}$ are as in Fig. 9 and $\mathsf{k.tr}^*_{7i} := ([\mathsf{t}_{\delta_{ij}}]_1, \mathfrak{z}_i, \bar{t}_{\mathfrak{z} ij}, [\mathsf{W}'_{\mathfrak{z} ij7}]_1)$.

Next, $\boldsymbol{C}_i \cdot [t_{lo}, t_{mid}, t_{hi}]^\intercal_1 = [\mathsf{t}_{\delta_{i1}}, \mathsf{t}_{\delta_{i2}}, \mathsf{t}_{\delta_{i3}}]^\intercal_1$. Define $(\bar{t}_{\mathfrak{z} lo,i}, \bar{t}_{\mathfrak{z} mid,i}, \bar{t}_{\mathfrak{z} hi,i})^\intercal := \boldsymbol{C}_i^{-1} \cdot (\bar{t}_{\mathfrak{z} i1}, \bar{t}_{\mathfrak{z} i2}, \bar{t}_{\mathfrak{z} i3})^\intercal$ and $[W_{lo,i}, W_{mid,i}, W_{hi,i}]_1 := \boldsymbol{C}_i^{-1} [\mathsf{W}'_{\mathfrak{z} i17}, \mathsf{W}'_{\mathfrak{z} i27}, \mathsf{W}'_{\mathfrak{z} i37}]^\intercal_1$. Thus, $\mathsf{k.tr}_{7i}$, $\mathsf{k.tr}_{8i}$, and $\mathsf{k.tr}_{9i}$ are accepting KZG transcripts for every i. □

Theorem 5 (Subtree extractor). *Let $\mathcal{T}$ be a $\boldsymbol{\kappa}_{\mathsf{Plonk}}$-tree of Plonk's (resp., $\boldsymbol{\kappa}_{\mathsf{san}}$-tree of SanPlonk's or $\boldsymbol{\kappa}_{\mathsf{small}}$-tree of SmallPlonk's) accepting transcripts. Let $\hat{\mathcal{T}}_{\beta\gamma\alpha} = (\mathsf{tr}_{ijk})$ be a subtree of $\mathcal{T}$ for any fixed β, γ, and α, where tr_{ijk} are as in Eq. (13). Assume that KZG has evaluation binding and computational $(\kappa_{\mathsf{kzg}}+1)$-special soundness. In the case of Plonk (resp., SmallPlonk), assume the $\mathsf{par}^{\mathsf{plonk}}_n$-SplitRSDH (resp., $\mathsf{par}^{\mathsf{smallplonk}}_n$-SplitRSDH) assumption holds. There exists a DPT extractor $\mathsf{Ext}^{\mathsf{sub}}_{\mathsf{ss}}$ that, given $\hat{\mathcal{T}}_{\beta\gamma\alpha}$, outputs $(\mathsf{z}(X), \mathsf{a}(X), \mathsf{b}(X), \mathsf{c}(X))$, where $\mathsf{z}(X)$, $\mathsf{a}(X)$, $\mathsf{b}(X)$, and $\mathsf{c}(X)$ are consistent with the commitments and all $\kappa_{\mathfrak{z}}$ openings of $[z, a, b, c]_1$. Moreover, $\mathsf{t}(X)$ (defined as in Eq. (7)) is a polynomial.*

Proof. Let $\mathcal{T}$ be an accepting transcript $(\kappa_\beta, \kappa_\gamma, \kappa_\alpha, \kappa_{\mathfrak{z}}, \kappa_\delta, \kappa_v)$-tree and let $\hat{\mathcal{T}}_{\beta\gamma\alpha}$ be its $(1, 1, 1, \kappa_{\mathfrak{z}}, \kappa_\delta, \kappa_v)$-subtree for any fixed α, β, γ. By Lemma 3, $\mathsf{TE}_{*\mathsf{plonk}}(\mathsf{ck}, \hat{\mathcal{T}}_{\beta\gamma\alpha})$ extracts accepting KZG transcripts $\mathsf{k.tr}_{ki}$ and $\mathsf{k.tr}^{\omega}_i$ for every i and k. Consider separately the cases of (1) Plonk and SmallPlonk and (2) SanPlonk. The extractors and adversaries on most figures (say, Fig. 10) correspond to both.

Case of Plonk And SmallPlonk. In Fig. 10, we depict an extractor $\mathsf{Ext}^{\mathsf{sub}}_{\mathsf{ss}}$. $\mathsf{Ext}^{\mathsf{sub}}_{\mathsf{ss}}$ invokes $\mathsf{Ext}^{\mathsf{kzg}}_{\mathsf{ss}}(\mathsf{ck}, \mathbf{k.tr}^{\boldsymbol{\omega}})$ and $\mathsf{Ext}^{\mathsf{kzg}}_{\mathsf{ss}}(\mathsf{ck}, \mathbf{k.tr}_k)$ for $k \in [2, 4]$, extracting polynomials $\mathsf{z}(X)$, $\mathsf{a}(X)$, $\mathsf{b}(X)$,

$\mathsf{Ext}^{\mathsf{sub}}_{\mathsf{ss}}(\mathsf{ck}, \hat{\mathfrak{T}}_{\beta\gamma\alpha})$

$((\mathbf{k.tr}_k)_{k\in[1,\kappa_v+2]}, \mathbf{k.tr}^{\boldsymbol{\omega}}) \leftarrow \mathsf{TE}_{*\mathsf{plonk}}(\mathsf{ck}, \hat{\mathfrak{T}}_{\beta\gamma\alpha})$;
$\mathsf{z}(X) \leftarrow \mathsf{Ext}^{\mathsf{kzg}}_{\mathsf{ss}}(\mathsf{ck}, \mathbf{k.tr}^{\boldsymbol{\omega}})$; $\mathsf{a}(X) \leftarrow \mathsf{Ext}^{\mathsf{kzg}}_{\mathsf{ss}}(\mathsf{ck}, \mathbf{k.tr}_2)$;
$\mathsf{b}(X) \leftarrow \mathsf{Ext}^{\mathsf{kzg}}_{\mathsf{ss}}(\mathsf{ck}, \mathbf{k.tr}_3)$; $\mathsf{c}(X) \leftarrow \mathsf{Ext}^{\mathsf{kzg}}_{\mathsf{ss}}(\mathsf{ck}, \mathbf{k.tr}_4)$;
$\mathsf{t}_{\mathsf{lo}}(X) \leftarrow \mathsf{Ext}^{\mathsf{kzg}}_{\mathsf{ss}}(\mathsf{ck}, \mathbf{k.tr}_7)$; $\mathsf{t}_{\mathsf{mid}}(X) \leftarrow \mathsf{Ext}^{\mathsf{kzg}}_{\mathsf{ss}}(\mathsf{ck}, \mathbf{k.tr}_8)$; $\mathsf{t}_{\mathsf{hi}}(X) \leftarrow \mathsf{Ext}^{\mathsf{kzg}}_{\mathsf{ss}}(\mathsf{ck}, \mathbf{k.tr}_9)$;
return $(\mathsf{z}(X), \mathsf{a}(X), \mathsf{b}(X), \mathsf{c}(X), \mathsf{t}_{\mathsf{lo}}(X), \mathsf{t}_{\mathsf{mid}}(X), \mathsf{t}_{\mathsf{hi}}(X))$;

Fig. 10. Plonk's/SanPlonk's/SmallPlonk's computational special-soundness extractor.

$\mathcal{B}^{\mathsf{kzg}}_{\mathsf{ss}}(\mathsf{ck})$

$\hat{\mathfrak{T}}_{\beta\gamma\alpha} \leftarrow \mathcal{A}^{\mathsf{plonk}}_{\mathsf{ss}}(\mathsf{ck})$;
$((\mathbf{k.tr}_k)_{k\in[1,\kappa_v+2]}, \mathbf{k.tr}^{\boldsymbol{\omega}}) \leftarrow \mathsf{TE}_{*\mathsf{plonk}}(\mathsf{ck}, \hat{\mathfrak{T}}_{\beta\gamma\alpha})$;
$(\mathsf{z}(X), \mathsf{a}(X), \mathsf{b}(X), \mathsf{c}(X), \mathsf{t}_{\mathsf{lo}}(X), \mathsf{t}_{\mathsf{mid}}(X), \mathsf{t}_{\mathsf{hi}}(X)) \leftarrow \mathsf{Ext}^{\mathsf{sub}}_{\mathsf{ss}}(\mathsf{ck}, \hat{\mathfrak{T}}_{\beta\gamma\alpha})$;
if $([z]_1, \mathsf{z}(X)) \notin \mathcal{R}_{\mathsf{ck},\mathbf{k.tr}^{\boldsymbol{\omega}}}$ **then return** $\mathbf{k.tr}^{\boldsymbol{\omega}}$; **fi**
if $([a]_1, \mathsf{a}(X)) \notin \mathcal{R}_{\mathsf{ck},\mathbf{k.tr}_2}$ **then return** $\mathbf{k.tr}_2$; **fi**
if $([b]_1, \mathsf{b}(X)) \notin \mathcal{R}_{\mathsf{ck},\mathbf{k.tr}_3}$ **then return** $\mathbf{k.tr}_3$; **fi**
if $([c]_1, \mathsf{c}(X)) \notin \mathcal{R}_{\mathsf{ck},\mathbf{k.tr}_4}$ **then return** $\mathbf{k.tr}_4$; **fi**
if $([t_{lo}]_1, \mathsf{t}_{\mathsf{lo}}(X)) \notin \mathcal{R}_{\mathsf{ck},\mathbf{k.tr}_7}$ **then return** $\mathbf{k.tr}_7$; **fi**
if $([t_{mid}]_1, \mathsf{t}_{\mathsf{mid}}(X)) \notin \mathcal{R}_{\mathsf{ck},\mathbf{k.tr}_8}$ **then return** $\mathbf{k.tr}_8$; **fi**
if $([t_{hi}]_1, \mathsf{t}_{\mathsf{hi}}(X)) \notin \mathcal{R}_{\mathsf{ck},\mathbf{k.tr}_9}$ **then return** $\mathbf{k.tr}_9$; **fi**
return $\perp$;

Fig. 11. Plonk's/SanPlonk's KZG's computational special-soundness adversary $\mathcal{B}^{\mathsf{kzg}}_{\mathsf{ss}}$.

and $\mathsf{c}(X)$ of degree $\leq \kappa_{\mathsf{kzg}}$ (see Eq. (12)). After executing $\mathsf{Ext}^{\mathsf{sub}}_{\mathsf{ss}}$, we use the following procedure to possibly set one of the "bad" flags:

(i) $\mathsf{bad}_{\mathsf{ext}} \leftarrow \mathsf{false}$; $\mathsf{bad}_{\mathsf{evb}} \leftarrow \mathsf{false}$; $\mathsf{bad}_{\mathsf{rhino}} \leftarrow \mathsf{false}$;
(ii) if $([z]_1, \mathsf{z}(X)) \notin \mathcal{R}_{\mathsf{ck},\mathbf{k.tr}_1} \vee ([a]_1, \mathsf{a}(X)) \notin \mathcal{R}_{\mathsf{ck},\mathbf{k.tr}_2} \vee ([b]_1, \mathsf{b}(X)) \notin \mathcal{R}_{\mathsf{ck},\mathbf{k.tr}_3} \vee ([c]_1, \mathsf{c}(X)) \notin \mathcal{R}_{\mathsf{ck},\mathbf{k.tr}_4}$ (see Eq. (1)) then $\mathsf{bad}_{\mathsf{ext}} \leftarrow \mathsf{true}$; abort;
(iii) for $i \in [\kappa_{\mathsf{kzg}}+2, \kappa_{\mathfrak{z}}]$: if $\mathsf{z}(\mathfrak{z}_i\omega) \neq \bar{z}_{\omega i} \vee \mathsf{a}(\mathfrak{z}_i) \neq \bar{a}_i \vee \mathsf{b}(\mathfrak{z}_i) \neq \bar{b}_i \vee \mathsf{c}(\mathfrak{z}_i) \neq \bar{c}_i$ then $\mathsf{bad}_{\mathsf{evb}} \leftarrow \mathsf{true}$; abort;
(iv) for $i \in [1, \kappa_{\mathfrak{z}}]$: if $\mathsf{S}_{\sigma 1}(\mathfrak{z}_i) \neq \bar{s}_{\sigma 1 i} \vee \mathsf{S}_{\sigma 2}(\mathfrak{z}_i) \neq \bar{s}_{\sigma 2 i}$ then $\mathsf{bad}_{\mathsf{evb}} \leftarrow \mathsf{true}$; abort;
(v) if $\mathsf{Z}_{\mathbb{H}}(X) \nmid (\mathsf{F}(X) = \mathsf{F}_0(X) + \alpha\mathsf{F}_1(X) + \alpha^2\mathsf{F}_2(X))$, then $\mathsf{bad}_{\mathsf{rhino}} \leftarrow \mathsf{true}$;

Importantly, only one of the "bad" flags is set at a time. Thus, for example, $\mathsf{bad}_{\mathsf{evb}} = \mathsf{true}$ implies that $\mathsf{bad}_{\mathsf{ext}} = \mathsf{false}$. Let $\mathcal{E}$ be the event $\mathsf{Ext}^{\mathsf{sub}}_{\mathsf{ss}}$ succeeds. Thus, $\mathcal{E}$ is the event that $\mathsf{a}(X)$, $\mathsf{b}(X)$, $\mathsf{c}(X)$, and $\mathsf{z}(X)$ are consistent with the commitments and all openings, and $\mathsf{t}(X) = (\mathsf{F}_0(X) + \alpha\mathsf{F}_1(X) + \alpha^2\mathsf{F}_2(X))/\mathsf{Z}_{\mathbb{H}}(X)$ is a polynomial. Let $\overline{\mathsf{bad}}$ be the event none of the bad flags was set. We analyze the success probability of $\mathsf{Ext}^{\mathsf{sub}}_{\mathsf{ss}}$. For this, we make the following claims.

1. **Claim 1**. $\Pr[\mathcal{E}|\overline{\mathsf{bad}}] = 1$. Really, assume that $\overline{\mathsf{bad}}$ holds. Since $\mathsf{bad}_{\mathsf{ext}} = \mathsf{bad}_{\mathsf{evb}} = \mathsf{false}$, we get that $\mathsf{a}(X)$, $\mathsf{b}(X)$, $\mathsf{c}(X)$, $\mathsf{z}(X)$ are consistent with the commitments and all openings. Since $\mathsf{bad}_{\mathsf{rhino}} = \mathsf{false}$, $\mathsf{t}(X) = (\mathsf{F}_0(X) + \alpha\mathsf{F}_1(X) + \alpha^2\mathsf{F}_2(X))/\mathsf{Z}_{\mathbb{H}}(X)$ is a polynomial.
2. **Claim 2**. There exists a KZG's computational $(\kappa_{\mathsf{kzg}}+1)$-special-soundness adversary $\mathcal{B}^{\mathsf{kzg}}_{\mathsf{ss}}$ (see Fig. 11), such that $\Pr[\mathcal{B}^{\mathsf{kzg}}_{\mathsf{ss}}$ succeeds $\mid \mathsf{bad}_{\mathsf{ext}}] = 1$.

$\mathcal{C}_{\mathsf{evb}}(\mathsf{p},\mathsf{ck})$

$\hat{\mathfrak{T}}_{\beta\gamma\alpha} \leftarrow \mathcal{A}_{\mathsf{ss}}^{\mathsf{plonk}}(\mathsf{ck}); ((\mathbf{k.tr}_k)_{k\in[1,\kappa_v+2]}, \mathbf{k.tr}^{\boldsymbol{\omega}}) \leftarrow \mathsf{TE}_{*\mathsf{plonk}}(\mathsf{ck}, \hat{\mathfrak{T}}_{\beta\gamma\alpha})$;

$(\mathsf{z}(X), \mathsf{a}(X), \mathsf{b}(X), \mathsf{c}(X), \mathsf{t}_{\mathsf{lo}}(X), \mathsf{t}_{\mathsf{mid}}(X), \mathsf{t}_{\mathsf{hi}}(X)) \leftarrow \mathsf{Ext}_{\mathsf{ss}}^{\mathsf{sub}}(\mathsf{ck}, \hat{\mathfrak{T}}_{\beta\gamma\alpha})$;

for $i \in [\kappa_{\mathsf{kzg}}+2, \kappa_{\mathfrak{z}}]$ **do**

  if $\mathsf{z}(\mathfrak{z}_i\omega) \neq \bar{z}_{\omega i}$ **then**

    return $([z]_1, \mathfrak{z}_i\omega, \bar{z}_{\omega i}, [\mathsf{W}'_{\omega i \mathbf{j}_1}]_1, \mathsf{z}(\mathfrak{z}_i\omega), [(\mathsf{z}(x)-\mathsf{z}(\mathfrak{z}_i\omega))/(x-\mathfrak{z}_i\omega)]_1)$; **fi**

  if $\mathsf{a}(\mathfrak{z}_i) \neq \bar{a}_i$ **then return** $([a]_1, \mathfrak{z}_i, \bar{a}_i, [\mathsf{W}'_{\omega i \mathbf{j}_2}]_1, \mathsf{a}(\mathfrak{z}_i), [(\mathsf{a}(x)-\mathsf{a}(\mathfrak{z}_i))/(x-\mathfrak{z}_i)]_1)$; **fi**

  if $\mathsf{b}(\mathfrak{z}_i) \neq \bar{b}_i$ **then return** $([b]_1, \mathfrak{z}_i, \bar{b}_i, [\mathsf{W}'_{\omega i \mathbf{j}_3}]_1, \mathsf{b}(\mathfrak{z}_i), [(\mathsf{b}(x)-\mathsf{b}(\mathfrak{z}_i))/(x-\mathfrak{z}_i)]_1)$; **fi**

  if $\mathsf{c}(\mathfrak{z}_i) \neq \bar{c}_i$ **then return** $([c]_1, \mathfrak{z}_i, \bar{c}_i, [\mathsf{W}'_{\omega i \mathbf{j}_4}]_1, \mathsf{c}(\mathfrak{z}_i), [(\mathsf{c}(x)-\mathsf{c}(\mathfrak{z}_i))/(x-\mathfrak{z}_i)]_1)$; **fi**

  if $\mathsf{t}_{\mathsf{lo}}(\mathfrak{z}_i) \neq \bar{t}_{\mathfrak{z}lo,i}$ **then return** $([t_{lo}]_1, \mathfrak{z}_i, \bar{t}_{\mathfrak{z}lo,i}, [\mathsf{W}'_{\omega i \mathbf{j}_7}]_1, \mathsf{t}_{\mathsf{lo}}(\mathfrak{z}_i), [(\mathsf{t}_{\mathsf{lo}}(x)-\mathsf{t}_{\mathsf{lo}}(\mathfrak{z}_i))/(x-\mathfrak{z}_i)]_1)$; **fi**

  if $\mathsf{t}_{\mathsf{mid}}(\mathfrak{z}_i) \neq \bar{t}_{\mathfrak{z}mid,i}$ **then return** $([t_{mid}]_1, \mathfrak{z}_i, \bar{t}_{\mathfrak{z}mid,i}, [\mathsf{W}'_{\omega i \mathbf{j}_8}]_1, \mathsf{t}_{\mathsf{mid}}(\mathfrak{z}_i), [(\mathsf{t}_{\mathsf{mid}}(x)-\mathsf{t}_{\mathsf{mid}}(\mathfrak{z}_i))/(x-\mathfrak{z}_i)]_1)$; **fi**

  if $\mathsf{t}_{\mathsf{hi}}(\mathfrak{z}_i) \neq \bar{t}_{\mathfrak{z}hi,i}$ **then return** $([t_{hi}]_1, \mathfrak{z}_i, \bar{t}_{\mathfrak{z}hi,i}, [\mathsf{W}'_{\omega i \mathbf{j}_9}]_1, \mathsf{t}_{\mathsf{hi}}(\mathfrak{z}_i), [(\mathsf{t}_{\mathsf{hi}}(x)-\mathsf{t}_{\mathsf{hi}}(\mathfrak{z}_i))/(x-\mathfrak{z}_i)]_1)$; **fi**

endfor ;

for $i \in [1, \kappa_{\mathfrak{z}}]$ **do**

  $\mathsf{r}_i(X) \leftarrow \Lambda_{0i}(X) + \alpha\Lambda_{1i}(X) + \alpha^2\Lambda_{2i}(X) - \mathsf{Z}_{\mathbb{H}}(\mathfrak{z}_i)\cdot(\mathsf{t}_{\mathsf{lo}}(X) + \mathfrak{z}_i^n\mathsf{t}_{\mathsf{mid}}(X) + \mathfrak{z}_i^{2n}\mathsf{t}_{\mathsf{hi}}(X))$;

  if $\mathsf{r}_i(\mathfrak{z}_i) \neq 0$ **then return** $([r_i]_1, \mathfrak{z}_i, 0, [\mathsf{W}'_{\mathfrak{z}i11}]_1, \mathsf{r}_i(\mathfrak{z}_i), [\mathsf{r}_i(x)-\mathsf{r}_i(\mathfrak{z}_i)/(x-\mathfrak{z}_i)]_1)$;

endfor

for $i \in [1, \kappa_{\mathfrak{z}}]$ **do**

  if $\mathsf{S}_{\sigma 1}(\mathfrak{z}_i) \neq \bar{s}_{\sigma 1 i}$ **then**

    return $([s_{\sigma 1}]_1, \mathfrak{z}_i, \bar{s}_{\sigma 1 i}, [\mathsf{W}'_{\omega i \mathbf{j}_5}]_1, \mathsf{S}_{\sigma 1}(\mathfrak{z}_i), [(\mathsf{S}_{\sigma 1}(x)-\mathsf{S}_{\sigma 1}(\mathfrak{z}_i))/(x-\mathfrak{z}_i)]_1)$; **fi**

  if $\mathsf{S}_{\sigma 2}(\mathfrak{z}_i) \neq \bar{s}_{\sigma 2 i}$ **then**

    return $([s_{\sigma 2}]_1, \mathfrak{z}_i, \bar{s}_{\sigma 2 i}, [\mathsf{W}'_{\omega i \mathbf{j}_6}]_1, \mathsf{S}_{\sigma 2}(\mathfrak{z}_i), [(\mathsf{S}_{\sigma 2}(x)-\mathsf{S}_{\sigma 2}(\mathfrak{z}_i))/(x-\mathfrak{z}_i)]_1)$; **fi**

endfor

return $\bot$;

Fig. 12. KZG's evaluation-binding adversary $\mathcal{C}_{\mathsf{evb}}$.

Recall from Item ii that $\mathsf{bad}_{\mathsf{ext}}$ is set if one of the four bad events happens. $\mathcal{B}_{\mathsf{ss}}^{\mathsf{kzg}}$ just tests which of the cases is true and returns the corresponding transcript. Clearly, $\Pr[\mathcal{B}_{\mathsf{ss}}^{\mathsf{kzg}} \text{ succeeds} \mid \mathsf{bad}_{\mathsf{ext}}] = 1$.

3. **Claim 3**. There exists an evaluation-binding adversary $\mathcal{C}_{\mathsf{evb}}$ (see Fig. 12) for KZG, such that $\Pr[\mathcal{C}_{\mathsf{evb}} \text{ succeeds} \mid \mathsf{bad}_{\mathsf{evb}}] = 1$.
   Assume that $\mathsf{bad}_{\mathsf{evb}} = \mathsf{true}$. (Note that $\mathsf{bad}_{\mathsf{evb}} = \mathsf{true}$ means that $\mathsf{bad}_{\mathsf{ext}} = \mathsf{false}$, that is, $\mathsf{Ext}_{\mathsf{ss}}^{\mathsf{sub}}$ managed to extract all polynomials.) Then, one of the bad cases in Item iii or Item iv happens. $\mathcal{C}_{\mathsf{evb}}$ just finds out which of these events happens, and depending on the case, returns a collision. By the correctness of the extraction, and the completeness property of KZG, any of the returned values in Fig. 12 *is* a collision. Thus, $\Pr[\mathcal{C}_{\mathsf{evb}} \text{ succeeds} \mid \mathsf{bad}_{\mathsf{evb}}] = 1$.
4. **Claim 4**. In the case of Plonk or SmallPlonk, there exists a SplitRSDH adversary $\mathcal{D}_{\text{splitrsdh}}$ (see Fig. 16), such that $\Pr[\mathcal{D}_{\text{splitrsdh}} \text{ succeeds} \mid \mathsf{bad}_{\mathsf{rhino}}] = 1$. In Section D.2, we prove this claim as a separate theorem, Theorem 9.

Recalling all three bad events are disjoint,

$$\begin{aligned}\Pr[\overline{\mathcal{E}}] &= \Pr[\overline{\mathcal{E}}|\overline{\mathsf{bad}}]\Pr[\overline{\mathsf{bad}}] + \Pr[\overline{\mathcal{E}}|\mathsf{bad}_{\mathsf{ext}}]\Pr[\mathsf{bad}_{\mathsf{ext}}] + \Pr[\overline{\mathcal{E}}|\mathsf{bad}_{\mathsf{evb}}]\Pr[\mathsf{bad}_{\mathsf{evb}}] \\ &\quad + \Pr[\overline{\mathcal{E}}|\mathsf{bad}_{\mathsf{rhino}}]\Pr[\mathsf{bad}_{\mathsf{rhino}}] \\ &\leq 0 + \Pr[\mathsf{bad}_{\mathsf{ext}}] + \Pr[\mathsf{bad}_{\mathsf{evb}}] + \Pr[\mathsf{bad}_{\mathsf{rhino}}] \ .\end{aligned}$$

Since C_{evb} succeeds whenever $\mathsf{bad}_{\mathsf{evb}}$ is set and KZG is evaluation binding, $\Pr[\mathsf{bad}_{\mathsf{evb}}] = \mathsf{negl}(\lambda)$. Similarly, $\Pr[\mathsf{bad}_{\mathsf{ext}}] = \Pr[\mathsf{bad}_{\mathsf{rhino}}] = \mathsf{negl}(\lambda)$. Thus, $\Pr[\overline{\mathcal{E}}] \leq \mathsf{negl}(\lambda)$. This proves the claim.

Case of SanPlonk. We postpone the proof of this case to Section D.3. □

5.5 Full Computational Special-Soundness Proof

Finally, we are ready to prove the computational special soundness of Plonk's variants. For this, we combine the subtree analysis of Section 5.4, KZG's binding (required to guarantee that extracted polynomials like $\mathsf{a}(X)$ are the same in all subtrees), and an analysis of the permutation argument.

Theorem 6. *Let $n \in \mathsf{poly}(\lambda)$ and κ_{kzg}, $\boldsymbol{\kappa}_{\mathsf{Plonk}}$, and $\boldsymbol{\kappa}_{\mathsf{san}}$ be as in Eq. (12).*

1. *If KZG has computational $(\kappa_{\mathsf{kzg}} + 1)$-special soundness and evaluation binding, and $\mathsf{par}_n^{\mathsf{smallplonk}}$-SplitRSDH holds, then SmallPlonk has computational $\boldsymbol{\kappa}_{\mathsf{Plonk}}$-special soundness.*
2. *If KZG has computational $(\kappa_{\mathsf{kzg}}+1)$-special soundness and evaluation binding, and $\mathsf{par}_n^{\mathsf{plonk}}$-SplitRSDH holds, then Plonk has computational $\boldsymbol{\kappa}_{\mathsf{Plonk}}$-special soundness.*
3. *If KZG has computational $(\kappa_{\mathsf{kzg}}+1)$-special soundness and evaluation binding, then SanPlonk has computational $\boldsymbol{\kappa}_{\mathsf{san}}$-special soundness.*

We postpone the proof to Section D.4. Note that we prove special soundness for the information-theoretic argument underlying Plonk as an implicit sub-claim in the proof of the previous theorem. To our knowledge, this is the first time that such a strong security property has been proven for Plonk's information-theoretic component. We analyze the security of the Fiat-Shamir compiled Plonk in Section E.

6 An Oracle Framework for AGMOS

In Section 4, we proposed a new assumption (SplitRSDH). In Section B, prove it is secure in the AGMOS (AGM with oblivious sampling, [LPS23]). We emphasize that we only use AGMOS as a sanity check that the new standard-looking assumption will likely be secure. This is a standard and least controversial use of the ideal group models. However, to have this part of the paper in a more solid foundation, we recast AGMOS in the type-safe oracle framework of Jaeger and Mohan (Crypto 2024, [JM24]). Importantly, [JM24] proved an AGM to GGM lifting result that also carries over to our framework for AGMOS.

The resulting variant of AGMOS is more natural than AGMOS's original description in [LPS23]. Namely, [LPS23] added sampling oracles on top of the AGM framework of [FKL18] that restricts the adversaries (by requiring them to output explanations) but does not have any other oracles. On the other hand, the oracle AGM framework of [JM24] restricts only the group access by providing oracles to group operations and other interesting functionalities. To the oracle framework of [JM24], our AGMOS framework just adds two more sampling oracles. We hope this makes AGMOS simpler to understand and use.

The AGM [FKL18] is a popular idealized model to prove the security of various assumptions and protocols. Compared to GGM, AGM offers two significant advantages. First, it is more realistic, allowing the adversaries to see the bit-representations of group elements. (Shoup's GGM [Sho97] assumes the bit-representations are randomized, and Maurer's GGM [Mau05] does not reveal bit-representations.) Second, AGM proofs are shorter than GGM proofs. GGM proofs usually start and end with somewhat boilerplate steps, which are common to all GGM proofs. AGM allows one to abstract these steps away and thus directly concentrate on the meat of the proof.

Fuchsbauer, Kiltz, and Loss [FKL18] left certain aspects of the AGM unformalized. This gave rise to a number of criticisms [Zha22,ZZK22] and fixes [Zha22,JM24]. In Crypto 2024, Jaeger and Mohan [JM24] proposed a rigorous interpretation of the AGM that answers the criticisms. In particular, they prove that under mild assumptions, an AGM proof of security (in their interpretation) implies a GGM proof of security; see [JM24, Theorem 2] for the concrete statement. We will follow [JM24] when describing AGM and constructing AGM proofs. More precisely, we will use their AGM oracle framework similar to Maurer's type-safe oracle GGM framework [Mau05]. The sole difference of their oracle framework from Maurer's framework is the existence of the encode oracle.

Fix a pairing description $\mathsf{p} \leftarrow \mathsf{Pgen}(1^\lambda)$ and let $\mathsf{encode}_\iota : \mathbb{G}_\iota \mapsto \{0,1\}^{\mathsf{poly}(\lambda)}$ be a fixed injective encoding of elements of $\mathbb{G}_\iota$ as a bit-representation. In the oracle framework of bilinear AGM, there are two tables, **Val** (values) and **Expl** (explanations). An algebraic adversary $\mathcal{A}$ is given handles to **Val** and access to a fixed family $\Pi_{\mathcal{O}}$ of oracles. We will denote the input from the challenger in $\mathbb{G}_\iota$ by $[\mathbb{x}_\iota]_\iota$ that are sent to the wrapper extractor $\mathsf{Ext}^{\mathcal{A}}$. We assume $[\mathbb{x}_\iota]_\iota$ always includes $[1]_1$ and $[1]_2$. Let $\mathbb{x} = ([\mathbb{x}_1]_1, [\mathbb{x}_2]_2, [\mathbb{x}_T]_T)$. $\mathsf{Ext}^{\mathcal{A}}$ initializes **Val** with the elements of $\mathbb{x}$ and the corresponding entries of **Expl** with unit vectors $\boldsymbol{e}_i$ (corresponding to the explanations of type $\mathbb{x}_\iota[i] = \boldsymbol{e}_i^\intercal \mathbb{x}_\iota$).

$\mathsf{Ext}^{\mathcal{A}}$ gives $\mathcal{A}$ an access to oracles and handles corresponding to $\mathbb{x}$. Each oracle call can modify one element of **Val** and the same element of **Expl**. $\mathcal{A}$ returns to $\mathsf{Ext}^{\mathcal{A}}$ some bit-strings (e.g., field elements) and handles j_i corresponding to its output group elements. $\mathsf{Ext}^{\mathcal{A}}$ then returns $\mathcal{A}$'s output bit-string together the group elements $\mathbf{Val}[j_i]$ and the explanations $\mathbf{Expl}[j_i]$. Clearly, $\mathsf{Ext}^{\mathcal{A}}$ is PPT. Notably, Ext depends only on $\mathcal{A}$ being algebraic (i.e., using the oracles to perform group operations) and the access to **Val** and **Expl**, but it does not depend on $\mathcal{A}$'s code. (Jaeger and Mohan [JM24] describe this extractor in their Figure 9.)

Most oracles only operate on group elements based on handles. All oracles can output $\perp$ (for example, when asked to add elements from different groups). In the bilinear setting, the two most important oracles are $\mathcal{O}p$ (takes two elements of the same group, referenced by handles i and j, and stores their sum to a location k) and $\mathcal{P}ai\imath$ (takes elements of groups $\mathbb{G}_1$ and $\mathbb{G}_2$, referenced by handles i and j, and stores their pairing to a location k). The oracles also compute explanations of their group element output. The explanation of $\mathcal{O}p$'s output is the sum of explanations of its inputs. The explanation of $\mathcal{P}ai\imath$'s output is a tensor product of the input explanations. Two oracles that output non-group elements are $\mathcal{E}q$ (takes two elements of the same group, referenced by handles i and j, and returns 1 if they are equal) and $\mathcal{E}ncode$ (takes a group element, referenced by a handle i, and returns its bit representation). See Fig. 13.

Additional oracles are possible if they work on handles and are efficient to implement. E.g., [JM24] considers a scalar multiplication oracle; however, it can be implemented using $\mathcal{O}p$. Similarly, $\mathcal{E}q$ can be implemented by using $\mathcal{E}ncode$. If not being careful, adding or removing oracles can change the model drastically. Crucially, the presence of $\mathcal{E}ncode$ is the only difference between the oracle framework of AGM and Maurer's GGM; in the latter, only $\mathcal{E}q$ can be used to deduce information about the bit-representation of group elements.

According to [JM24], the only difference between the GGM, (Maurer's) AGM, and the standard model is that in the GGM, the adversary has no access to $\mathcal{E}ncode$ and $\mathcal{D}ecode$, in Maurer's AGM, the adversary has access to $\mathcal{E}ncode$, and in the standard model, the adversary has access to both $\mathcal{E}ncode$ and $\mathcal{D}ecode$. Here, $\mathcal{D}ecode$, an inverse of $\mathcal{E}ncode$, takes as an input a bit-string (a bit-representation of a group element) and outputs the group element.

Lipmaa, Parisella, and Siim [LPS23] recently defined AGMOS (AGM with oblivious sampling). AGMOS is more realistic than AGM since AGMOS adversaries are given the additional power of sampling group elements without knowing their discrete logarithms. Importantly, as shown in [LPS23], specific uses of KZG [KZG10] are secure in AGM but not in AGMOS.

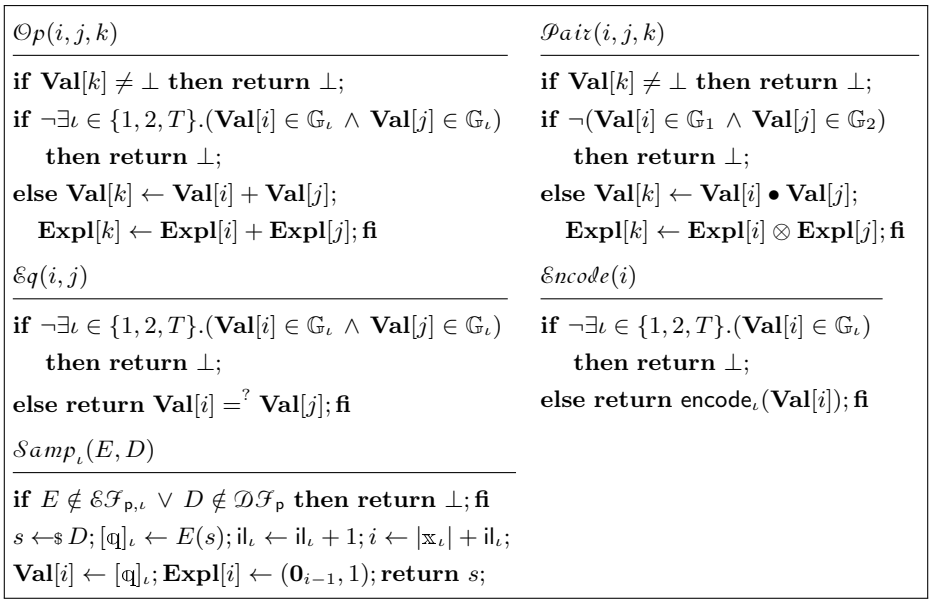

Fig. 13. The description of all considered oracles.

Let $\mathcal{EF}_{\mathsf{p},\iota}$ be a set of (polynomially many) functions $\mathbb{F} \to \mathbb{G}_\iota$. $\mathcal{EF}_{\mathsf{p},\iota}$ can include elliptic-curve hashing [Ica09]. Let $\mathcal{DF}_{\mathsf{p}}$ be a family of distributions over $\mathbb{F}$. Lipmaa, Parisella, and Siim [LPS23] introduced two oracles $\mathcal{S}amp_1$ and $\mathcal{S}amp_2$, one for each $\mathbb{G}_1$ and $\mathbb{G}_2$. The ith query (E, D) to $\mathcal{S}amp_\iota$ consists of a function $E \in \mathcal{EF}_{\mathsf{p},\iota}$ and a distribution $D \in \mathcal{DF}_{\mathsf{p}}$. The $\mathcal{S}amp_\iota$ oracle samples a random field element $s_i \leftarrow\!\!\$\ D$, computes $[\mathfrak{q}_{\iota_i}]_\iota \leftarrow E(s_i)$, stores $[\mathfrak{q}_{\iota_i}]_\iota$ in the table (with a self-referential explanation $(0, \ldots, 0, 1)$), and returns a field element s_i. One also keeps track of the number of $\mathcal{S}amp_\iota$ queries il_ι at any moment. The only difference between the AGMOS and the AGM is that in the AGMOS, the adversary has access to $\mathcal{S}amp_1$ and $\mathcal{S}amp_2$. More precisely, an AGMOS adversary has oracle access to Let $\Pi_{\mathcal{O}} = \{\mathcal{O}p, \mathcal{P}air, \mathcal{E}q, \mathcal{E}ncode, \mathcal{S}amp_1, \mathcal{S}amp_2\}$.

AGMOS proofs use the already described extractor $\mathsf{Ext}^{\mathcal{A}}$. Moreover, the proofs rely on the following two assumptions, also used in the current work. (The assumptions do not depend on the details of [JM24].)

Let $d_1(\lambda), d_2(\lambda) \in \mathsf{poly}(\lambda)$. Pgen is $(d_1(\lambda), d_2(\lambda))$-*PDL (Power Discrete Logarithm, [Lip12]) secure* if for any non-uniform PPT $\mathcal{A}$, $\mathsf{Adv}^{\mathrm{pdl}}_{d_1,d_2,\mathsf{Pgen},\mathcal{A}}(\lambda) :=$

$$\Pr\left[\mathcal{A}(\mathsf{p}, [(x^i)_{i=0}^{d_1}]_1, [(x^i)_{i=0}^{d_2}]_2) = x \mid \mathsf{p} \leftarrow \mathsf{Pgen}(1^\lambda), x \leftarrow\!\!\$\ \mathbb{F}\right] \approx_\lambda 0 \ .$$

Let $\mathcal{EF}$ be a family of functions and $\mathcal{DF}$ a family of distributions. We say that Pgen is $(\mathcal{EF}, \mathcal{DF})$-TOFR (*Tensor Oracle FindRep*, [LPS23]) secure if for any PPT $\mathcal{A}$, $\mathsf{Adv}^{\mathrm{tofr}}_{\mathsf{Pgen},\mathcal{A}}(\lambda) :=$

$$\Pr\left[\boldsymbol{v} \neq \mathbf{0} \wedge \boldsymbol{v}^{\mathsf{T}} \cdot \begin{pmatrix} 1 \\ \mathfrak{q}_1 \\ \mathfrak{q}_2 \\ \mathfrak{q}_1 \otimes \mathfrak{q}_2 \end{pmatrix} = 0 \,\middle|\, \mathsf{p} \leftarrow \mathsf{Pgen}(1^\lambda); \boldsymbol{v} \leftarrow \mathcal{A}^{\mathcal{S}amp_1, \mathcal{S}amp_2}(\mathsf{p}) \right] \approx_\lambda 0 \ .$$

Depending on the application, one can restrict the family of distributions. The smaller $\mathcal{DF}$ is, the weaker TOFR will become. See [LPS23] for details.

A *reduction* $\mathcal{B}$ from an assumption A to an assumption B is a map that maps an A-adversary $\mathcal{A}$ to a B-adversary $\mathcal{B}^{\mathcal{A}}$. $\mathcal{B}$ is *model-preserving* [JM24, Definition 9], if $\mathcal{B}^{\mathcal{A}}$ is a type-safe (i.e. Maurer) generic adversary whenever $\mathcal{A}$ is one. That is, $\mathcal{B}$ does not use $\mathcal{E}ncode$ (or $\mathcal{D}ecode$). $\mathcal{B}$ is *efficient* if (1) it runs in time polynomial in the time of $\mathcal{A}$, and (2) $\mathcal{B}^{\mathcal{A}}$'s success in breaking B is at most a constant-times different from the success of $\mathcal{A}$ in breaking A. Jaeger and Mohan [JM24, Theorem 9] proved the following result, which we state informally.

Fact 1 *If a model-preserving and efficient algebraic reduction exists from assumption A to B and B is generically hard, then A is generically hard.*

Acknowledgments. We thank Antonio Faonio and Joseph Jaeger for useful discussions about [FFR24,JM24], respectively. Lipmaa and Siim were co-funded by the European Union and Estonian Research Council via grant PRG2531 and project TEM-TA119.

References

AFK22. Thomas Attema, Serge Fehr, and Michael Klooß. Fiat-shamir transformation of multi-round interactive proofs. In Eike Kiltz and Vinod Vaikuntanathan, editors, *TCC 2022, Part I*, volume 13747 of *LNCS*, pages 113–142. Springer, Cham, November 2022. doi:10.1007/978-3-031-22318-1_5. 1, E, E.1, 7, E.1, E.2

AFKR23. Thomas Attema, Serge Fehr, Michael Klooß, and Nicolas Resch. The Fiat–Shamir transformation of $(\Gamma_1, \ldots, \Gamma_\mu)$-special-sound interactive proofs. Cryptology ePrint Archive, Report 2023/1945, 2023. URL: https://eprint.iacr.org/2023/1945. 1.1

BB08. Dan Boneh and Xavier Boyen. Short signatures without random oracles and the SDH assumption in bilinear groups. *Journal of Cryptology*, 21(2):149–177, April 2008. doi:10.1007/s00145-007-9005-7. B

BBB+18. Benedikt Bünz, Jonathan Bootle, Dan Boneh, Andrew Poelstra, Pieter Wuille, and Greg Maxwell. Bulletproofs: Short proofs for confidential transactions and more. In *2018 IEEE Symposium on Security and Privacy*, pages 315–334. IEEE Computer Society Press, May 2018. doi:10.1109/SP.2018.00020. 2.1

BBHR18. Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. Fast reed-solomon interactive oracle proofs of proximity. In Ioannis Chatzigiannakis, Christos Kaklamanis, Dániel Marx, and Donald Sannella, editors, *ICALP 2018*, volume 107 of *LIPIcs*, pages 14:1–14:17. Schloss Dagstuhl, July 2018. doi:10.4230/LIPIcs.ICALP.2018.14. 1, 2.1

BFHK24. Balthazar Bauer, Pooya Farshim, Patrick Harasser, and Markulf Kohlweiss. The uber-knowledge assumption: A bridge to the AGM. *CiC*, 1(3):31, 2024. doi:10.62056/anr-zoja5. 1

BFS20. Benedikt Bünz, Ben Fisch, and Alan Szepieniec. Transparent SNARKs from DARK compilers. In Anne Canteaut and Yuval Ishai, editors, *EUROCRYPT 2020, Part I*, volume 12105 of *LNCS*, pages 677–706. Springer, Cham, May 2020. doi:10.1007/978-3-030-45721-1_24. 2.1

CDS94. Ronald Cramer, Ivan Damgård, and Berry Schoenmakers. Proofs of partial knowledge and simplified design of witness hiding protocols. In Yvo Desmedt, editor, *CRYPTO'94*, volume 839 of *LNCS*, pages 174–187. Springer, Berlin, Heidelberg, August 1994. doi:10.1007/3-540-48658-5_19. 1

CFF+21. Matteo Campanelli, Antonio Faonio, Dario Fiore, Anaïs Querol, and Hadrián Rodríguez. Lunar: A toolbox for more efficient universal and updatable zkSNARKs and commit-and-prove extensions. In Mehdi Tibouchi and Huaxiong Wang, editors, *ASIACRYPT 2021, Part III*, volume 13092 of *LNCS*, pages 3–33. Springer, Cham, December 2021. doi:10.1007/978-3-030-92078-4_1. 1, 1.1, 3.1

CFF+24. Matteo Campanelli, Antonio Faonio, Dario Fiore, Tianyu Li, and Helger Lipmaa. Lookup arguments: Improvements, extensions and applications to zero-knowledge decision trees. In Qiang Tang and Vanessa Teague, editors, *PKC 2024, Part II*, volume 14602 of *LNCS*, pages 337–369. Springer, Cham, April 2024. doi:10.1007/978-3-031-57722-2_11. 1, 1.1

CGH98. Ran Canetti, Oded Goldreich, and Shai Halevi. The random oracle methodology, revisited (preliminary version). In *30th ACM STOC*, pages 209–218. ACM Press, May 1998. doi:10.1145/276698.276741. 1

CHM+20. Alessandro Chiesa, Yuncong Hu, Mary Maller, Pratyush Mishra, Psi Vesely, and Nicholas P. Ward. Marlin: Preprocessing zkSNARKs with universal and updatable SRS. In Anne Canteaut and Yuval Ishai, editors, *EUROCRYPT 2020, Part I*, volume 12105 of *LNCS*, pages 738–768. Springer, Cham, May 2020. doi:10.1007/978-3-030-45721-1_26. 1, 1.1, 1.1, 2.1, 3.1, 3.2, 3.2

Den02. Alexander W. Dent. Adapting the weaknesses of the random oracle model to the generic group model. In Yuliang Zheng, editor, *ASIACRYPT 2002*, volume 2501 of *LNCS*, pages 100–109. Springer, Berlin, Heidelberg, December 2002. doi:10.1007/3-540-36178-2_6. 1

DG23. Quang Dao and Paul Grubbs. Spartan and bulletproofs are simulation-extractable (for free!). In Carmit Hazay and Martijn Stam, editors, *EUROCRYPT 2023, Part II*, volume 14005 of *LNCS*, pages 531–562. Springer, Cham, April 2023. doi:10.1007/978-3-031-30617-4_18. 1.1

DL08. Giovanni Di Crescenzo and Helger Lipmaa. Succinct NP Proofs from an Extractability Assumption. In Arnold Beckmann, Costas Dimitracopoulos, and Benedikt Löwe, editors, *Computability in Europe, CIE 2008*, volume 5028 of *LNCS*, pages 175–185, Athens, Greece, June 15–20, 2008. Springer, Heidelberg. 1

EFG22. Liam Eagen, Dario Fiore, and Ariel Gabizon. cq: Cached quotients for fast lookups. Cryptology ePrint Archive, Report 2022/1763, 2022. URL: https://eprint.iacr.org/2022/1763. 1, 1.1

FFR24. Antonio Faonio, Dario Fiore, and Luigi Russo. Real-world universal zkSNARKs are non-malleable. In Bo Luo, Xiaojing Liao, Jun Xu, Engin Kirda, and David Lie, editors, *ACM CCS 2024*, pages 3138–3151. ACM Press, October 2024. doi:10.1145/3658644.3690351. 1, 1, 1.1, 4, 3.2, 3.2, 3.2, 5, 3.3, 6, A, C

FKL18. Georg Fuchsbauer, Eike Kiltz, and Julian Loss. The algebraic group model and its applications. In Hovav Shacham and Alexandra Boldyreva, editors, *CRYPTO 2018, Part II*, volume 10992 of *LNCS*, pages 33–62. Springer, Cham, August 2018. doi:10.1007/978-3-319-96881-0_2. 3, 2.1, 4, 6

GKM+18. Jens Groth, Markulf Kohlweiss, Mary Maller, Sarah Meiklejohn, and Ian Miers. Updatable and universal common reference strings with applications to zk-SNARKs. In Hovav Shacham and Alexandra Boldyreva, editors, *CRYPTO 2018, Part III*, volume 10993 of *LNCS*, pages 698–728. Springer, Cham, August 2018. doi:10.1007/978-3-319-96878-0_24. 1

GKP22. Chaya Ganesh, Hamidreza Khoshakhlagh, and Roberto Parisella. NIWI and new notions of extraction for algebraic languages. In Clemente Galdi and Stanislaw Jarecki, editors, *SCN 22*, volume 13409 of *LNCS*, pages 687–710. Springer, Cham, September 2022. doi:10.1007/978-3-031-14791-3_30. 1.1

GLS+23. Alexander Golovnev, Jonathan Lee, Srinath T. V. Setty, Justin Thaler, and Riad S. Wahby. Brakedown: Linear-time and field-agnostic SNARKs for R1CS. In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part II*, volume 14082 of *LNCS*, pages 193–226. Springer, Cham, August 2023. doi:10.1007/978-3-031-38545-2_7. 1

GR19. Alonso González and Carla Ràfols. Shorter pairing-based arguments under standard assumptions. In Steven D. Galbraith and Shiho Moriai, editors, *ASIACRYPT 2019, Part III*, volume 11923 of *LNCS*, pages 728–757. Springer, Cham, December 2019. doi:10.1007/978-3-030-34618-8_25. 2

Gro10. Jens Groth. Short pairing-based non-interactive zero-knowledge arguments. In Masayuki Abe, editor, *ASIACRYPT 2010*, volume 6477 of *LNCS*, pages 321–340. Springer, Berlin, Heidelberg, December 2010. doi:10.1007/978-3-642-17373-8_19. 1

Gro16. Jens Groth. On the size of pairing-based non-interactive arguments. In Marc Fischlin and Jean-Sébastien Coron, editors, *EUROCRYPT 2016, Part II*, volume 9666 of *LNCS*, pages 305–326. Springer, Berlin, Heidelberg, May 2016. doi:10.1007/978-3-662-49896-5_11. 1

GW11. Craig Gentry and Daniel Wichs. Separating succinct non-interactive arguments from all falsifiable assumptions. In Lance Fortnow and Salil P. Vadhan, editors, *43rd ACM STOC*, pages 99–108. ACM Press, June 2011. doi:10.1145/1993636.1993651. 1

GW20a. Ariel Gabizon and Zachary J. Williamson. plookup: A simplified polynomial protocol for lookup tables. Cryptology ePrint Archive, Report 2020/315, 2020. URL: https://eprint.iacr.org/2020/315. 1

GW20b. Ariel Gabizon and Zachary J. Williamson. Proposal: The Turbo-PLONK program syntax for specifying SNARK programs. 2020. URL: https://docs.zkproof.org/pages/standards/accepted-workshop3/proposal-turbo_plonk.pdf. 1, 1.1

GWC19. Ariel Gabizon, Zachary J. Williamson, and Oana Ciobotaru. PLONK: Permutations over Lagrange-bases for oecumenical noninteractive arguments of knowledge. Cryptology ePrint Archive, Report 2019/953, 2019. URL: https://eprint.iacr.org/2019/953. 1, 1, 1.1, 1.1, 2, 3.1, 3.2, 5, 5.1, 5.1, 5.1, 5.2, D.1, E.2, F

Ica09. Thomas Icart. How to hash into elliptic curves. In Shai Halevi, editor, *CRYPTO 2009*, volume 5677 of *LNCS*, pages 303–316. Springer, Berlin, Heidelberg, August 2009. doi:10.1007/978-3-642-03356-8_18. 1, 6

JM24. Joseph Jaeger and Deep Inder Mohan. Generic and algebraic computation models: When AGM proofs transfer to the GGM. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part V*, volume 14924 of *LNCS*, pages 14–45. Springer, Cham, August 2024. doi:10.1007/978-3-031-68388-6_2. 1, 5, 1.1, 6, 6, 6, B, B, C

JR10. Tibor Jager and Andy Rupp. The semi-generic group model and applications to pairing-based cryptography. In Masayuki Abe, editor, *ASIACRYPT 2010*, volume 6477 of *LNCS*, pages 539–556. Springer, Berlin, Heidelberg, December 2010. doi:10.1007/978-3-642-17373-8_31. 6

KRS25. Dmitry Khovratovich, Ron D. Rothblum, and Lev Soukhanov. How to Prove False Statements: Practical Attacks on Fiat-Shamir. Technical Report 2025/118, IACR, January 25, 2025. URL: https://eprint.iacr.org/2025/118. 1

KZG10. Aniket Kate, Gregory M. Zaverucha, and Ian Goldberg. Constant-size commitments to polynomials and their applications. In Masayuki Abe, editor, *ASIACRYPT 2010*, volume 6477 of *LNCS*, pages 177–194. Springer, Berlin, Heidelberg, December 2010. doi:10.1007/978-3-642-17373-8_11. 1, 2.1, 2.1, 6

Lip12. Helger Lipmaa. Progression-free sets and sublinear pairing-based non-interactive zero-knowledge arguments. In Ronald Cramer, editor, *TCC 2012*, volume 7194 of *LNCS*, pages 169–189. Springer, Berlin, Heidelberg, March 2012. doi:10.1007/978-3-642-28914-9_10. 2.1, 6, B

Lip24. Helger Lipmaa. Polymath: Groth16 is not the limit. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part X*, volume 14929 of *LNCS*, pages 170–206. Springer, Cham, August 2024. doi:10.1007/978-3-031-68403-6_6. 1

LPS23. Helger Lipmaa, Roberto Parisella, and Janno Siim. Algebraic group model with oblivious sampling. In Guy N. Rothblum and Hoeteck Wee, editors, *TCC 2023, Part IV*, volume 14372 of *LNCS*, pages 363–392. Springer, Cham, November / December 2023. doi:10.1007/978-3-031-48624-1_14. 1, 1, 1.1, 1.1, 2.1, 3.2, 3.3, 6, 6, A, B, B

LPS24. Helger Lipmaa, Roberto Parisella, and Janno Siim. Constant-size zk-SNARKs in ROM from falsifiable assumptions. In Marc Joye and Gregor Leander, editors, *EUROCRYPT 2024, Part VI*, volume 14656 of *LNCS*, pages 34–64. Springer, Cham, May 2024. doi:10.1007/978-3-031-58751-1_2. 1, 1.1, 1.1, 1, 2, 2, 2.1, 2.1, 3, E.1, E.1, 8

LSZ22. Helger Lipmaa, Janno Siim, and Michal Zajac. Counting vampires: From univariate sumcheck to updatable ZK-SNARK. In Shweta Agrawal and Dongdai Lin, editors, *ASIACRYPT 2022, Part II*, volume 13792 of *LNCS*, pages 249–278. Springer, Cham, December 2022. doi:10.1007/978-3-031-22966-4_9. 1, 1.1, 3.1

Mau05. Ueli M. Maurer. Abstract models of computation in cryptography (invited paper). In Nigel P. Smart, editor, *10th IMA International Conference on Cryptography and Coding*, volume 3796 of *LNCS*, pages 1–12. Springer, Berlin, Heidelberg, December 2005. doi:10.1007/11586821_1. 3, 6

Mic94. Silvio Micali. CS proofs (extended abstracts). In *35th FOCS*, pages 436–453. IEEE Computer Society Press, November 1994. doi:10.1109/SFCS.1994.365746. 1

PST13. Charalampos Papamanthou, Elaine Shi, and Roberto Tamassia. Signatures of correct computation. In Amit Sahai, editor, *TCC 2013*, volume 7785 of *LNCS*, pages 222–242. Springer, Berlin, Heidelberg, March 2013. doi:10.1007/978-3-642-36594-2_13. 2.1

RZ21. Carla Ràfols and Arantxa Zapico. An algebraic framework for universal and updatable SNARKs. In Tal Malkin and Chris Peikert, editors, *CRYPTO 2021, Part I*, volume 12825 of *LNCS*, pages 774–804, Virtual Event, August 2021. Springer, Cham. doi:10.1007/978-3-030-84242-0_27. 1, 1.1, 3.1

Sef24. Marek Sefranek. How (not) to simulate PLONK. In Clemente Galdi and Duong Hieu Phan, editors, *SCN 24, Part I*, volume 14973 of *LNCS*, pages 96–117. Springer, Cham, September 2024. doi:10.1007/978-3-031-71070-4_5. 1.1, F, 9, F

Sho97. Victor Shoup. Lower bounds for discrete logarithms and related problems. In Walter Fumy, editor, *EUROCRYPT'97*, volume 1233 of *LNCS*, pages 256–266. Springer, Berlin, Heidelberg, May 1997. doi:10.1007/3-540-69053-0_18. 3, 6

STW24. Srinath T. V. Setty, Justin Thaler, and Riad S. Wahby. Unlocking the lookup singularity with Lasso. In Marc Joye and Gregor Leander, editors, *EUROCRYPT 2024, Part VI*, volume 14656 of *LNCS*, pages 180–209. Springer, Cham, May 2024. doi:10.1007/978-3-031-58751-1_7. 1

Zha22. Mark Zhandry. To label, or not to label (in generic groups). In Yevgeniy Dodis and Thomas Shrimpton, editors, *CRYPTO 2022, Part III*, volume 13509 of *LNCS*, pages 66–96. Springer, Cham, August 2022. doi:10.1007/978-3-031-15982-4_3. 1, 1, 6

ZZK22. Cong Zhang, Hong-Sheng Zhou, and Jonathan Katz. An analysis of the algebraic group model. In Shweta Agrawal and Dongdai Lin, editors, *ASIACRYPT 2022, Part IV*, volume 13794 of *LNCS*, pages 310–322. Springer, Cham, December 2022. doi:10.1007/978-3-031-22972-5_11. 1, 6

A Insecurity of LinTrick

For the sake of completeness, we reproduce the attack of [LPS23,FFR24] against knowledge-soundness of LinTrick.

The adversary $\mathcal{A}$ chooses some tuple $\{\mathsf{a}_\imath(X)\}$ of polynomials, such that $V := \{\mathsf{g}_t(\mathbf{a}(X))\}_{t=1}^{n_b}$ is linearly dependent. $\mathcal{A}$ chooses a non-zero vector $\bar{\boldsymbol{d}}^*$ that is orthogonal to V. That is, $\sum_{t=1}^{n_b} \mathsf{g}_t(\mathbf{a}(X))\bar{d}_t^* = 0$. $\mathcal{A}$ samples $[D]_1 \leftarrow_\$ \mathbb{G}_1$ obliviously, and sets $[d_t]_1 \leftarrow \bar{d}_t^*[D]_1$ for all t. $\mathcal{A}$ sets $\bar{a}_\imath \leftarrow \mathsf{a}_\imath(\mathfrak{z})$, for all $\imath$, and $[h]_1 \leftarrow [(\sum_{\imath=1}^{n_a} \beta^{\imath-1}(\mathsf{a}_\imath(x) - \bar{a}_\imath))/(x-\mathfrak{z})]_1$. Clearly,

$$\sum_{t=1}^{n_b} \mathsf{g}_t(\bar{\boldsymbol{a}})d_t = \sum_{t=1}^{n_b} \mathsf{g}_t(\mathbf{a}(\mathfrak{z}))\bar{d}_t^* D = \left(\sum_{t=1}^{n_b} \mathsf{g}_t(\mathbf{a}(\mathfrak{z}))\bar{d}_t^*\right) D = 0 \ ,$$

and thus

$$\left[\sum_{\imath=1}^{n_a} \beta^{\imath-1}(\mathsf{a}_\imath(x) - \bar{a}_\imath) + \beta^{n_a} \sum_{t=1}^{n_b} \mathsf{g}_t(\bar{\boldsymbol{a}})d_t\right]_1 \bullet [1]_2 = \left[\sum_{\imath=1}^{n_a} \beta^{\imath-1}(\mathsf{a}_\imath(x) - \bar{a}_\imath)\right]_1$$

$$= [h]_1 \bullet [x - \mathfrak{z}]_2 \ .$$

Thus, the LinTrick verifier accepts, but it is impossible to extract from obliviously sampled $[D]_1$ and thus from any of $[d_t]_1$.

B Security of SplitRSDH in the AGMOS

Next, we prove that par-SplitRSDH is secure in the AGMOS (AGM with oblivious sampling, [LPS23]). We give the proof in the new oracle framework of Section 6. Because of Fact 1 in Section 6, the reduction implies that SplitRSDH is generically hard.

Theorem 7. *As in Definition 3, let* $\mathsf{par} = (m, n_\psi, n_1, n_\mathcal{S}, (\psi_k(X))_{k=1}^m)$. *Assume* $m, n_\psi, n_1, n_\mathcal{S} \in \mathsf{poly}(\lambda)$ *are positive integers so that* $n_\mathcal{S} > n_1 + n_\psi + 1$. *Assume* $\psi_k(X) \in \mathbb{F}_{\leq n_\psi}[X]$. *Fix* $\mathcal{EF}$ *and* $\mathcal{DF}$. *If the* $(n_1, 1)$*-PDL and* $(\mathcal{EF}, \mathcal{DF})$*-TOFR assumptions hold, then* par*-SplitRSDH holds in the AGMOS. More precisely, for every PPT* $\mathcal{A}$*, there exist PPT* $\mathcal{B}_{\mathrm{PDL}}$ *and PPT* $\mathcal{C}_{\mathrm{TOFR}}$*, such that*

$$\mathsf{Adv}^{\mathrm{splitrsdh}}_{\mathsf{Pgen},\mathsf{par},\mathcal{A}}(\lambda) \leq \mathsf{Adv}^{\mathrm{pdl}}_{n_1,1,\mathsf{Pgen},\mathcal{B}_{\mathrm{PDL}}}(\lambda) + \mathsf{Adv}^{\mathrm{tofr}}_{\mathsf{Pgen},\mathcal{C}_{\mathrm{TOFR}}}(\lambda) \ .$$

As usual, AGM and AGMOS proofs require many boilerplate steps to construct formal reductions. For ease of parsing, we have marked one paragraph of the following proof as the "meat of the analysis": this is essentially the only part that deeply depends on the concrete assumption. (Alternatively, it is a more formal rewording of the intuition we gave just after Definition 3.) Readers familiar with AGM(OS) proofs may skip other parts of the proof.

Proof. Let $\mathcal{A}$ be an $(\mathcal{EF}, \mathcal{DF})$-AGMOS adversary against the SplitRSDH assumption that succeeds with some non-negligible probability ε. In Fig. 14, we depict a PDL adversary $\mathcal{B}_{\mathrm{PDL}}$ and a TOFR adversary $\mathcal{C}_{\mathrm{TOFR}}$, such that $\Pr[\mathcal{A} \text{ is successful}] \leq \Pr[\mathcal{B}_{\mathrm{PDL}} \text{ is successful} \mid E] + \Pr[\mathcal{C}_{\mathrm{TOFR}} \text{ is successful} \mid \neg E]$ for the event E that the check on line 12 succeeds.

More precisely, $\mathcal{B}_{\mathrm{PDL}}$ obtains an input ck while $\mathcal{C}_{\mathrm{TOFR}}$ samples ck itself. In the oracle framework of [JM24], it means $\mathcal{B}_{\mathrm{PDL}}$ can only use oracles to access ck while $\mathcal{C}_{\mathrm{TOFR}}$ knows x and can use it in computation. According to Section 6, there exists an extractor $\mathsf{Ext}^{\mathcal{A}}$ who outputs the explanations behind $\mathcal{A}$'s output group elements. More precisely, $\mathsf{Ext}^{\mathcal{A}}(\mathsf{p}, \mathsf{ck})$ prepares the tables **Val** and **Expl** for $\mathcal{A}$, then calls $\mathcal{A}$, and then returns an output Vals what a non-algebraic $\mathcal{A}$ would return together with underlying explanations Expl. Both reductions call $\mathsf{Ext}^{\mathcal{A}}$, obtaining its output. We use the explanations to define a polynomial $V(X, \mathfrak{Q})$, where $\mathfrak{Q}$ is the vector of indeterminates corresponding to sampling oracle queries. This polynomial is defined so that $V(x, \mathfrak{q}) = 0$ iff the SplitRSDH adversary succeeds in satisfying the last verification in Definition 3, that is,

$$\sum_{k=1}^{m} ([\tau_k]_1 \bullet [\psi_k(x)]_2) = [1]_1 \bullet [L(x)]_2 + [\varphi]_1 \bullet [\mathsf{Z}_\mathcal{S}(x)]_2 \ .$$

Next, we give a detailed description of the reductions and analyze their success probabilities. Consider the reduction $\mathcal{B}_{\mathrm{PDL}}$. (The analysis of $\mathcal{C}_{\mathrm{TOFR}}$ differs minimally.) In line 2, $\mathcal{B}_{\mathrm{PDL}}$ first calls $\mathsf{Ext}^{\mathcal{A}}$ with correct input. Recall from Definition 3 that

$$\psi_k(X) \in \mathbb{F}_{\leq n_\psi}[X] \text{ and } n_\mathcal{S} > n_1 + n_\psi + 1 \ .$$

The algebraic adversary $\mathcal{A}$, invoked by $\mathsf{Ext}^{\mathcal{A}}$, has access to $\Pi_\mathcal{O}$ that includes the sampling oracles. Let $\mathsf{bad}_\mathcal{A}$ be the event that $\mathcal{A}$ succeeds in breaking SplitRSDH. (By this we mean that

$\boxed{\mathcal{B}_{\mathrm{PDL}}(\mathsf{p}, \mathsf{ck} = ([1, x, \ldots, x^{n_1}]_1, [1, x]_2))}$: $\boxed{\mathcal{C}_{\mathrm{TOFR}}(\mathsf{p})}$:

1 : $\boxed{x \leftarrow\!\$\, \mathbb{F}; \mathsf{ck} \leftarrow ([1, x, \ldots, x^{n_1}]_1, [1, x]_2);}$ // Only in the TOFR reduction:
2 : $(\mathsf{Vals} = (\mathcal{S} = \{\mathfrak{z}_i\}_{i=1}^{n_S}, L(X), [(\tau_k)_{k=1}^m, \varphi]_1); \mathsf{Expl}) \leftarrow \mathsf{Ext}^{\mathcal{A}}(\mathsf{p}, \mathsf{ck})$;
3 : Parse Expl as $((\{\tau'_k(X), \hat{\boldsymbol{\tau}}_k\}_{k=1}^m, (\varphi'(X), \hat{\boldsymbol{\varphi}})))$;
4 : // Defining verification polynomial based on extractor's output
5 : **for** $k \in [1, m]$ **do** $\tau_k(X, \mathbb{Q}) := \tau'_k(X) + \hat{\boldsymbol{\tau}}_k^{\mathsf{T}} \mathbb{Q}$;
6 : $\varphi(X, \mathbb{Q}) := \varphi'(X) + \hat{\boldsymbol{\varphi}}^{\mathsf{T}} \mathbb{Q}$;
7 : $V_0(X) := \sum_{k=1}^{m} \tau'_k(X)\psi_k(X) - L(X) - \varphi'(X)\mathsf{Z}_{\mathcal{S}}(X)$;
8 : **for** $j \in [1, \mathsf{il}]$ **do** $V_j(X) := \sum_{k=1}^{m} \hat{\tau}_{kj}\psi_k(X) - \hat{\varphi}_j(X)\mathsf{Z}_{\mathcal{S}}(X)$;
9 : $V(X, \mathbb{Q}) := V_0(X) + \sum_{j=1}^{\mathsf{il}} V_j(X)\mathbb{Q}_j$;
10 : **if** $V(X, \mathbb{Q}) = 0$ **then return** $\perp$;
11 : // Finishing off either PDL or TOFR reduction
12 : **if** $\boxed{\forall j \in [0, \mathsf{il}].[V_j(x)]_T = [0]_T}$ **then** // PDL Reduction
13 : Let $j_0 \in [0, \mathsf{il}]$ be s.t. $(V_{j_0}(X) \neq 0)$;
14 : $(x_1, \ldots, x_{n_1}) \leftarrow \mathbf{Solve}(V_{j_0}(X) = 0)$;
15 : **for** $i \in [1, n_1]$ **do if** $\boxed{[x]_1 = x_i[1]_1}$ **then return** x_i;
16 : **else** // TOFR Reduction
17 : **for** $j \in [1, \mathsf{il}]$ **do** $v_j \leftarrow V_j(x)$;
18 : **return** $\boldsymbol{v}$;

Fig. 14. The PDL and TOFR reductions in the proof of Theorem 7. We have boxed the lines where the reduction handles group elements.

Vals, returned by $\mathsf{Ext}^{\mathcal{A}}$ correspond to a break of SplitRSDH.) Thus, $\Pr[\mathsf{bad}_{\mathcal{A}}] = \varepsilon \neq \mathsf{negl}(\lambda)$. That is, according to Definition 3, the following holds with probability ε:

$$(\mathcal{S} \subset \mathbb{F}) \wedge (|\mathcal{S}| = n_{\mathcal{S}}) \wedge (L(X) \in \mathbb{F}[X]) \wedge (L(X) \in [n_1 + n_\psi + 1, n_{\mathcal{S}} - 1]) \wedge$$
$$\left(\boxed{\textstyle\sum_{k=1}^{m} [\tau_k \psi_k(x)]_1 = [L(x)]_1 + [\varphi \mathsf{Z}_{\mathcal{S}}(x)]_1}\right)$$

Since SplitRSDH is not publicly verifiable, a model-preserving reduction cannot test whether $\mathsf{bad}_{\mathcal{A}}$ holds (more precisely, it cannot test that the boxed equality holds). However, this is unimportant since we show that either $\mathcal{B}_{\mathrm{PDL}}$ or $\mathcal{C}_{\mathrm{TOFR}}$ succeeds with high probability whenever $\mathsf{bad}_{\mathcal{A}} = \mathsf{true}$.

Assume now $\mathsf{bad}_{\mathcal{A}} = \mathsf{true}$. $\mathsf{Ext}^{\mathcal{A}}(\mathsf{p}, \mathsf{ck})$ outputs the values on line 2, such that $\tau'_k(X), \varphi'(X) \in \mathbb{F}_{\leq n_1}[X]$ and $\hat{\tau}_{kj}, \hat{\varphi}_j \in \mathbb{F}$. As obvious from Section 6, if $\mathsf{bad}_{\mathcal{A}} = \mathsf{true}$, the extraction always aucceeds, that is, for all k, $[\tau_k]_1 = [\tau'_k(x)]_1 + \hat{\boldsymbol{\tau}}_k^{\mathsf{T}}[\mathfrak{q}]_1$ and $[\varphi]_1 = [\varphi'(x)]_1 + \hat{\boldsymbol{\varphi}}^{\mathsf{T}}[\mathfrak{q}]_1$. Here, $\mathfrak{q}$ is a vector of discrete logarithms of the answers of the oblivious sampling oracle $\mathcal{S}amp_1$. (Since $\mathcal{A}$ does not output $\mathbb{G}_2$ elements, we ignore the oracle $\mathcal{S}amp_2$.) As explained earlier, this extractor has access to the tables **Val** and **Expl** and does not otherwise depend on $\mathcal{A}$. Clearly, if $\mathcal{A}$ is model-preserving, then so is Ext: $\mathsf{Ext}^{\mathcal{A}}$ never applies $\mathcal{E}ncode$ or $\mathcal{D}ecode$ to any group elements unless $\mathcal{A}$ does.

Next, given the extractor's outputs, $\mathcal{B}_{\mathrm{PDL}}$ defines verification polynomials $V(X, \mathbb{Q})$ and $V_j(X)$, such that

$$V(X, \mathbb{Q}) := V_0(X) + \sum_{j=1}^{\mathsf{il}} V_j(X)\mathbb{Q}_j \ .$$

Since the verifier accepts, $V(x, \mathfrak{q}) = 0$.

"Meat of the analysis." Let us show that if $\mathsf{bad}_{\mathcal{A}} = \mathsf{true}$, then $\mathcal{B}_{\mathrm{PDL}}$ and $\mathcal{C}_{\mathrm{TOFR}}$ abort on line 10 (i.e., $V(X, \mathbb{Q}) = 0$) with probability 0. Really, assume $V(X, \mathbb{Q}) = 0$ as a polynomial. Thus also $V_0(X) = 0$. Since $\tau'_k(X) \in \mathbb{F}_{\leq n_1}[X]$, $\psi_k(X) \in \mathbb{F}_{\leq n_\psi}[X]$, $n_{\mathcal{S}} > n_1 + n_\psi + 1$, and $\deg L(X) < n_{\mathcal{S}}$ (see Definition 3), it follows from $V_0(X) = \sum_{k=1}^{m} \tau'_k(X)\psi_k(X) - L(X) - \varphi'(X)\mathsf{Z}_{\mathcal{S}}(X) = 0$ (see line 7) that $\varphi'(X) = 0$. Let

$$T(X) := \sum_{k=1}^{m} \tau'_k(X)\psi_k(X) \in \mathbb{F}_{\leq n_1 + n_\psi}[X] \ .$$

Then, $T(X) = L(X)$. Since $\deg T(X) \leq n_1 + n_\psi$, we have $\deg L(X) \leq n_1 + n_\psi$. A contradiction to $\deg L(X) \geq n_1 + n_\psi + 1$. Thus, neither reduction ever aborts on line 10.

Now, we know that $V(x, \mathfrak{q}) = 0$ but $V(X, \mathbb{Q}) \neq 0$. Let E be the event that the check on line 12 succeeds, i.e., $[V_j(x)]_T = [0]_T$ for all $j \in [0, \mathsf{il}]$. If $E = \mathsf{true}$, we finish the PDL reduction. Otherwise, we finish the TOFR reduction.

$\underline{E = \mathsf{true}}$. Since $E = \mathsf{true}$, $V_j(x) = 0$ for all $j \in [0, \mathsf{il}]$. Since $V(X, \mathbb{Q}) \neq 0$, $V_{j_0}(X) \neq 0$ for some $j_0 \in [0, \mathsf{il}]$ (note that this includes $j_0 = 0$). Thus, $V_{j_0}(X) \neq 0$, but $V_{j_0}(x) = 0$. Clearly, $V_{j_0}(X)$ is univariate polynomial of degree at most n_1. Thus, it has at most n_1 roots. The PDL adversary computes x as one of the roots of $V_{j_0}(X)$.

$\underline{E = \mathsf{false}}$. Since $E = \mathsf{false}$, $\exists j \in [0, \mathsf{il}].(V_j(x) \neq 0)$ but the verifier accepts ($V(x, \mathfrak{q}) = 0$). In this case, we construct an adversary $\mathcal{C}_{\mathrm{TOFR}}$ that breaks the TOFR assumption. $\mathcal{C}_{\mathrm{TOFR}}$ starts exactly as $\mathcal{B}_{\mathrm{PDL}}$, with the only difference that $\mathcal{C}_{\mathrm{TOFR}}$ samples the trapdooe x and constructs ck itself. Knowing x, $\mathcal{C}_{\mathrm{TOFR}}$ outputs the vector $\boldsymbol{v}$ defined by setting $v_j = V_j(x)$ for $j \in [0, \mathsf{il}]$. Thus, if $E = \mathsf{false}$, $\mathcal{C}_{\mathrm{TOFR}}$ is successful in breaking the TOFR assumption.

To sum it up,

$$\Pr[\mathcal{A} \text{ is successful}] \leq \Pr[\mathcal{B}_{\mathrm{PDL}} \text{ fails} \mid E] + \Pr[\mathcal{C}_{\mathrm{TOFR}} \text{ fails} \mid \neg E] \ .$$

□

AGM to GGM Lifting. To use the lifting result of [JM24], we must show that $\mathcal{B}_{\mathrm{PDL}}$ and $\mathcal{C}_{\mathrm{TOFR}}$ are model-preserving and efficient. (We already know that PDL [Lip12] and TOFR [LPS23] are generically hard.) Assume $\mathcal{A}$ is a generic adversary. Thus, $\mathsf{Ext}^{\mathcal{A}}$ is also generic. The rest of the reductions touches the adversary's group element outputs (and, in the case of $\mathcal{B}_{\mathrm{PDL}}$, the input ck) in the following steps:

- Line 12 uses $\mathcal{O}p$ in $\mathbb{G}_2$, $\mathcal{P}air$, and $\mathcal{E}q$ in $\mathbb{G}_T$ to check if $E = \mathsf{true}$.
- Line 15 uses $\mathcal{E}q$ in $\mathbb{G}_1$ to find the correct root.

Thus, both reductions are model-preserving. Since PDL and TOFR are generically hard, we obtain that SplitRSDH is secure in Maurer's GGM.

The first use of oracles is unnecessary: there is no need to check if $E = \mathsf{true}$ or not (without this check, the success probability of the reduction can only increase). The second use is needed to solve PDL with a high probability.

More precisely, following a similar analysis as [BB08,Lip12] (where the latter analyzed n-PDL), for a generic adversary to break n-ARSDH assumptions for any $n < o(p^{1/3})$ with constant probability $\varepsilon > 0$ requires $\Omega(\sqrt{\varepsilon p/n})$ generic operations, where p is the group size. We get the same result for SplitRSDH by substituting n by $n_1 + n_\psi$.

C Knowledge Soundness of LinTrick in the AGMOS

Recall that LinTrick is defined in Fig. 4. Faonio, Fiore, and Russo proved that LinTrick is (policy-based) simulation extractable in the AGM. They claimed but did not formally prove it is secure in the AGMOS. Next, we will fill this gap and give detailed proof that LinTrick is knowledge-sound in the new oracle framework of the AGMOS, see Section 6. Since we use knowledge-soundness of interactive protocols only in this theorem, we will define it informally, requiring that for any PPT adversary $\mathcal{A}$, there exists a PPT $\mathsf{Ext}_{\mathsf{ks}}$, such that the probability that $\mathcal{A}$ succeeds in making the verifier accept but $\mathsf{Ext}_{\mathsf{ks}}$ does not extract the witness, is negligible. In the case of LinTrick, the latter means that $\mathsf{Ext}_{\mathsf{ks}}$ does not succeed in extracting all polynomials $\mathsf{a}_\imath(X)$ and $\mathsf{d}_t(X)$ that are consistent with the polynomial commitments and the openings.

The proof of Theorem 8 follows the template of the proof of Theorem 7, with the most significant change being that in the case of $V(X, \mathfrak{Q}) = 0$, we now construct a knowledge-soundness extractor $\mathsf{Ext}_{\mathsf{ks}}$. Recall from [FFR24] that polynomials $c_t(X)$ are *n-independent*, if for any $d_t(X) \in \mathbb{F}_{\le n}[X]$, $\sum c_t(X)d_t(X) = 0$ implies $d_t = 0$ for all t [FFR24].

Theorem 8. *Let n_a, n, g_t, and $n_{\mathsf{g}} := \max_t \deg(\mathsf{g}_t(\mathbf{a}(X)))$ be such that $d^* := n_a + n_{\mathsf{g}} + n$ satisfies $d^*/|\mathbb{F}| = \mathsf{negl}(\lambda)$. Assume $\mathsf{g}_t(\boldsymbol{a}(X))$ are n-independent. Fix $\mathcal{EF}$ and $\mathcal{DF}$. If the $(n,1)$-PDL and $(\mathcal{EF},\mathcal{DF})$-TOFR assumptions hold, then LinTrick is knowledge-sound in the AGMOS. More precisely, for every PPT knowledge-soundness AGMOS adversary $\mathcal{A}$, there exist PPT $\mathsf{Ext}_{\mathsf{ks}}$, $\mathcal{B}_{\mathrm{PDL}}$, and $\mathcal{C}_{\mathrm{TOFR}}$, s.t.*

$$\mathsf{Adv}^{\mathrm{ks}}_{\mathsf{Pgen},\mathsf{par},\mathcal{A},\mathsf{Ext}_{\mathsf{ks}}}(\lambda) \le \mathsf{Adv}^{\mathrm{pdl}}_{n,1,\mathsf{Pgen},\mathcal{B}_{\mathrm{PDL}}}(\lambda) + \mathsf{Adv}^{\mathrm{tofr}}_{\mathsf{Pgen},\mathcal{C}_{\mathrm{TOFR}}}(\lambda) + d^*/|\mathbb{F}| \ .$$

Proof. Let $\mathcal{A}$ be an $(\mathcal{EF},\mathcal{DF})$-AGMOS adversary against the knowledge-soundness of LinTrick that succeeds with some non-negligible probability ε. In Fig. 15, we depict a knowledge-soundness extractor $\mathsf{Ext}_{\mathsf{ks}}$, PDL adversary $\mathcal{B}_{\mathrm{PDL}}$, and a TOFR adversary $\mathcal{C}_{\mathrm{TOFR}}$, such that $\Pr[\mathcal{A}$ is successful w.r.t. $\mathsf{Ext}_{\mathsf{ks}}] \le \Pr[\mathcal{B}_{\mathrm{PDL}}$ is successful $\mid E] + \Pr[\mathcal{C}_{\mathrm{TOFR}}$ is successful $\mid \neg E] + \mathsf{negl}(\lambda)$ for the event E that the check on line 16 succeeds.

More precisely, $\mathsf{Ext}_{\mathsf{ks}}$ and $\mathcal{B}_{\mathrm{PDL}}$ obtain an input ck while $\mathcal{C}_{\mathrm{TOFR}}$ samples ck itself. In the oracle framework of [JM24], it means $\mathsf{Ext}_{\mathsf{ks}}$ and $\mathcal{B}_{\mathrm{PDL}}$ can only use oracles to access ck while $\mathcal{C}_{\mathrm{TOFR}}$ knows x and can use it in computation. According to Section 6, there exists an extractor $\mathsf{Ext}^{\mathcal{A}}$ who outputs the explanations behind $\mathcal{A}$'s output group elements. More precisely, $\mathsf{Ext}^{\mathcal{A}}(\mathsf{p},\mathsf{ck})$ prepares the tables **Val** and **Expl** for $\mathcal{A}$, then calls $\mathcal{A}$, and then returns an output Vals what a non-algebraic $\mathcal{A}$ would return together with underlying explanations Expl. Both reductions call $\mathsf{Ext}^{\mathcal{A}}$, obtaining its output. We use the explanations to define a polynomial $V(X, \mathfrak{Q})$, where $\mathfrak{Q}$ is the vector of indeterminates corresponding to sampling oracle queries. This polynomial is defined so that $V(x, \mathfrak{q}) = 0$ iff the knowledge-soundness adversary succeeds in satisfying the verification equation in Fig. 4, that is,

$$\left[\sum_{\imath=1}^{n_a} \beta^{\imath-1}(a_\imath - \bar{a}_\imath) + \beta^{n_a}\sum_{t=1}^{n_b} \mathsf{g}_t(\bar{\boldsymbol{a}})d_t\right]_1 \bullet [1]_2 = [h]_1 \bullet [x - \mathfrak{z}]_2 \ . \tag{17}$$

Next, we give a detailed description of the extractor and reductions and analyze their success probabilities. Consider the reduction $\mathcal{B}_{\mathrm{PDL}}$. (The analysis of $\mathcal{C}_{\mathrm{TOFR}}$ differs minimally.) In line

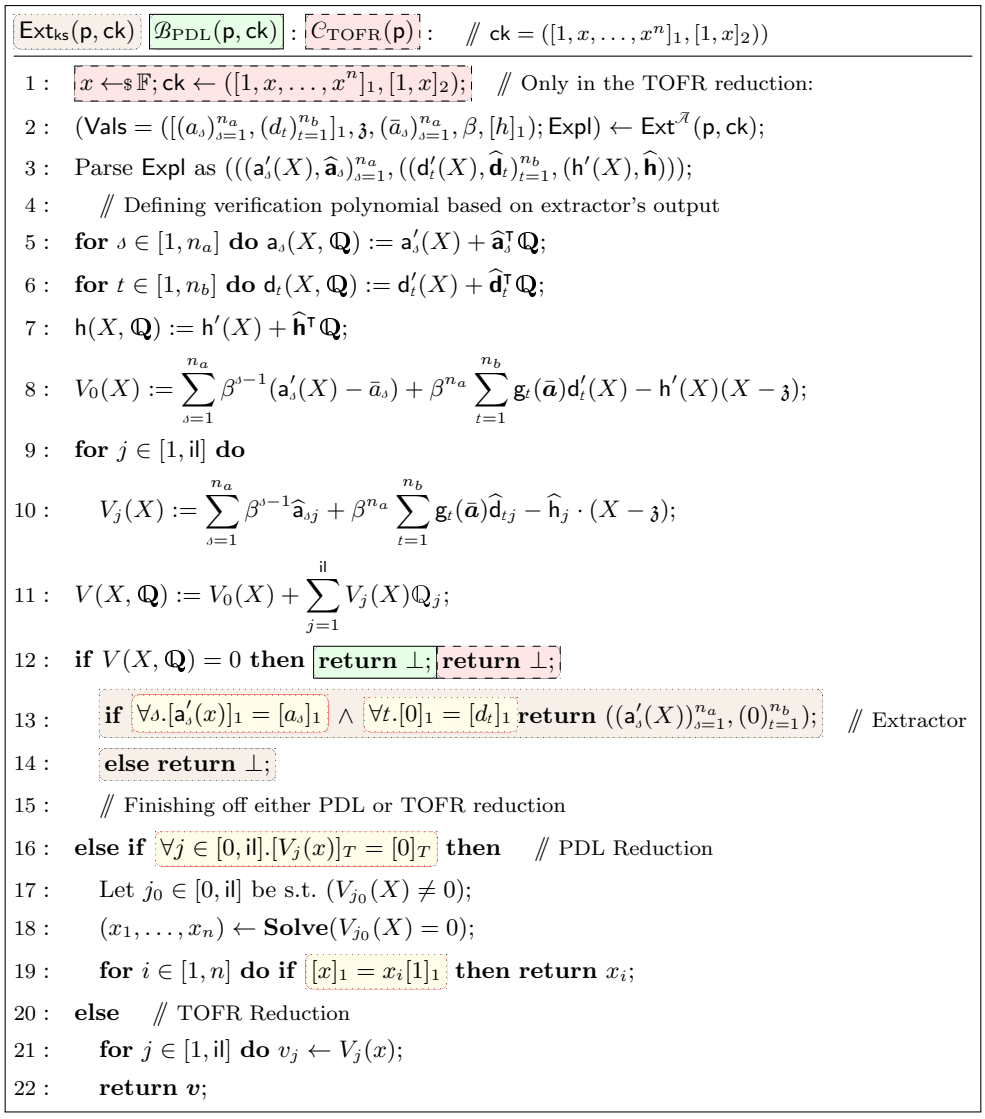

Fig. 15. The knowledge-soundness extractor and PDL and TOFR reductions in the proof of Theorem 8. We have boxed the lines where the reduction handles group elements.

2, $\mathcal{B}_{\mathrm{PDL}}$ first calls $\mathsf{Ext}^{\mathcal{A}}$ with correct input. The algebraic adversary $\mathcal{A}$, invoked by $\mathsf{Ext}^{\mathcal{A}}$, has access to $\Pi_{\mathcal{O}}$ that includes the sampling oracles. Let $\mathsf{bad}_{\mathcal{A}}$ be the event that $\mathcal{A}$ succeeds in breaking the knowledge-soundness of the LinTrick. (By this, we mean that Vals, returned by $\mathsf{Ext}^{\mathcal{A}}$ corresponds to a break of LinTrick.) Thus, $\Pr[\mathsf{bad}_{\mathcal{A}}] = \varepsilon \neq \mathsf{negl}(\lambda)$. That is, according to Fig. 4, Eq. (17) holds with probability ε, but the values output by $\mathsf{Ext}_{\mathsf{ks}}$ are incorrect.

Assume now $\mathsf{bad}_{\mathcal{A}} = \mathsf{true}$. $\mathsf{Ext}^{\mathcal{A}}(\mathsf{p},\mathsf{ck})$ outputs the values on line 2, such that $\mathsf{a}'_\imath(X), \mathsf{d}'(X)_t, \mathsf{h}'(X) \in \mathbb{F}_{\leq n}[X]$ and $\widehat{\mathsf{a}}_{\imath j}, \widehat{\mathsf{d}}_{tj}, \widehat{\mathsf{h}}_j \in \mathbb{F}$. As obvious from Section 6, if $\mathsf{bad}_{\mathcal{A}} = \mathsf{true}$, the extraction always succeeds, that is, for all $\imath$ and t, $[a_\imath]_1 = [\mathsf{a}'_\imath(x)]_1 + \widehat{\mathbf{a}}_\imath^{\mathsf{T}}[\mathbf{q}]_1$, $[d_t]_1 = [\mathsf{d}'_t(x)]_1 + \widehat{\mathbf{d}}_t^{\mathsf{T}}[\mathbf{q}]_1$, and $[h]_1 = [\mathsf{h}'(x)]_1 + \widehat{\mathbf{h}}^{\mathsf{T}}[\mathbf{q}]_1$. Here, $\mathbf{q}$ is a vector of discrete logarithms of the answers of the oblivious sampling oracle $\mathcal{S}amp_1$. (Since $\mathcal{A}$ does not output $\mathbb{G}_2$ elements, we ignore the oracle $\mathcal{S}amp_2$.) As explained earlier, this extractor has access to the tables **Val** and **Expl** and does not otherwise depend on $\mathcal{A}$. Clearly, if $\mathcal{A}$ is model-preserving, then so is Ext: $\mathsf{Ext}^{\mathcal{A}}$ never applies $\mathcal{E}ncode$ or $\mathcal{D}ecode$ to any group elements unless $\mathcal{A}$ does.

Next, given the extractor's outputs, $\mathscr{B}_{\mathrm{PDL}}$ defines verification polynomials $V(X, \mathbb{Q})$ and $V_j(X)$, such that

$$V(X, \mathbb{Q}) := V_0(X) + \sum_{j=1}^{\mathsf{il}} V_j(X)\mathbb{Q}_j \ .$$

Since the verifier accepts, $V(x, \mathfrak{q}) = 0$.

"Meat of the analysis." Let us show that if $\mathsf{bad}_{\mathscr{A}} = \mathsf{true}$, then $\mathsf{Ext}_{\mathsf{ks}}$ abort on line 14 with probability $d^*/|\mathbb{F}|$. Really, assume $V(X, \mathbb{Q}) = 0$ as a polynomial. Thus also $V_j(X) = 0$ for each $j \geq 0$.

First, consider $V_0(X) = 0$. That is, $\sum_{\jmath=1}^{n_a} \beta^{\jmath-1}(\mathsf{a}'_\jmath(X) - \bar{a}_\jmath) + \beta^{n_a} \cdot \sum_{t=1}^{n_b} \mathsf{g}_t(\bar{\boldsymbol{a}})\mathsf{d}'_t(X) - \mathsf{h}'(X)(X - \mathfrak{z}) = 0$. Setting $X = \mathfrak{z}$, we get

$$\sum_{\jmath=1}^{n_a} \beta^{\jmath-1}(\mathsf{a}'_\jmath(\mathfrak{z}) - \bar{a}_\jmath) + \beta^{n_a} \sum_{t=1}^{n_b} \mathsf{g}_t(\bar{\boldsymbol{a}})\mathsf{d}'_t(\mathfrak{z}) = 0 \ .$$

Applying the Schwartz-Zippel lemma over the random choice of β, we get that, with probability $\geq 1 - n_a/|\mathbb{F}|$, it holds that $\mathsf{a}'_\jmath(\mathfrak{z}) = \bar{a}_\jmath$ and $\sum_{t=1}^{n_b} \mathsf{g}_t(\bar{\boldsymbol{a}})\mathsf{d}'_t(\mathfrak{z}) = 0$ for all $\jmath$. Thus, $\sum_{t=1}^{n_b} \mathsf{g}_t(\mathsf{a}'(\mathfrak{z}))\mathsf{d}'_t(\mathfrak{z}) = 0$. Since $\mathfrak{z}$ is chosen uniformly at random, after applying the Schwartz-Zippel lemma once more, we get that

$$\sum_{t=1}^{n_b} \mathsf{g}_t(\mathsf{a}'(X))\mathsf{d}'_t(X) = 0 \tag{18}$$

as a polynomial with probability $1 - (n_a + n_{\mathsf{g}} + n)/|\mathbb{F}| = 1 - d^*/|\mathbb{F}|$.

Next, consider $V_j(X) = 0$ for some $j > 0$. That is, $V_j(X) := \sum_{\jmath=1}^{n_a} \beta^{\jmath-1}\widehat{\mathsf{a}}_{\jmath j} + \beta^{n_a} \sum_{t=1}^{n_b} \mathsf{g}_t(\bar{\boldsymbol{a}})\widehat{\mathsf{d}}_{tj} - \widehat{\mathsf{h}}_j \cdot (X - \mathfrak{z}) = 0$. Considering the coefficient of X, we get $\widehat{\mathsf{h}}_j = 0$. Since β is random, from Schwartz-Zippel over β we get that, with probability $\geq 1 - n_a/|\mathbb{F}|$, $\widehat{\mathsf{a}}_{\jmath j} = 0$ and $\sum_{t=1}^{n_b} \mathsf{g}_t(\bar{\mathsf{a}}'(\mathfrak{z}))\widehat{\mathsf{d}}_{tj} = \sum_{t=1}^{n_b} \mathsf{g}_t(\bar{\boldsymbol{a}})\widehat{\mathsf{d}}_{tj} = 0$. (The probability loss is counted once per the whole analysis instead of once per every j.) This holds for every j. Since $\mathfrak{z}$ is chosen uniformly at random, after applying the Schwartz-Zippel lemma once more, we get that

$$\sum_{t=1}^{n_b} \mathsf{g}_t(\mathsf{a}'(X))\widehat{\mathsf{d}}_{tj} = 0 \ . \tag{19}$$

Thus, with probability $1 - d^*/|\mathbb{F}|$, $\mathsf{a}_\jmath(X, \mathbb{Q}) = \mathsf{a}'_\jmath(X)$, $\sum_{t=1}^{n_b} \mathsf{g}_t(\mathsf{a}'(X))\mathsf{d}_t(X, \mathbb{Q}) = \sum_{t=1}^{n_b} \mathsf{g}_t(\mathsf{a}'(X))\mathsf{d}'_t(X)$, and $\mathsf{h}(X, \mathbb{Q}) = \mathsf{h}'(X)$.

Using n-independence, we get from Eq. (18) that $\mathsf{d}'_t(X) = 0$ for every t. Using n-independence (although linear independence is sufficient), we get from Eq. (19) that $\widehat{\mathsf{d}}_{tj} = 0$ for every t and j. Notably, we need to rely on n-independence since we do not have a guarantee that $\mathsf{d}'_t(X) = 0$ and $\widehat{\mathsf{d}}_{tj}(X) = 0$ for each t.

Combining the above analysis, we get that the extracted polynomials $\mathsf{a}_\jmath(X, \mathbb{Q}) = \mathsf{a}'_\jmath(X)$ and $\sum_{t=1}^{n_b} \mathsf{g}_t(\mathsf{a}'(X))d_t(X, \mathbb{Q}) = \sum_{t=1}^{n_b} \mathsf{g}_t(\mathsf{a}'(X))\mathsf{d}'_t(X)$ satisfy $[\mathsf{a}'_\jmath(x)]_1 = [a_\jmath]_1$ and $[\sum_{t=1}^{n_b} \mathsf{g}_t(\mathsf{a}'(\mathfrak{z}))\mathsf{d}'_t(\mathfrak{z})]_1 = [0]_1$. More precisely, if $V(X, \mathbb{Q}) = 0$, the latter holds with probability $1 - d^*/|\mathbb{F}|$, where $d^*/|\mathbb{F}|$ is the Schwartz-Zippel induced probability that the extractor aborts on line 14.

(The rest of the proof— except the very last line—is boilerplate and copied from the proof of Theorem 7 for completeness.) Now, we know that $V(x, \mathfrak{q}) = 0$ but $V(X, \mathbb{Q}) \neq 0$. Let E be the event that the check on line 16 succeeds, i.e., $[V_j(x)]_T = [0]_T$ for all $j \in [0, \mathsf{il}]$. If $E = \mathsf{true}$, we finish the PDL reduction. Otherwise, we finish the TOFR reduction.

$\underline{E = \mathsf{true}.}$ Since $E = \mathsf{true}$, $V_j(x) = 0$ for all $j \in [0, \mathsf{il}]$. Since $V(X, \mathfrak{Q}) \neq 0$, $V_{j_0}(X) \neq 0$ for some $j_0 \in [0, \mathsf{il}]$ (note that this includes $j_0 = 0$). Thus, $V_{j_0}(X) \neq 0$, but $V_{j_0}(x) = 0$. Clearly, $V_{j_0}(X)$ is univariate polynomial of degree at most n. Thus, it has at most n roots. The PDL adversary computes x as one of the roots of $V_{j_0}(X)$.

$\underline{E = \mathsf{false}.}$ Since $E = \mathsf{false}$, $\exists j \in [0, \mathsf{il}].(V_j(x) \neq 0)$ but the verifier accepts ($V(x, \mathfrak{q}) = 0$). In this case, we construct an adversary $\mathcal{C}_{\mathrm{TOFR}}$ that breaks the TOFR assumption. $\mathcal{C}_{\mathrm{TOFR}}$ starts exactly as $\mathcal{B}_{\mathrm{PDL}}$, with the only difference that $\mathcal{C}_{\mathrm{TOFR}}$ samples the trapdoor x and constructs ck itself. Knowing x, $\mathcal{C}_{\mathrm{TOFR}}$ outputs the vector $\boldsymbol{v}$ defined by setting $v_j = V_j(x)$ for $j \in [0, \mathsf{il}]$. Thus, if $E = \mathsf{false}$, $\mathcal{C}_{\mathrm{TOFR}}$ is successful in breaking the TOFR assumption.

To sum it up,

$$\Pr[\mathcal{A} \text{ is successful w.r.t. } \mathsf{Ext}_{\mathsf{ks}}] \leq \Pr[\mathcal{B}_{\mathrm{PDL}} \text{ fails} \mid E] + \Pr[\mathcal{C}_{\mathrm{TOFR}} \text{ fails} \mid \neg E] + \tfrac{d^*}{|\mathbb{F}|} \ .$$

□

AGM to GGM Lifting. By an analysis similar to that in Section B, we obtain that LinTrick is knowledge-sound in Maurer's GGM.

Discussion: AGMOS vs Standard Model Security. We will next give an intuitive explanation of why LinTrick has knowledge-soundness in the AGMOS Theorem 8 (under the assumption of n-independence) but does not have computational special soundness in the standard model Theorem 3. In the AGMOS, the extractor $\mathsf{Ext}^{\mathcal{A}}$ extracts *some* low-degree polynomials $\mathsf{d}_t(X, \mathfrak{Q})$ from the polynomial commitments. We obtain that Eqs. (18) and (19) hold by analyzing the verifier's evaluation equation. Since the polynomials $\mathsf{g}_t(\mathbf{a}'(X))$ are n-independent, it follows that $\mathsf{d}_t(X, \mathfrak{Q}) = 0$ for each t. This holds even when the extracted polynomials $\mathsf{d}_t(X, \mathfrak{Q})$ can depend on indeterminates corresponding to obliviously sampled elements. Indeed, to use the n-independence of polynomials, it is just needed that *some* low-degree polynomials can be extracted from $[d_\imath]_1$.

On the other hand, in the standard model, one cannot extract a field element from an oblivious sampled polynomial commitments $[d_t]_1$. One can only extract some polynomials (even ones depending on new indeterminates) after $[d_t]_1$ has been opened to a field element. In the special-soundness game, $[a_\imath]_1$ is opened at $n + 1$ different points; from these openings, one can interpolate the underlying polynomial $\mathsf{a}_\imath(X)$. However, in LinTrick, the only $[d_\imath]_1$-dependent commitment that is opened is the sum $\sum \mathsf{g}_t(\bar{\boldsymbol{a}})[d_t]_1$. Crucially, the latter is opened to 0 at any transcript during a computation special soundness proof, giving us no information about $[d_t]_1$: instead, we just know that some linear combination of $[d_t]_1$ is equal to $[0]_1$.

The impossibility result Theorem 3 formalizes it, proving that if one extracted the polynomials, one could break the discrete logarithm problem. This is a natural result: it formalizes the meaning of oblivious sampling, showing that sometimes (like in the case of LinTrick), oblivious sampling hides the discrete logarithms of sampled elements in the standard model.

D Postponed Material from Section 5

D.1 Plonk's Polynomials That Define A Specific Circuit

The following polynomials, along with the integer n, uniquely define our circuit:

- $\mathsf{q}_{\mathsf{M}}(X), \mathsf{q}_{\mathsf{L}}(X), \mathsf{q}_{\mathsf{R}}(X), \mathsf{q}_{\mathsf{O}}(X), \mathsf{q}_{\mathsf{C}}(X)$ are selector polynomials that define the circuit's arithmetization. We refer to [GWC19] for the explanation how they are defined based on an arithmetic circuit.

– $\mathsf{S}_{\mathsf{ID}_1}(X) = X$, $\mathsf{S}_{\mathsf{ID}_2}(X) = k_1 X$, $\mathsf{S}_{\mathsf{ID}_3}(X) = k_2 X$ encode an identity permutation respectively on groups $k_0 \cdot \mathbb{H}$, $k_1 \cdot \mathbb{H}$, and $k_2 \cdot \mathbb{H}$. Here, $k_0 := 1$ and $k_1, k_2 \in \mathbb{F}$ are chosen such that $\mathbb{H}, k_1 \cdot \mathbb{H}, k_2 \cdot \mathbb{H}$ are distinct cosets of $\mathbb{H}$ in $\mathbb{F}^*$, and thus consist of $3n$ distinct elements. For example, one can take ω to be a quadratic residue in $\mathbb{F}$, k_1 to be any quadratic non-residue, and k_2 to be a quadratic non-residue not contained in $k_1 \cdot \mathbb{H}$.
– Let us denote $\mathbb{H}' := \mathbb{H} \cup (k_1 \cdot \mathbb{H}) \cup (k_2 \cdot \mathbb{H})$. Let $\sigma : [1, 3n] \to [1, 3n]$ be a permutation. We encode an element $i \in [1, 3n]$ in $\mathbb{H}'$ such that if we express $i = \vartheta n + j$ for the unique $\vartheta \in [0, 2]$ and $0 \le j < n$, then $\mathbb{H}'[i] = k_\vartheta \omega^j$. Finally, define $\sigma^*(i) := \mathbb{H}'[\sigma(i)]$, which is an injective map on $\mathbb{H}'$. We encode σ^* by the three permutation polynomials $\mathsf{S}_{\sigma 1}(X) := \sum_{i=1}^{n} \sigma^*(i)\mathsf{L}_i(X)$, $\mathsf{S}_{\sigma 2}(X) := \sum_{i=1}^{n} \sigma^*(n+i)\mathsf{L}_i(X)$, and $\mathsf{S}_{\sigma 3}(X) := \sum_{i=1}^{n} \sigma^*(2n+i)\mathsf{L}_i(X)$.

D.2 Rhino for Plonk

In this section, we prove Theorem 9. Recall that SplitRSDH is defined in Definition 3. Let $\mathsf{L}_i^{\mathcal{S}}(X) = \prod_{j \neq i} \frac{X - \mathfrak{z}_j}{\mathfrak{z}_i - \mathfrak{z}_j}$ be Lagrange polynomials of $\mathcal{S} = \{\mathfrak{z}_i\}_{i=1}^{n_{\mathcal{S}}}$

Theorem 9. *Consider Plonk (with* $\mathsf{par} = \mathsf{par}_n^{\mathsf{plonk}}$*) or SmallPlonk (with* $\mathsf{par} = \mathsf{par}_n^{\mathsf{smallplonk}}$*). There exists a* par*-SplitRSDH adversary* $\mathcal{D}_{\mathrm{splitrsdh}}$ *(see Fig. 16), such that* $\Pr[\mathcal{D}_{\mathrm{splitrsdh}}$ *succeeds* $\mid \mathsf{bad}_{\mathsf{rhino}}] = 1$.

Proof (Proof of Theorem 9). Assume $\mathsf{bad}_{\mathsf{rhino}} = \mathsf{true}$. Let $\mathcal{D}_{\mathrm{splitrsdh}}$ be the par-SplitRSDH adversary in Fig. 16. $\mathcal{D}_{\mathrm{splitrsdh}}$ uses $\mathcal{A}_{\mathsf{ss}}^{\mathsf{plonk}}$ to obtain a subtree $\hat{\mathcal{T}}_{\beta\gamma\alpha} = (\mathsf{tr}_{ijk})$ (see Eq. (13); note that $\hat{\mathcal{T}}_{\beta\gamma\alpha}$ contains, say, $[t_{lo}, t_{mid}, t_{hi}]_1$ in the case of Plonk or $[t]_1$ in the case of SmallPlonk). Then, $\mathcal{D}_{\mathrm{splitrsdh}}$ runs $\mathsf{TE}_{*\mathsf{plonk}}$ on $\hat{\mathcal{T}}_{\beta\gamma\alpha}$ to obtain several accepting KZG transcripts, and then runs $\mathsf{Ext}_{\mathsf{ss}}^{\mathsf{sub}}$ on $\hat{\mathcal{T}}_{\beta\gamma\alpha}$ to obtain polynomials $(\mathsf{z}(X), \mathsf{a}(X), \mathsf{b}(X), \mathsf{c}(X))$. After that, $\mathcal{D}_{\mathrm{splitrsdh}}$ computes the polynomials $\mathsf{F}_s(X)$, $\mathsf{F}(X)$ and $\mathsf{t}(X)$ (as defined in Eq. (7)). $\mathcal{D}_{\mathrm{splitrsdh}}$ also defines linearization polynomials $\Lambda_{0i}(X)$, $\Lambda_{1i}(X)$, $\Lambda_{2i}(X)$ (see Eq. (8)) of $\mathsf{F}_0(X)$, $\mathsf{F}_1(X)$, and $\mathsf{F}_2(X)$, and their batched sum $\Lambda_i(X)$ corresponding to $\mathsf{F}(X)$. Clearly, $[\Lambda_{si}(x)]_1 = [\Lambda_{si}]_1$ (from Fig. 9) for $s \in [0, 2]$ and $i \in [1, \kappa_{\mathfrak{z}}]$. If the SplitRSDH's requirement $\mathsf{Z}_{\mathbb{H}}(X) \nmid \mathsf{F}(X)$ is satisfied (this holds always since $\mathsf{bad}_{\mathsf{rhino}} = \mathsf{true}$, see Item v), $\mathcal{D}_{\mathrm{splitrsdh}}$ outputs a SplitRSDH adversary's output.

Let us argue that $\mathcal{D}_{\mathrm{splitrsdh}}$ is successful, i.e., the output satisfies par-SplitRSDH's conditions. First, it is easy to see that $\mathcal{S}$ is of size $\kappa_{\mathfrak{z}}$ since the values $\mathfrak{z}_i$ output by $\mathcal{A}_{\mathsf{ss}}^{\mathsf{plonk}}$ are mutually different by the definition of computational special soundness. Second, $L(X) = \sum_{i=1}^{\kappa_{\mathfrak{z}}} \mathsf{t}(\mathfrak{z}_i)\mathsf{L}_i^{\mathcal{S}}(X)$ as defined in Fig. 16 has degree $\le \kappa_{\mathfrak{z}} - 1$. It remains to show that $\deg(L) \ge \kappa_{\mathsf{kzg}} + n_\psi + 1$ and that

$$[t_{lo}]_1 \bullet [1]_2 + [t_{mid}]_1 \bullet [x^n]_2 + [t_{hi}]_1 \bullet [x^{2n}]_2 = [1]_1 \bullet [L(x)]_2 + [\varphi]_1 \bullet [\mathsf{Z}_{\mathcal{S}}(x)]_2 \tag{20}$$

in the case of Plonk or

$$[t]_1 \bullet [1]_2 = [1]_1 \bullet [L(x)]_2 + [\varphi]_1 \bullet [\mathsf{Z}_{\mathcal{S}}(x)]_2 \tag{21}$$

in the case of SmallPlonk. (here, we took into account the concrete par).

Recall from the proof of Theorem 5 that $\mathsf{bad}_{\mathsf{rhino}} = \mathsf{true}$ implies $\mathsf{bad}_{\mathsf{evb}} = \mathsf{bad}_{\mathsf{ext}} = \mathsf{false}$. Thus,

1. $[z]_1 = [\mathsf{z}(x)]_1$, $[a]_1 = [\mathsf{a}(x)]_1$, $[b]_1 = [\mathsf{b}(x)]_1$, $[c]_1 = [\mathsf{c}(x)]_1$, and
2. $\mathsf{z}(\mathfrak{z}_i\omega) = \bar{z}_{\omega i}$, $\mathsf{a}(\mathfrak{z}_i) = \bar{a}_i$, $\mathsf{b}(\mathfrak{z}_i) = \bar{b}_i$, and $\mathsf{c}(\mathfrak{z}_i) = \bar{c}_i$ for $i \in [1, \kappa_{\mathfrak{z}}]$.

Thus, $\mathsf{F}_s(\mathfrak{z}_i) = \Lambda_{si}(\mathfrak{z}_i)$, $\mathsf{F}(\mathfrak{z}_i) = \Lambda_i(\mathfrak{z}_i)$, and $\mathsf{t}(\mathfrak{z}_i) = \Lambda_i(\mathfrak{z}_i)/\mathsf{Z}_{\mathbb{H}}(\mathfrak{z}_i)$ for $s \in \{0, 1, 2\}$ and $i \in [1, \kappa_{\mathfrak{z}}]$.

$\mathcal{D}_{\mathrm{splitrsdh}}(\mathsf{p},\mathsf{ck})$

$\hat{\mathfrak{T}}_{\beta\gamma\alpha} \leftarrow \mathscr{A}^{\mathsf{plonk}}_{\mathsf{ss}}(\mathsf{ck}); ((\mathbf{k.tr}_k)_{k\in[1,\kappa_v+2]}, \mathbf{k.tr}^{\omega}) \leftarrow \mathsf{TE}_{*\mathsf{plonk}}(\mathsf{ck}, \hat{\mathfrak{T}}_{\beta\gamma\alpha})$;

$(\mathsf{z}(X), \mathsf{a}(X), \mathsf{b}(X), \mathsf{c}(X)) \leftarrow \mathsf{Ext}^{\mathsf{sub}}_{\mathsf{ss}}(\mathsf{ck}, \hat{\mathfrak{T}}_{\beta\gamma\alpha})$;

$\mathsf{F}_0(X) \leftarrow \mathsf{a}(X)\mathsf{b}(X)\mathsf{q}_{\mathsf{M}}(X) + \mathsf{a}(X)\mathsf{q}_{\mathsf{L}}(X) + \mathsf{b}(X)\mathsf{q}_{\mathsf{R}}(X) + \mathsf{c}(X)\mathsf{q}_{\mathsf{O}}(X) + \mathsf{PI}(X) + \mathsf{q}_{\mathsf{C}}(X)$;

$\mathsf{F}_1(X) \leftarrow (\mathsf{a}(X) + \beta X + \gamma)(\mathsf{b}(X) + \beta k_1 X + \gamma)(\mathsf{c}(X) + \beta k_2 X + \gamma)\mathsf{z}(X)$
$\quad - (\mathsf{a}(X) + \beta \mathsf{S}_{\sigma 1}(X) + \gamma)(\mathsf{b}(X) + \beta \mathsf{S}_{\sigma 2}(X) + \gamma)(\mathsf{c}(X) + \beta \mathsf{S}_{\sigma 3}(X) + \gamma)\mathsf{z}(\omega X)$;

$\mathsf{F}_2(X) \leftarrow (\mathsf{z}(X) - 1)\mathsf{L}_1(X)$;

$\mathsf{F}(X) \leftarrow \mathsf{F}_0(X) + \alpha \mathsf{F}_1(X) + \alpha^2 \mathsf{F}_2(X)$;

$\mathsf{t}(X) \leftarrow \mathsf{F}(X)/\mathsf{Z}_{\mathbb{H}}(X)$;

if $\mathsf{Z}_{\mathbb{H}}(X) \mid \mathsf{F}(X)$ **then return** $\bot$; **fi**

for $i \in [1, \kappa_{\mathfrak{z}}]$ **do**

$\quad \Lambda_{0i}(X) \leftarrow \bar{a}_i \bar{b}_i \mathsf{q}_{\mathsf{M}}(X) + \bar{a}_i \mathsf{q}_{\mathsf{L}}(X) + \bar{b}_i \mathsf{q}_{\mathsf{R}}(X) + \bar{c}_i \mathsf{q}_{\mathsf{O}}(X) + \mathsf{PI}(\mathfrak{z}) + \mathsf{q}_{\mathsf{C}}(X)$;

$\quad \Lambda_{1i}(X) \leftarrow (\bar{a}_i + \beta \mathfrak{z}_i + \gamma)(\bar{b}_i + \beta k_1 \mathfrak{z}_i + \gamma)(\bar{c}_i + \beta k_2 \mathfrak{z}_i + \gamma)\mathsf{z}(X)$
$\quad\quad - (\bar{a}_i + \beta \bar{s}_{\sigma 1} + \gamma)(\bar{b}_i + \beta \bar{s}_{\sigma 2} + \gamma)(\bar{c}_i + \beta \mathsf{S}_{\sigma 3}(X) + \gamma)\bar{z}_{\omega,i}$;

$\quad \Lambda_{2i}(X) \leftarrow (\mathsf{z}(X) - 1)\mathsf{L}_1(\mathfrak{z}_i)$;

$\quad \Lambda_i(X) \leftarrow \Lambda_{0i}(X) + \alpha \Lambda_{1i}(X) + \alpha^2 \Lambda_{2i}(X)$;

$\quad \chi'_i(X) \leftarrow \frac{\Lambda_i(X) - \Lambda_i(\mathfrak{z}_i)}{X - \mathfrak{z}_i}$;

$\quad [\mathsf{W}'_{\mathfrak{z}i1}, \ldots, \mathsf{W}'_{\mathfrak{z}i\kappa_v}]_1^{\mathsf{T}} \leftarrow \boldsymbol{V}_i^{-1} [\mathsf{W}_{\mathfrak{z}i1}, \ldots, \mathsf{W}_{\mathfrak{z}i\kappa_v}]_1^{\mathsf{T}}$; // $\boldsymbol{V}_i$ as in Eq. (15)

$\quad [\chi_i]_1 \leftarrow \frac{1}{\mathsf{Z}_{\mathbb{H}}(\mathfrak{z}_i)} [\chi'_i(x) - \mathsf{W}'_{\mathfrak{z}i1}]_1$; // $\mathsf{W}'_{\mathfrak{z}i1}$ is as in Line 8 in Fig. 9

$\quad \Delta_i \leftarrow 1/\prod_{j \neq i} (\mathfrak{z}_i - \mathfrak{z}_j)$; **endfor**

$[\varphi]_1 \leftarrow \sum_{i=1}^{\kappa_{\mathfrak{z}}} \Delta_i [\chi_i]_1$;

$L(X) \leftarrow \sum_{i=1}^{\kappa_{\mathfrak{z}}} \mathsf{t}(\mathfrak{z}_i) \mathsf{L}_i^{\mathcal{S}}(X)$; // $\mathsf{L}_i^{\mathcal{S}}(X)$ are Lagrange basis polynomials

// Only the return value depends on whether we have Plonk or SmallPlonk

return $(\mathcal{S} = \{\mathfrak{z}_i\}_{i=1}^{\kappa_{\mathfrak{z}}}, L(X), [t, \varphi]_1)$; // In SmallPlonk

return $(\mathcal{S} = \{\mathfrak{z}_i\}_{i=1}^{\kappa_{\mathfrak{z}}}, L(X), [t_{lo}, t_{mid}, t_{hi}, \varphi]_1)$; // In Plonk

Fig. 16. The SplitRSDH adversary $\mathcal{D}_{\mathrm{splitrsdh}}$.

Denote $y_i := t_{lo} + \mathfrak{z}_i^n t_{mid} + \mathfrak{z}_i^{2n} t_{hi}$ in the case of Plonk and $y_i := t$ in the case of SmallPlonk. Recall

$$[r_i]_1 = [\Lambda_i(x) - \mathsf{Z}_{\mathbb{H}}(\mathfrak{z}_i) y_i]_1 \tag{22}$$

from the lines 3, 4, and 6 in Fig. 9. Next, we use the fact that $\mathcal{D}_{\mathrm{splitrsdh}}$ knows $\Lambda_i(X)$ as a polynomial. Since it can open both $[r_i]_1$ and $[\Lambda_i(x)]_1$ at $\mathfrak{z}_i$, it can also open $[y_i]_1$. More precisely, let

$$\chi'_i(X) = (\Lambda_i(X) - \Lambda_i(\mathfrak{z}_i))/(X - \mathfrak{z}_i) = (\Lambda_i(X) - \mathsf{F}(\mathfrak{z}_i))/(X - \mathfrak{z}_i)$$

be the KZG opening polynomial of $\Lambda_i(X)$, $i \in [1, \kappa_{\mathfrak{z}}]$, at point $\mathfrak{z}_i$. The second equality follows from $\mathsf{bad}_{\mathsf{evb}} = \mathsf{bad}_{\mathsf{ext}} = \mathsf{false}$.

Recall that as part of $\mathsf{k.tr}_{1i}$ (see line 9 in Fig. 9), $[\mathsf{W}'_{\mathfrak{z}i1}]_1$ is the opening proof of $[r_i]_1$ at $\mathfrak{z}_i$ to 0. Since $\mathsf{k.tr}_{1i}$ is accepting, Eq. (22) implies that $\mathsf{W}'_{\mathfrak{z}i1} \cdot (x - \mathfrak{z}_i) = r_i$. It follows from this, $\mathsf{bad}_{\mathsf{evb}} = \mathsf{bad}_{\mathsf{ext}} = \mathsf{false}$, and the definition of $\chi'_i(X)$ that

$$\mathsf{W}'_{\mathfrak{z}i1} \cdot (x - \mathfrak{z}_i) = r_i = \chi'_i(x)(x - \mathfrak{z}_i) + \Lambda_i(\mathfrak{z}_i) - \mathsf{Z}_{\mathbb{H}}(\mathfrak{z}_i) y_i$$

and thus

$$\chi_i'(x) - \mathsf{W}'_{\mathfrak{z}i1} = \tfrac{1}{x-\mathfrak{z}_i}(\mathsf{Z}_{\mathbb{H}}(\mathfrak{z}_i)y_i - \Lambda_i(\mathfrak{z}_i))$$

for $i \in [1, \kappa_{\mathfrak{z}}]$. Since $\Lambda_i(\mathfrak{z}_i) = \mathsf{F}(\mathfrak{z}_i)$ and $\mathsf{t}(\mathfrak{z}_i) = \mathsf{F}(\mathfrak{z}_i)/\mathsf{Z}_{\mathbb{H}}(\mathfrak{z}_i)$, we get $\chi_i'(x) - \mathsf{W}'_{\mathfrak{z}i1} = \frac{1}{x-\mathfrak{z}_i} \cdot (\mathsf{Z}_{\mathbb{H}}(\mathfrak{z}_i)y_i - \mathsf{F}(\mathfrak{z}_i)) = \frac{\mathsf{Z}_{\mathbb{H}}(\mathfrak{z}_i)}{x-\mathfrak{z}_i}(y_i - \mathsf{t}(\mathfrak{z}_i))$. Defining $\chi_i \leftarrow \frac{1}{\mathsf{Z}_{\mathbb{H}}(\mathfrak{z}_i)}(\chi_i'(x) - \mathsf{W}'_{\mathfrak{z}i1})$ as in Fig. 16 for $i \in [1, \kappa_{\mathfrak{z}}]$, we get

$$y_i - \mathsf{t}(\mathfrak{z}_i) = \chi_i(x - \mathfrak{z}_i) \ . \tag{23}$$

We multiply $\kappa_{\mathfrak{z}}$ equations Eq. (23) individually by $\mathsf{L}_i^S(X)$ and then sum the results, getting

$$\sum_{i=1}^{\kappa_{\mathfrak{z}}} (y_i - \mathsf{t}(\mathfrak{z}_i))\mathsf{L}_i^S(x) = \sum_{i=1}^{\kappa_{\mathfrak{z}}} \chi_i(x - \mathfrak{z}_i)\mathsf{L}_i^S(x) \ . \tag{24}$$

Since $\mathsf{L}_i^S(X) = \prod_{j \neq i} \frac{X-\mathfrak{z}_j}{\mathfrak{z}_i - \mathfrak{z}_j} = \Delta_i \prod_{j \neq i}(X - \mathfrak{z}_j) = \Delta_i \frac{\mathsf{Z}_S(X)}{X-\mathfrak{z}_i}$, $(X - \mathfrak{z}_i)\mathsf{L}_i^S(X) = \Delta_i \mathsf{Z}_S(X)$. Let $L(X) = \sum_{i=1}^{\kappa_{\mathfrak{z}}} \mathsf{t}(\mathfrak{z}_i)\mathsf{L}_i^S(X)$ be the interpolating polynomial of $\{(\mathfrak{z}_i, \mathsf{t}(\mathfrak{z}_i))\}_{i=1}^{\kappa_{\mathfrak{z}}}$. Denote $T(x) := t_{lo} + t_{mid}x^n + t_{hi}x^{2n}$ in the case of Plonk and $T(x) := t$ in the case of SmallPlonk. In the case of Plonk,

$$\begin{aligned} T(x) - L(x) &= t_{lo} + t_{mid}x^n + t_{hi}x^{2n} - L(x) \\ &\overset{(*)}{=} t_{lo} + t_{mid} \sum_{i=1}^{\kappa_{\mathfrak{z}}} \mathfrak{z}_i^n \mathsf{L}_i^S(x) + t_{hi} \sum_{i=1}^{\kappa_{\mathfrak{z}}} \mathfrak{z}_i^{2n} \mathsf{L}_i^S(x) - L(x) \\ &= \sum_{i=1}^{\kappa_{\mathfrak{z}}} (y_i - \mathsf{t}(\mathfrak{z}_i))\mathsf{L}_i^S(x) \ , \end{aligned}$$

where (*) follows from (say) $X^n = \sum_{i=1}^{\kappa_{\mathfrak{z}}} \mathfrak{z}_i^n \mathsf{L}_i^S(X)$. In the case of SmallPlonk,

$$T(x) - L(x) = t - L(x) = \sum_{i=1}^{\kappa_{\mathfrak{z}}} (y_i - \mathsf{t}(\mathfrak{z}_i))\mathsf{L}_i^S(x) \ .$$

Thus,

$$T(x) - L(x) \overset{24}{=} \sum_{i=1}^{\kappa_{\mathfrak{z}}} \chi_i(x - \mathfrak{z}_i)\mathsf{L}_i^S(x) = \mathsf{Z}_S(x) \sum_{i=1}^{\kappa_{\mathfrak{z}}} \Delta_i \chi_i = \mathsf{Z}_S(x)\varphi \ .$$

Thus, Eq. (20) (in the case of Plonk) or Eq. (21) (in the case of SmallPlonk) holds.

Finally, clearly $\deg L(X) \leq \kappa_{\mathfrak{z}} - 1$. On the other hand, assume $\deg L(X) \leq \kappa_{\mathsf{kzg}} + n_\psi \leq \kappa_{\mathfrak{z}} - n - 1$. In the case of Plonk, the last inequality holds due to the choice of $\mathsf{par}_n^{\mathsf{plonk}}$, since $\kappa_{\mathfrak{z}} = 4\kappa_{\mathsf{kzg}} + 1 = 4n + 21 \geq (n+5) + 2n + n + 1 = \kappa_{\mathsf{kzg}} + n_\psi + n + 1$. In SmallPlonk, it is even more obvious since $\kappa_{\mathfrak{z}} = 4\kappa_{\mathsf{kzg}} + 1 = 12n + 21 \geq (3n+5) + 0 + n + 1 = \kappa_{\mathsf{kzg}} + n_\psi + n + 1$. Since $L(X)$ and $\mathsf{F}(X)/\mathsf{Z}_{\mathbb{H}}(X)$ match on $\kappa_{\mathfrak{z}}$ points $\mathfrak{z}_i$ and both $L(X)\mathsf{Z}_{\mathbb{H}}(X)$ and $\mathsf{F}(X)$ have degree $\leq \kappa_{\mathfrak{z}} - 1$, $L(X) = \mathsf{F}(X)/\mathsf{Z}_{\mathbb{H}}(X)$, which means $\mathsf{Z}_{\mathbb{H}}(X) \mid \mathsf{F}(X)$, contradiction with $\mathsf{bad}_{\mathsf{rhino}} = \mathsf{true}$. Thus, if $\mathsf{Z}_{\mathbb{H}}(X) \nmid \mathsf{F}(X)$, then $\deg L(X) \in [\kappa_{\mathsf{kzg}} + n_\psi + 1, \kappa_{\mathfrak{z}} - 1]$. □

D.3 SanPlonk's Case from Theorem 5

Proof (Finishing the proof of Theorem 5).

Case of SanPlonk. The case of SanPlonk is similar to Plonk but differs in quite many details. For the sake of clarity, we highlight additional text, but we also removed some text that is not

relevant for SanPlonk. (Intuitively, the removed text corresponds to the part in Plonk's proof where one handles the reduction to SplitRSDH.)

In Fig. 10, we depict an extractor $\mathsf{Ext}_{\mathsf{ss}}^{\mathsf{sub}}$. $\mathsf{Ext}_{\mathsf{ss}}^{\mathsf{sub}}$ invokes $\mathsf{Ext}_{\mathsf{ss}}^{\mathsf{kzg}}(\mathsf{ck}, \mathbf{k.tr}^{\omega})$ and $\mathsf{Ext}_{\mathsf{ss}}^{\mathsf{kzg}}(\mathsf{ck}, \mathbf{k.tr}_k)$ for $k \in [2,4] \cup [7,9]$, extracting polynomials $\mathsf{z}(X)$, $\mathsf{a}(X)$, $\mathsf{b}(X)$, $\mathsf{c}(X)$, $\mathsf{t}_{\mathsf{lo}}(X)$, $\mathsf{t}_{\mathsf{mid}}(X)$, and $\mathsf{t}_{\mathsf{hi}}(X)$ of at most degree $\kappa_{\mathsf{kzg}} = n+5$. After executing $\mathsf{Ext}_{\mathsf{ss}}^{\mathsf{sub}}$, we use the following procedure to possibly set one of the "bad" flags:

(i') $\mathsf{bad}_{\mathsf{ext}} \leftarrow \mathsf{false}$; $\mathsf{bad}_{\mathsf{evb}} \leftarrow \mathsf{false}$;

(ii') if $([z]_1, \mathsf{z}(X)) \notin \mathcal{R}_{\mathsf{ck},\mathbf{k.tr}_1} \vee ([a]_1, \mathsf{a}(X)) \notin \mathcal{R}_{\mathsf{ck},\mathbf{k.tr}_2} \vee ([b]_1, \mathsf{b}(X)) \notin \mathcal{R}_{\mathsf{ck},\mathbf{k.tr}_3} \vee ([c]_1, \mathsf{c}(X)) \notin \mathcal{R}_{\mathsf{ck},\mathbf{k.tr}_4} \vee ([t_{lo}]_1, \mathsf{t}_{\mathsf{lo}}(X)) \notin \mathcal{R}_{\mathsf{ck},\mathbf{k.tr}_7} \vee ([t_{mid}]_1, \mathsf{t}_{\mathsf{mid}}(X)) \notin \mathcal{R}_{\mathsf{ck},\mathbf{k.tr}_8} \vee ([t_{lo}]_1, \mathsf{t}_{\mathsf{hi}}(X)) \notin \mathcal{R}_{\mathsf{ck},\mathbf{k.tr}_9}$ (see Eq. (1)) then $\mathsf{bad}_{\mathsf{ext}} \leftarrow \mathsf{true}$; abort;

(iii') for $i \in [\kappa_{\mathsf{kzg}} + 2, \kappa_{\mathfrak{z}}]$: if $\mathsf{z}(\mathfrak{z}_i\omega) \neq \bar{z}_{\omega i} \vee \mathsf{a}(\mathfrak{z}_i) \neq \bar{a}_i \vee \mathsf{b}(\mathfrak{z}_i) \neq \bar{b}_i \vee \mathsf{c}(\mathfrak{z}_i) \neq \bar{c}_i \vee \mathsf{t}_{\mathsf{lo}}(\mathfrak{z}_i) \neq \bar{t}_{\mathfrak{z}lo,i} \vee \mathsf{t}_{\mathsf{mid}}(\mathfrak{z}_i) \neq \bar{t}_{\mathfrak{z}mid,i} \vee \mathsf{t}_{\mathsf{hi}}(\mathfrak{z}_i) \neq \bar{t}_{\mathfrak{z}hi,i}$ then $\mathsf{bad}_{\mathsf{evb}} \leftarrow \mathsf{true}$; abort;

(iv') for $i \in [1, \kappa_{\mathfrak{z}}]$:
- if $\mathsf{S}_{\sigma 1}(\mathfrak{z}_i) \neq \bar{s}_{\sigma 1 i} \vee \mathsf{S}_{\sigma 2}(\mathfrak{z}_i) \neq \bar{s}_{\sigma 2 i}$ then $\mathsf{bad}_{\mathsf{evb}} \leftarrow \mathsf{true}$; abort;
- $\mathsf{r}_i(X) \leftarrow \Lambda_{0i}(X) + \alpha\Lambda_{1i}(X) + \alpha^2\Lambda_{2i}(X) - \mathsf{Z}_{\mathbb{H}}(\mathfrak{z}_i) \cdot (\mathsf{t}_{\mathsf{lo}}(X) + \mathfrak{z}_i^n \mathsf{t}_{\mathsf{mid}}(X) + \mathfrak{z}_i^{2n} \mathsf{t}_{\mathsf{hi}}(X))$;
- if $\mathsf{r}_i(\mathfrak{z}_i) \neq 0$ then $\mathsf{bad}_{\mathsf{evb}} \leftarrow \mathsf{true}$;

Importantly, only one of the "bad" flags is set at a time. Thus, for example, $\mathsf{bad}_{\mathsf{evb}} = \mathsf{true}$ implies that $\mathsf{bad}_{\mathsf{ext}} = \mathsf{false}$. Let $\mathcal{E}$ be the event $\mathsf{Ext}_{\mathsf{ss}}^{\mathsf{sub}}$ succeeds and $\overline{\mathsf{bad}}$ be the event none of the bad flags was set. Thus, $\mathcal{E}$ is the event that $\mathsf{a}(X)$, $\mathsf{b}(X)$, $\mathsf{c}(X)$, $\mathsf{z}(X)$, $\mathsf{t}_{\mathsf{lo}}(X)$, $\mathsf{t}_{\mathsf{mid}}(X)$, and $\mathsf{t}_{\mathsf{hi}}(X)$ are consistent with the commitments and all openings, and $\mathsf{t}(X) = (\mathsf{F}_0(X) + \alpha\mathsf{F}_1(X) + \alpha^2\mathsf{F}_2(X))/\mathsf{Z}_{\mathbb{H}}(X)$ is a polynomial. We analyze the success probability of $\mathsf{Ext}_{\mathsf{ss}}^{\mathsf{sub}}$. For this, we make the following claims.

1. **Claim 1**. $\Pr[\mathcal{E}|\overline{\mathsf{bad}}] = 1$.
   Really, assume that $\overline{\mathsf{bad}}$ holds. Since $\mathsf{bad}_{\mathsf{ext}} = \mathsf{bad}_{\mathsf{evb}} = \mathsf{false}$, we get that $\mathsf{a}(X)$, $\mathsf{b}(X)$, $\mathsf{c}(X)$, $\mathsf{z}(X)$, $\mathsf{t}_{\mathsf{lo}}(X)$, $\mathsf{t}_{\mathsf{mid}}(X)$, $\mathsf{t}_{\mathsf{hi}}(X)$ are consistent with the commitments and all openings.
   Recall that in the case of Plonk, we deduced here that since $\mathsf{bad}_{\mathsf{rhino}} = \mathsf{false}$, $\mathsf{t}(X) = (\mathsf{F}_0(X) + \alpha\mathsf{F}_1(X) + \alpha^2\mathsf{F}_2(X))/\mathsf{Z}_{\mathbb{H}}(X)$ is a polynomial. In the case of SanPlonk, we handle this differently. Since $\mathsf{bad}_{\mathsf{evb}} = \mathsf{false}$, $\mathsf{r}_i(X) = 0$ for all $i \in [1, \kappa_{\mathfrak{z}}]$. Let $g(X) := \mathsf{F}(X) - \mathsf{Z}_{\mathbb{H}}(X)\mathsf{t}^*(X)$, where $\mathsf{t}^*(X) := \mathsf{t}_{\mathsf{lo}}(X) + X^n\mathsf{t}_{\mathsf{mid}}(X) + X^{2n}\mathsf{t}_{\mathsf{hi}}(X)$. Since for $i \in [1, \kappa_{\mathfrak{z}}]$, $\mathsf{F}_s(\mathfrak{z}_i) = \Lambda_{si}(\mathfrak{z}_i)$, we have $g(\mathfrak{z}_i) = 0$. Since this holds for $\kappa_{\mathfrak{z}} = 4\kappa_{\mathsf{kzg}} + 1$ evaluation points and $\deg g(X) \leq 4\kappa_{\mathsf{kzg}}$, $g(X) = 0$. Thus, $\mathsf{t}(X)$ is a polynomial.
2. **Claim 2**. There exists a KZG's computational $(\kappa_{\mathsf{kzg}} + 1)$-special-soundness adversary $\mathcal{B}_{\mathsf{ss}}^{\mathsf{kzg}}$ (see Fig. 11), such that $\Pr[\mathcal{B}_{\mathsf{ss}}^{\mathsf{kzg}} \text{ succeeds} \mid \mathsf{bad}_{\mathsf{ext}}] = 1$.
   Recall from Item ii that $\mathsf{bad}_{\mathsf{ext}}$ is set if one of the seven bad events happens. $\mathcal{B}_{\mathsf{ss}}^{\mathsf{kzg}}$ just tests which of the cases is true and returns the corresponding transcript. Clearly, $\Pr[\mathcal{B}_{\mathsf{ss}}^{\mathsf{kzg}} \text{ succeeds} \mid \mathsf{bad}_{\mathsf{ext}}] = 1$.
3. **Claim 3**. There exists an evaluation-binding adversary $\mathcal{C}_{\mathsf{evb}}$ (see Fig. 12) for KZG, such that $\Pr[\mathcal{C}_{\mathsf{evb}} \text{ succeeds} \mid \mathsf{bad}_{\mathsf{evb}}] = 1$.
   Assume that $\mathsf{bad}_{\mathsf{evb}} = \mathsf{true}$. (Note that $\mathsf{bad}_{\mathsf{evb}} = \mathsf{true}$ means that $\mathsf{bad}_{\mathsf{ext}} = \mathsf{false}$, that is, $\mathsf{Ext}_{\mathsf{ss}}^{\mathsf{sub}}$ managed to extract all polynomials.) Then, one of the bad cases in Item iii or Item iv happens. $\mathcal{C}_{\mathsf{evb}}$ just finds out which of these events happens, and depending on the case, returns a collision. By the correctness of the extraction, and the completeness property of KZG, any of the returned values in Fig. 12 *is* a collision. Thus, $\Pr[\mathcal{C}_{\mathsf{evb}} \text{ succeeds} \mid \mathsf{bad}_{\mathsf{evb}}] = 1$.

   Thus,

$$\begin{aligned}\Pr[\overline{\mathcal{E}}] &= \Pr[\overline{\mathcal{E}}|\overline{\mathsf{bad}}]\Pr[\overline{\mathsf{bad}}] + \Pr[\overline{\mathcal{E}}|\mathsf{bad}_{\mathsf{ext}}]\Pr[\mathsf{bad}_{\mathsf{ext}}] + \Pr[\overline{\mathcal{E}}|\mathsf{bad}_{\mathsf{evb}}]\Pr[\mathsf{bad}_{\mathsf{evb}}] \\ &\leq 0 + \Pr[\mathsf{bad}_{\mathsf{ext}}] + \Pr[\mathsf{bad}_{\mathsf{evb}}]\end{aligned}$$

Since $\mathcal{C}_{\mathsf{evb}}$ succeeds whenever $\mathsf{bad}_{\mathsf{evb}}$ is set and KZG is evaluation binding, $\Pr[\mathsf{bad}_{\mathsf{evb}}] = \mathsf{negl}(\lambda)$. Similarly, $\Pr[\mathsf{bad}_{\mathsf{ext}}] = \mathsf{negl}(\lambda)$. Thus, $\Pr[\overline{\mathcal{E}}] \leq \mathsf{negl}(\lambda)$. This proves the claim. □

$\mathsf{Ext}^{*\mathsf{plonk}}_{\mathsf{ss}}(\mathsf{ck}, \mathbb{x}, \mathcal{T})$

Pick an arbitrary $(1, 1, 1, \kappa_{\mathfrak{z}}, \kappa_\delta, \kappa_v)$-subtree $\hat{\mathcal{T}}$ of $\mathcal{T}$.

$(\mathsf{z}(X), \mathsf{a}(X), \mathsf{b}(X), \mathsf{c}(X), \mathsf{t}_{\mathsf{lo}}(X), \mathsf{t}_{\mathsf{mid}}(X), \mathsf{t}_{\mathsf{hi}}(X)) \leftarrow \mathsf{Ext}^{\mathsf{sub}}_{\mathsf{ss}}(\mathsf{ck}, \hat{\mathcal{T}})$;

for $i \in [1, n]$ **do**

$\bar{w}_i \leftarrow \mathsf{a}(\omega^i); \bar{w}_{n+i} \leftarrow \mathsf{b}(\omega^i); \bar{w}_{2n+i} \leftarrow \mathsf{c}(\omega^i)$; **endfor**

return $\mathbb{w} \leftarrow (\bar{w}_i)_{i=\ell+1}^{3n}$;

Fig. 17. The computational special-soundness extractor $\mathsf{Ext}^{*\mathsf{plonk}}_{\mathsf{ss}}$ for Plonk/SanPlonk.

D.4 Proof of Theorem 6

In this section, we restate and then prove the theorem Theorem 6 from Section 5.

Theorem 10. *Let $n \in \mathsf{poly}(\lambda)$ and κ_{kzg}, $\boldsymbol{\kappa}_{\mathsf{Plonk}}$, and $\boldsymbol{\kappa}_{\mathsf{san}}$ be as in Eq. (12).*

1. *If KZG has computational $(\kappa_{\mathsf{kzg}} + 1)$-special soundness and evaluation binding, and $\mathsf{par}_n^{\mathsf{smallplonk}}$-SplitRSDH holds, then SmallPlonk has computational $\boldsymbol{\kappa}_{\mathsf{Plonk}}$-special soundness.*
2. *If KZG has computational $(\kappa_{\mathsf{kzg}}+1)$-special soundness and evaluation binding, and $\mathsf{par}_n^{\mathsf{plonk}}$-SplitRSDH holds, then Plonk has computational $\boldsymbol{\kappa}_{\mathsf{Plonk}}$-special soundness.*
3. *If KZG has computational $(\kappa_{\mathsf{kzg}}+1)$-special soundness and evaluation binding, then SanPlonk has computational $\boldsymbol{\kappa}_{\mathsf{san}}$-special soundness.*

Proof. Fix

$$\mathscr{P} = \{\mathsf{q}_{\mathsf{M}}(X), \mathsf{q}_{\mathsf{L}}(X), \mathsf{q}_{\mathsf{R}}(X), \mathsf{q}_{\mathsf{O}}(X), \mathsf{q}_{\mathsf{C}}(X), \mathsf{S}_{\sigma 1}(X), \mathsf{S}_{\sigma 2}(X), \mathsf{S}_{\sigma 3}(X)\}$$

for a relation defined by encoding a circuit as in Section 5.1. Let $\mathscr{A}_{\mathsf{ss}}$ be a PPT adversary in the computational special-soundness game that outputs a $(\kappa_\beta, \kappa_\gamma, \kappa_\alpha, \kappa_{\mathfrak{z}}, \kappa_\delta, \kappa_v)$-tree $\mathcal{T}$. We describe the computational special-soundness extractor $\mathsf{Ext}^{*\mathsf{plonk}}_{\mathsf{ss}}$ for Plonk in Fig. 17. The extractor picks an arbitrary $(1, 1, 1, \kappa_{\mathfrak{z}}, \kappa_\delta, \kappa_v)$-subtree $\hat{\mathcal{T}} = \hat{\mathcal{T}}_{\beta\gamma\alpha}$ of $\mathcal{T}$ and runs the subtree extractor $\mathsf{Ext}^{\mathsf{sub}}_{\mathsf{ss}}$ from Theorem 5 on it to obtain the polynomials $\mathsf{z}(X)$, $\mathsf{a}(X)$, $\mathsf{b}(X)$, $\mathsf{c}(X)$, $\mathsf{t}_{\mathsf{lo}}(X)$, $\mathsf{t}_{\mathsf{mid}}(X)$, and $\mathsf{t}_{\mathsf{hi}}(X)$. In the honest protocol, the witness is encoded in $\mathsf{a}(X)$, $\mathsf{b}(X)$, and $\mathsf{c}(X)$. Namely, $\bar{w}_i \leftarrow \mathsf{a}(\omega^i)$, $\bar{w}_{n+i} \leftarrow \mathsf{b}(\omega^i)$, $\bar{w}_{2n+i} \leftarrow \mathsf{c}(\omega^i)$ for $i \in [1, n]$, and $\mathbb{w} = (\bar{w}_i)_{i=\ell+1}^{3n}$. The rest of the proof shows that $(\mathbb{x} = (\bar{w}_i)_{i=1}^{\ell}, \mathbb{w}) \in \mathcal{R}_{\mathscr{P}}$.

To be sure we compute a correct witness, we must assume that for each β_i, we have κ_γ mutually different values γ_{ij}. Whence the double index on challenges γ_{ij}, despite they are sent in the same round by Plonk and its variants' prover. One can be implement this by rewinding the protocol with $\kappa_\beta\kappa_\gamma$ different challenges $(\beta_i\gamma_{ij})$. Alternatively, one can consider Plonk as the interactive argument where the prover first receives the challenge β from the verifier, then replies with an empty message, then receives the challenge γ, and goes on with the execution.[7]

Let

$$\mathsf{SubTrees}_{\mathcal{T}} := \{\hat{\mathcal{T}}_{\beta_i\gamma_{ij}\alpha_{ijk}} : i \in [1, \kappa_\beta], j \in [1, \kappa_\gamma], k \in [1, \kappa_\alpha]\}$$

be the set of all $(1, 1, 1, \kappa_{\mathfrak{z}}, \kappa_\delta, \kappa_v)$-subtrees of $\mathcal{T}$. For each $\hat{\mathcal{T}} \in \mathsf{SubTrees}_{\mathcal{T}}$, we can apply the extractor from Theorem 5 and obtain the subtree transcript

$$\mathsf{s.tr}_{\hat{\mathcal{T}}} = (\mathsf{z}_{\hat{\mathcal{T}}}(X), \mathsf{a}_{\hat{\mathcal{T}}}(X), \mathsf{b}_{\hat{\mathcal{T}}}(X), \mathsf{c}_{\hat{\mathcal{T}}}(X)) \ ,$$

[7] We note that this does not change actual Plonk/SanPlonk: when applying Fiat-Shamir, one defines $\beta = \mathsf{H}(\mathit{view}, 0)$ and $\gamma = \mathsf{H}(\mathit{view}, 1)$ for a random oracle H. To rewind only γ and not β, one can reprogram the random oracle at input $(\mathit{view}, 1)$ but not input $(\mathit{view}, 0)$.

$\mathcal{A}_{\mathsf{bind}}(\mathsf{ck})$

$(\mathbb{x}, \mathcal{T}) \leftarrow \mathsf{Ext}_{\mathsf{ss}}^{\mathsf{kzg}}(\mathsf{ck})$;
for $\hat{\mathcal{T}} \in \mathsf{SubTrees}_{\mathcal{T}}$ **do**
  $\mathsf{s.tr}_{\hat{\mathcal{T}}} \leftarrow \mathsf{Ext}_{\mathsf{ss}}^{\mathsf{sub}}(\mathsf{ck}, \hat{\mathcal{T}})$; **endfor** // Extracts polynomials from each subtree
for distinct $\hat{\mathcal{T}} \neq \hat{\mathcal{T}}' \in \mathsf{SubTrees}_{\mathcal{T}}$ **do**
  $(\mathsf{a}(X), \mathsf{b}(X), \mathsf{c}(X)) \leftarrow \mathsf{W}_{\hat{\mathcal{T}}}$; $(\mathsf{a}'(X), \mathsf{b}'(X), \mathsf{c}'(X)) \leftarrow \mathsf{W}_{\hat{\mathcal{T}}'}$;
  if $\mathsf{a}(X) \neq \mathsf{a}'(X)$ **then return** $([a]_1, \mathsf{a}(X), \mathsf{a}'(X))$;
  if $\mathsf{b}(X) \neq \mathsf{b}'(X)$ **then return** $([b]_1, \mathsf{b}(X), \mathsf{b}'(X))$;
  if $\mathsf{c}(X) \neq \mathsf{c}'(X)$ **then return** $([c]_1, \mathsf{c}(X), \mathsf{c}'(X))$; **endfor**
return $\bot$;

Fig. 18. The binding adversary $\mathcal{A}_{\mathsf{bind}}$ in Theorem 6.

where $\hat{\mathcal{T}} = \hat{\mathcal{T}}_{\beta_i\gamma_{ij}\alpha_{ijk}}$ and $i \in [1, \kappa_\beta]$, $j \in [1, \kappa_\gamma]$, and $k \in [1, \kappa_\alpha]$. Observe that the extracted polynomials may depend on the specific subtree $\hat{\mathcal{T}}$.

Next, we argue that the extractor $\mathsf{Ext}_{\mathsf{ss}}^{*\mathsf{plonk}}$ can fail only if one of the following events happens.

$\mathsf{bad}_{\mathsf{sub}}$: For some $\hat{\mathcal{T}} \in \mathsf{SubTrees}_{\mathcal{T}}$, $\mathsf{Ext}_{\mathsf{ss}}^{\mathsf{sub}}(\mathsf{ck}, \hat{\mathcal{T}})$ outputs $\mathsf{s.tr}_{\hat{\mathcal{T}}}$ such that either (1) $\mathsf{z}_{\hat{\mathcal{T}}}(X)$, $\mathsf{a}_{\hat{\mathcal{T}}}(X)$, $\mathsf{b}_{\hat{\mathcal{T}}}(X)$, and $\mathsf{c}_{\hat{\mathcal{T}}}(X)$ are inconsistent with the commitments $[z_{ij}, a, b, c]_1$, or (2) $\mathsf{t}_{\hat{\mathcal{T}}ijk}(X)$ (defined as in Eq. (7)) is not a polynomial.

$\mathsf{bad}_{\mathsf{bind}}$: (1) The event $\mathsf{bad}_{\mathsf{sub}}$ does not happen. (2) Define

$$\mathsf{W}_{\hat{\mathcal{T}}} := \{(\mathsf{a}(X), \mathsf{b}(X), \mathsf{c}(X)) : (\mathsf{z}(X), \mathsf{a}(X), \mathsf{b}(X), \mathsf{c}(X)) \leftarrow \mathsf{Ext}_{\mathsf{ss}}^{\mathsf{sub}}(\mathsf{ck}, \hat{\mathcal{T}})\} \ .$$

There exist two subtrees $\hat{\mathcal{T}} \neq \hat{\mathcal{T}}' \in \mathsf{SubTrees}_{\mathcal{T}}$, such that $\mathsf{W}_{\hat{\mathcal{T}}} \neq \mathsf{W}_{\hat{\mathcal{T}}'}$.

Theorem 5 implies that if KZG is evaluation binding and has computational special soundness (and the additional assumption SplitRSDH holds in the case of Plonk or SmallPlonk), then $\Pr[\mathsf{bad}_{\mathsf{sub}}]$ is negligible. The fact that $\Pr[\mathsf{bad}_{\mathsf{bind}}]$ is negligible follows straightforwardly from the binding property of KZG. For the sake of completeness we present the binding adversary $\mathcal{A}_{\mathsf{bind}}$ in Fig. 18.

In the following, suppose that neither $\mathsf{bad}_{\mathsf{sub}}$ or $\mathsf{bad}_{\mathsf{bind}}$ happened. Thus, $\mathsf{W}_{\hat{\mathcal{T}}}(X) = \mathsf{W}_{\hat{\mathcal{T}}'}(X)$ for any $\hat{\mathcal{T}}, \hat{\mathcal{T}} \in \mathsf{SubTrees}_{\mathcal{T}}$. This justifies the notation $\mathsf{s.tr}_{ijk} = (\mathsf{z}_{ij}(X), \mathsf{a}(X), \mathsf{b}(X), \mathsf{c}(X))$, where $\mathsf{a}(X)$, $\mathsf{b}(X)$, and $\mathsf{c}(X)$ do not depend on the specific subtree $\hat{\mathcal{T}}$ while $\mathsf{z}_{ij}(X)$ depends on β_i and γ_{ij}. Furthermore, each $\mathsf{t}_{ijk}(X) := \mathsf{F}_{ijk}(X)/\mathsf{Z}_{\mathbb{H}}(X)$ is a polynomial, where

$$\mathsf{F}_{ijk}(X) := \mathsf{F}_0(X) + \alpha_{ijk}\mathsf{F}_{1ij}(X) + \alpha_{ijk}^2\mathsf{F}_{2ij}(X) \ ,$$

and $\mathsf{F}_0(X)$, $\mathsf{F}_{1ij}(X)$ and $\mathsf{F}_{2ij}(X)$ are defined as in Eq. (7) (but they may depend on β_i and γ_{ij}). But then

$$\mathsf{F}_0(X) + \alpha_{ijk}\mathsf{F}_{1ij}(X) + \alpha_{ijk}^2\mathsf{F}_{2ij}(X) = \mathsf{Z}_{\mathbb{H}}(X)\mathsf{t}_{ijk}(X) \ .$$

Thus, for every $s \in [1, n]$,

$$\mathsf{F}_0(\omega^s) + \alpha_{ijk}\mathsf{F}_{1ij}(\omega^s) + \alpha_{ijk}^2\mathsf{F}_{2ij}(\omega^s) = 0 \ .$$

Let

$$\boldsymbol{A}_{ij} = \begin{pmatrix} 1 & \alpha_{ij1} & \alpha_{ij1}^2 \\ 1 & \alpha_{ij2} & \alpha_{ij2}^2 \\ 1 & \alpha_{ij3} & \alpha_{ij3}^2 \end{pmatrix} \ .$$

be a Vandermonde matrix. Then, $\boldsymbol{A}_{ij} \cdot (\mathsf{F}_0(\omega^s), \mathsf{F}_{1ij}(\omega^s), \mathsf{F}_{2ij}(\omega^s))^\intercal = 0$. Since α_{ij1}, α_{ij2}, and α_{ij3} are distinct, $\boldsymbol{A}_{ij}$ is invertible. Thus, $\mathsf{F}_0(\omega^s) = \mathsf{F}_{1ij}(\omega^s) = \mathsf{F}_{2ij}(\omega^s) = 0$. We analyze these three equalities individually.

$\underline{\mathsf{F}_0(\omega^s) = 0.}$ Let $\bar{w}_s := \mathsf{a}(\omega^s)$, $\bar{w}_{n+s} := \mathsf{b}(\omega^s)$, and $\bar{w}_{2n+s} := \mathsf{c}(\omega^s)$. Then, $\mathsf{F}_0(\omega^s) = 0$ iff

$$\mathsf{q}_{\mathsf{M}s}\bar{w}_s\bar{w}_{n+s} + \mathsf{q}_{\mathsf{L}s}\bar{w}_s + \mathsf{q}_{\mathsf{R}s}\bar{w}_{n+s} + \mathsf{q}_{\mathsf{O}s}\bar{w}_{2n+s} + \mathsf{q}_{C s} + \mathsf{PI}(\omega^s) = 0 \ .$$

For $s > \ell$, $\mathsf{PI}(\omega^s) = 0$ and we obtain the constraint in Eq. (5). Recall that for $s \leq \ell$, $\mathsf{q}_{\mathsf{M}s} = \mathsf{q}_{\mathsf{R}s} = \mathsf{q}_{\mathsf{O}s} = \mathsf{q}_{C s} = 0$ and $\mathsf{q}_{\mathsf{L}s} = -1$ (see Eq. (4)). Thus, $-\bar{w}_s + \mathsf{PI}(\omega^s) = -\bar{w}_s + w_s = 0$. It follows that $w_s = \bar{w}_s$ for $1 \leq s \leq \ell$. Hence, $\mathsf{a}(X)$ encoded the public statement $\mathbb{x}$ correctly.

$\underline{\mathsf{F}_{2ij}(\omega^s) = 0.}$ We get $\mathsf{F}_{2ij}(\omega^s) = (\mathsf{z}_{ij}(\omega^s) - 1)\mathsf{L}_1(\omega^s) = 0$ for all $\omega^s \in \mathbb{H}$. Thus, $\mathsf{z}_{ij}(\omega) = 1$.

$\underline{\mathsf{F}_{1ij}(\omega^s) = 0.}$ We will conclude from $\mathsf{F}_{1ij}(\omega^s) = 0$ that $w_j = w_{\sigma(j)}$ for all $j \in [1, 3n]$. We will first prove a warm-up lemma (Lemma 4), which we will later expand to a more technical result Lemma 5 that better suits our needs.

Lemma 4. *Let σ be a permutation on $[1, n]$ and $a_1, \ldots, a_n, b_1, \ldots, b_n \in \mathbb{F}$. Let $\beta_1, \ldots, \beta_{n+1} \in \mathbb{F}$ be mutually distinct and $\gamma_1, \ldots, \gamma_{n+1} \in \mathbb{F}$ be mutually distinct. If*

$$\prod_{s=1}^{n}(a_s + \beta_i\omega^s + \gamma_j) = \prod_{s=1}^{n}(b_s + \beta_i\omega^{\sigma(s)} + \gamma_j)$$

for all $i, j \in [1, n+1]$, then $b_s = a_{\sigma(s)}$ for all $s \in [1, n]$.

Proof. Consider the polynomials

$$f(Y, Z) := \prod_{s=1}^{n}(a_s + \omega^s Y + Z)$$

and

$$g(Y, Z) := \prod_{s=1}^{n}(b_s + \omega^{\sigma(s)} Y + Z) \ ;$$

the degree of Y or Z in both f and g is at most n. Denote $B := \{\beta_s\}_{s=1}^{n+1}$ and $\Gamma := \{\gamma_s\}_{s=1}^{t+1}$.

Define the Lagrange polynomials $L_s^B(Y) := \prod_{i \neq s} \frac{Y - \beta_i}{\beta_s - \beta_i}$ and $L_s^\Gamma(Z) := \prod_{k \neq s} \frac{Z - \gamma_k}{\gamma_s - \gamma_k}$ for $s = 1, \ldots, n+1$. Clearly, $\{L_i^B(Y) \cdot L_j^\Gamma(Z)\}_{ij}$ is a basis of bivariate polynomials where each variable has at most degree n. Thus, we can express f and g uniquely as

$$f(Y, Z) := \sum_{i=1}^{n+1}\sum_{j=1}^{n+1} f_{ij} L_i^B(Y) L_j^\Gamma(Z) \ ,$$
$$g(Y, Z) := \sum_{i=1}^{n+1}\sum_{j=1}^{n+1} g_{ij} L_i^B(Y) L_j^\Gamma(Z) \ ,$$

for some $f_{ij}, g_{ij} \in \mathbb{F}$. Since by the hypothesis of the lemma, $f(\beta_i, \gamma_j) = f_{ij} = g(\beta_i, \gamma_j) = g_{ij}$ for all $i, j \in [1, n+1]$, it follows that $f(X, Y) = g(X, Y)$.

Observe that the polynomials $a_i + \omega^i Y + Z$ and $b_s + \omega^{\sigma(s)} Y + Z$ are irreducible. Thus, $f(X, Y) = g(X, Y)$ implies that for every s there exists exactly one i such that

$$a_s + \omega^s Y + Z = b_i + \omega^{\sigma(i)} Y + Z \ .$$

Thus, $\omega^i = \omega^{\sigma(s)}$, which implies that $i = \sigma(s)$, which in turn implies that $a_i = a_{\sigma(s)} = b_s$. The result follows since the above holds for all $s \in [1, n]$. □

Next, we prove a more involved version of Lemma 4, that directly applies to Plonk. Let us first define the polynomials

$$f_\vartheta(Y,Z) := \prod_{s=1}^{n}(w_{\vartheta n+s} + k_\vartheta \omega^s Y + Z)$$

and

$$g_\vartheta(Y,Z) := \prod_{s=1}^{n}(w_{\vartheta n+s} + S_{\sigma,(\vartheta+1)}(\omega^s)Y + Z) \ .$$

for $\vartheta \in \{0,1,2\}$, where $k_0 := 1$ and k_1, k_2 are defined as in Section 5.1.

Lemma 5. *If*

$$\prod_{\vartheta=0}^{2} f_\vartheta(\beta_i, \gamma_{ij}) = \prod_{\vartheta=0}^{2} g_\vartheta(\beta_i, \gamma_{ij})$$

for all $i, j \in [1, 3n+1]$, then $w_s = w_{\sigma(s)}$ for all $s \in [1, 3n]$.

Proof. Let $f(Y,Z) := \prod_{\vartheta=0}^{2} f_\vartheta(Y,Z)$ and $g(Y,Z) := \prod_{\vartheta=0}^{2} g_\vartheta(Y,Z)$. Both f and g have at most degree $3n$ in both Y and Z. Using the same reasoning as in Lemma 4, we conclude that $f(Y,Z) = g(Y,Z)$. The polynomials $w_{\vartheta n+s} + k_\vartheta \omega^s Y + Z$ are irreducible and pairwise distinct for all $\vartheta \in \{0,1,2\}$ and $s \in [1,n]$. The same holds for the polynomials $w_{\vartheta n+s} + S_{\sigma,(\vartheta+1)}(\omega^s)Y + Z$.

Recall that $\sigma^*(i) = \mathbb{H}'[\sigma(i)]$ for all $i \in [1, 3n]$ and

$$S_{\sigma,(\vartheta+1)}(\omega^s) = \sigma^*(\vartheta n + s) = \mathbb{H}'[\sigma(\vartheta n + s)] \ .$$

Therefore, we can express

$$f(Y,Z) = \prod_{j=1}^{3n}(w_i + Y\mathbb{H}'[i] + Z), \ \ g(Y,Z) = \prod_{i=1}^{3n}(w_i + Y\mathbb{H}'[\sigma(i)] + Z) \ .$$

Just as in Lemma 4, each factor $w_i + Y\mathbb{H}'[\sigma(i)] + Z$ of f is equal to exactly one factor $w_{i'} + Y\mathbb{H}'[i'] + Z$. Thus, $\mathbb{H}'[\sigma(i)] = \mathbb{H}'[i']$ and $w_i = w_{i'}$. The first identity implies that $\sigma(i) = i'$ and the second implies that $w_i = w_{i'} = w_{\sigma(i)}$. Since this holds for all $i \in [1, 3n]$, we have proven the lemma. □

We prove one more small result before proving our main result.

Lemma 6. *For all s, i, j,*

$$(w_s + \beta_i S_{\sigma 1}(\omega^s) + \gamma_{ij}) \cdot (w_{n+s} + \beta_i S_{\sigma 2}(\omega^s) + \gamma_{ij}) \cdot (w_{2n+s} + \beta_i \cdot \mathsf{S}_{\sigma 3}(\omega^s) + \gamma_{ij}) \neq 0.$$

Proof. Consider $w_s + \beta_i S_{\sigma 1}(\omega^s) + \gamma_{ij}$ as an example. We already noted that $w_s + S_{\sigma 1}(\omega^s)Y + Z$ is an irreducible polynomial, which means it has no roots in $\mathbb{F}$. Thus, (β_i, γ_{ij}) is not a root and $w_s + \beta_i S_{\sigma 1}(\omega^s) + \gamma_{ij} \neq 0$. Similarly, the other two factors are non-zero. Hence, their product is non-zero. □

We will inductively show that

$$\mathsf{z}_{ij}(\omega^{s+1}) = \prod_{t=1}^{s} \frac{(w_t + \beta_i \omega^t + \gamma_{ij}) \cdot (w_{n+t} + \beta_i k_1 \omega^t + \gamma_{ij}) \cdot (w_{2n+t} + \beta_i k_2 \omega^t + \gamma_{ij})}{(w_t + \beta_i S_{\sigma 1}(\omega^t) + \gamma_{ij}) \cdot (w_{n+t} + \beta_i S_{\sigma 2}(\omega^t) + \gamma_{ij}) \cdot (w_{2n+t} + \beta_i \mathsf{S}_{\sigma 3}(\omega^t) + \gamma_{ij})} \ . \tag{25}$$

Since we already showed that $\mathsf{z}_{ij}(\omega) = 1$, the claim holds for $s = 0$. Suppose the statement holds for $\mathsf{z}_{ij}(\omega^s)$. From $\mathsf{F}_{1ij}(\omega^i) = 0$ (see Eq. (7)), we conclude

$$\mathsf{z}_{ij}(\omega^{s+1}) = \frac{\mathsf{z}_{ij}(\omega^s) \cdot (w_s + \beta_i\omega^s + \gamma_{ij}) \cdot (w_{n+s} + \beta_i k_1\omega^s + \gamma_{ij}) \cdot (w_{2n+s} + \beta_i k_2\omega^s + \gamma_{ij})}{(w_s + \beta_i S_{\sigma 1}(\omega^s) + \gamma_{ij}) \cdot (w_{n+s} + \beta_i S_{\sigma 2}(\omega^s) + \gamma_{ij}) \cdot (w_{2n+s} + \beta_i \mathsf{S}_{\sigma 3}(\omega^s) + \gamma_{ij})} \ .$$

Note that the division is well-defined according to Lemma 6 and Eq. (25) follows by expanding $\mathsf{z}_{ij}(\omega^s)$.

Since $\omega^{n+1} = \omega$, we have $\mathsf{z}_{ij}(\omega^{n+1}) = 1$, implying that

$$\begin{aligned}
&\prod_{t=1}^{n}(w_t + \beta_i\omega^t + \gamma_{ij}) \cdot (w_{n+t} + \beta_i k_1\omega^t + \gamma_{ij}) \cdot (w_{2n+t} + \beta_i k_2\omega^t + \gamma_{ij}) \\
&= \prod_{t=1}^{n}(w_t + \beta_i S_{\sigma 1}(\omega^t) + \gamma_{ij}) \cdot (w_{n+t} + \beta_i S_{\sigma 2}(\omega^t) + \gamma_{ij}) \cdot (w_{2n+t} + \beta_i \cdot \mathsf{S}_{\sigma 3}(\omega^t) + \gamma_{ij}) \ .
\end{aligned}$$

We can conclude from Lemma 5 that $w_i = w_{\sigma(i)}$ for all $i \in [1, 3n]$. Hence, $\mathbb{w}$ also satisfies the final constraint in Eq. (6). □

E Fiat-Shamir Transformation of Plonk

Let us recall the standard definition of adaptive knowledge-soundness for a non-interactive argument in the programmable ROM. (Recall that the extractor can program the random oracle in programmable ROM.) See, e.g., [AFK22].

Definition 4. *Let Π be a non-interactive argument in the ROM for a relation $\mathcal{R}$. Π has* adaptive knowledge-soundness *with knowledge error $k : \mathbb{N}^2 \to [0, 1]$, if there exists a polynomial $q(X)$, and an expected PPT extractor Ext, such that for any PPT adversary $\mathcal{A}$, $\mathsf{Adv}^{\mathrm{aks}}_{\Pi,\mathcal{A},\mathsf{Ext}}(\lambda) :=$*

$$\Pr\left[\begin{array}{l|l} \mathsf{V}(\mathtt{srs}, \mathbb{x}, \pi) = 1 & \mathsf{p} \leftarrow \mathsf{Pgen}(1^\lambda); \\ \wedge\, (\mathtt{srs}, \mathbb{x}, \mathbb{w}) \in \mathcal{R} & (\mathtt{srs}, \mathtt{td}_{\mathtt{srs}}) \leftarrow \mathsf{KGen}(\mathsf{p}); r \leftarrow\!\!\$\, \mathsf{RAND} \\ & (\mathbb{x}, \mathbb{w}, \pi) \leftarrow \mathsf{Ext}^{\mathcal{A}(\mathtt{srs};r)}(\mathtt{srs}, \mathtt{td}_{\mathtt{srs}}) \end{array}\right] \geq \frac{\varepsilon(\mathcal{A}) - k(\mathsf{Q}, \lambda)}{q(\lambda)} \ ,$$

where $\mathcal{A}$ can make up to Q RO queries and $\varepsilon(\mathcal{A})$ is the probability (over the answers of RO) that $\mathcal{A}$ makes the verifier accept. The notation $\mathsf{Ext}^{\mathcal{A}(\mathtt{srs};r)}$ indicates that the extractor can only call the deterministic adversary defined by fixing the random coins and the $\mathtt{srs}$. Here and in the rest of this section, RAND is the space of the possible random coins for the adversary $\mathcal{A}$.

E.1 Fiat-Shamir Knowledge-Soundness from CSS

In this section, we prove optimally tight (in the number of oracle calls allowed to the adversary) knowledge-soundness of Fiat-Shamir arguments in the ROM obtained from computational special sound (CSS) interactive arguments. Since the proof is a small modification of the one given in [AFK22] for the Fiat-Shamir compilation of (perfect) special-sound interactive arguments, we give a proof sketch.

In our proof, we use a routine defined in [AFK22] to compute a tree of accepting transcripts, given black-box access to a deterministic prover. In [AFK22], this routine coincides with the knowledge extractor. In particular, they define a knowledge-extractor Ext_{rew} with the following properties.

- Ext_{rew} has oracle access to a deterministic adversary $\mathcal{A}$ that can make up to Q queries to the random oracle. We write $\mathsf{Ext}^{\mathcal{A}}_{rew}$ to specify the deterministic adversary. Recall that Ext_{rew} can program the random oracle and forward arbitrary answers to $\mathcal{A}$ queries.

– Ext_{rew} runs in expected polynomial time (expected PPT).
– If the deterministic $\mathcal{A}$ computes a valid proof for a given choice of the RO, then Ext_{rew} computes a $\boldsymbol{\kappa}$-tree-of-accepting transcripts, except with negligible probability. Particularly, the loss of the extractor is linear in Q, the number of RO queries allowed to $\mathcal{A}$.

Note that the results in [AFK22] are worded differently. In particular, the requirement of computing a tree of accepting transcripts does not appear in the statement of any lemma. Instead, they have lemmas arguing about the extractor computing a valid witness; see Proposition 1 (Correctness) and Proposition 2 (Run Time and Success Probability) in [AFK22]. In the case of perfect special-soundness, computing a tree of accepting transcripts is equivalent to computing a witness since the special-soundness extractor never fails. In the case of CSS arguments, similarly to what is done in [LPS24], we show through a reduction that the special-soundness extractor $\mathsf{Ext}_{\mathsf{ss}}$ fails at most with negligible probability in computing a valid witness when given as an input a tree of accepting transcripts computed by Ext_{rew}.

We first restate, without proving, results from [AFK22] in the convenient terminology for our case. Let us first fix the notation $\mathsf{Ext}_{rew}^{\mathcal{A},\mathsf{RO}}$ to indicate that the extractor replies to $\mathcal{A}$ RO queries with a fixed choice of the oracle RO. Since we require that the adversary can make up to Q queries, RO can be seen as a random vector of length Q and can be implemented by lazy sampling.

Let Π be a computational $\boldsymbol{\kappa}$-special sound interactive argument for a relation $\mathcal{R}$. Let Π^{FS} be the non-interactive argument obtained by the Fiat-Shamir compilation of Π. For each PPT adversary $\mathcal{A}$ against the knowledge-soundness of Π^{FS} and extractor Ext, define the event

$$\mathsf{good}(\mathtt{srs}, r, \mathsf{Ext}) := \left[\begin{array}{l|l} \mathsf{V}(\mathtt{srs}, \mathbb{x}, \pi) = 1 \wedge & \mathsf{RO} \leftarrow_{\$} \mathbb{F}_p^{\mathsf{Q}+\mu}; \\ \mathcal{T} \text{ is a } \boldsymbol{\kappa}\text{-tree of accepting} & (\mathbb{x}, \pi) \leftarrow \mathcal{A}^{\mathsf{RO}}(\mathtt{srs}; r); \\ \text{transcripts} & \mathcal{T} \leftarrow \mathsf{Ext}^{\mathcal{A}(\mathtt{srs};r),\mathsf{RO}} \end{array}\right] ,$$

where $\mathsf{Ext}^{\mathcal{A},\mathsf{RO}}$ has black-box access to the deterministic adversary defined as $\mathcal{A}(\mathtt{srs}; r)$ for an arbitrary choice of $\mathtt{srs}$ and arbitrary random coins r.

Lemma 7 ([AFK22]). *Let $\boldsymbol{\kappa} = (\kappa_1, \ldots, \kappa_\mu)$. Assume κ_i and Q are polynomial in the security parameter. There exists an extractor Ext_{rew} such that*

$$\Pr[\mathsf{good}(\mathtt{srs}, r, \mathsf{Ext}_{rew})] \geq \frac{\varepsilon(\mathcal{A}(\mathtt{srs}; r)) - (\mathsf{Q}+1) \cdot \mathsf{Er}}{1 - \mathsf{Er}} ,$$

where $\varepsilon(\mathcal{A}(\mathtt{srs}; r))$ is the probability, over the choices of RO, that $\mathcal{A}(\mathtt{srs}; r)$ makes the verifier accept and $\mathsf{Er} = 1 - \prod_{j=1}^{\mu}(1 - (\kappa_j - 1)/p)$. Moreover, $\mathsf{Ext}_{rew}^{\mathcal{A},\mathsf{RO}}$'s expected runtime is bound by a function in $\mathcal{O}\Big(\mathsf{Q}\prod_{j=1}^{\mu}\kappa_j\Big)$.

The proof of the previous lemma is the same as the proof of Proposition 1 (Correctness) and Proposition 2 (Run Time and Success Probability) in [AFK22]. We now state the main result of this section.

Theorem 11. *Let $\boldsymbol{\kappa} = (\kappa_1, \ldots, \kappa_\mu)$. Assume each κ_i and Q are polynomial in the security parameter. Let Π be a $\boldsymbol{\kappa}$-CSS interactive argument for a relation $\mathcal{R}$. Let Π^{FS} be the non-interactive argument obtained by the Fiat-Shamir compilation of Π, and let $\mathsf{Ext}^{\mathcal{A}}$ be the extractor in Fig. 19. Then,*

$$\Pr\left[\begin{array}{l|l} \mathsf{V}(\mathtt{srs}, \mathbb{x}, \pi) = 1 & \mathsf{p} \leftarrow \mathsf{Pgen}(1^\lambda); r \leftarrow_{\$} \mathsf{RAND}; \\ \wedge (\mathtt{srs}, \mathbb{x}, \mathbb{w}) \in \mathcal{R} & (\mathtt{srs}, \mathtt{td}_{\mathtt{srs}}) \leftarrow \mathsf{KGen}(\mathsf{p}); \\ & (\mathbb{x}, \mathbb{w}, \pi) \leftarrow \mathsf{Ext}^{\mathcal{A}(\mathtt{srs},r)}(\mathtt{srs}); \end{array}\right] \geq \frac{\varepsilon_{\mathcal{A}} - \varepsilon_{ss} - (\mathsf{Q}+1)\mathsf{Er}}{1 - \mathsf{Er}} ,$$

$\mathsf{Ext}^{\mathcal{A}(\mathtt{srs},r)}$

1 : $\mathsf{RO} \leftarrow\!\!\$\ \mathbb{F}_p^{\mathsf{Q}}$;
2 : $(\mathbb{x}, \pi) \leftarrow \mathcal{A}^{\mathsf{RO}}(\mathtt{srs}; r)$;
3 : $\mathcal{T} \leftarrow \mathsf{Ext}_{rew}^{\mathcal{A}(\mathtt{srs},r),\mathsf{RO}}$;
4 : $\mathbb{w} \leftarrow \mathsf{Ext}_{\mathsf{ss}}(\mathtt{srs}, \mathcal{T})$;
5 : **return** $(\mathbb{x}, \mathbb{w}, \pi)$

Fig. 19. The knowledge extractor for Π^{FS}.

where Q *is the maximum number of queries allowed by* $\mathcal{A}$ *to the random oracle,* ε_{ss} *is an upper bound of the advantage of any expected PPT against the* $\boldsymbol{\kappa}$*-CSS of* Π*,* $\varepsilon(\mathcal{A})$ *is the probability that* $\mathcal{A}$ *makes the verifier accept, and*

$$\mathsf{Er} = 1 - \prod_{j=1}^{\mu} \left(1 - \frac{\kappa_j - 1}{p}\right) \ .$$

Proof (Sketch). The knowledge extractor for Π^{FS} is defined in Fig. 19. Since both $\mathcal{A}$ and $\mathsf{Ext}_{\mathsf{ss}}$ are (strict) PPT, and Ext_{rew} is expected PPT, then Ext is expected PPT too.

Let us define the event B as

$$B := \left[\begin{array}{l} \mathsf{V}(\mathtt{srs}, \mathbb{x}, \pi) = 1 \wedge \\ \mathcal{T} \text{ is a } \boldsymbol{\kappa}\text{-tree-of-accepting} \\ \text{transcripts} \end{array} \middle| \begin{array}{l} (\mathtt{srs}, \mathtt{td}_{\mathtt{srs}}) \leftarrow \mathsf{KGen}(\mathsf{p}); r \leftarrow\!\!\$\ \mathsf{RAND} \\ \mathsf{RO} \leftarrow\!\!\$\ \mathbb{F}_p^{\mathsf{Q}}; \\ (\mathbb{x}, \pi) \leftarrow \mathcal{A}^{\mathsf{RO}}(\mathtt{srs}; r); \\ \mathcal{T} \leftarrow \mathsf{Ext}_{rew}^{\mathcal{A}(\mathtt{srs},r),\mathsf{RO}}; \end{array} \right] ,$$

where the probability is over the sampling of $\mathtt{srs}$, random coins r, and oracle queries RO. We have that

$$\begin{aligned} \Pr[B] &= \sum_{\tilde{\mathtt{srs}},\tilde{r}} \Pr[\mathtt{srs} = \tilde{\mathtt{srs}} \ \wedge \ r = \tilde{r}] \Pr[\mathsf{good}(\tilde{\mathtt{srs}}, \tilde{r}, \mathsf{Ext}_{rew}^{\mathcal{A}(\tilde{\mathtt{srs}},\tilde{r})})] \\ &\geq \sum_{\tilde{\mathtt{srs}},\tilde{r}} \Pr[\mathtt{srs} = \tilde{\mathtt{srs}} \ \wedge \ r = \tilde{r}] \frac{\varepsilon(\mathcal{A}(\tilde{\mathtt{srs}}, \tilde{r})) - (\mathsf{Q}+1) \cdot \mathsf{Er}}{1 - \mathsf{Er}} \\ &= \frac{\varepsilon(\mathcal{A}) - (Q+1)\mathsf{Er}}{1 - \mathsf{Er}} \ . \end{aligned}$$

Here the first inequality follows from Theorem 11 and the second inequality uses the fact that $\sum_{\tilde{\mathtt{srs}},\tilde{r}} \Pr[\mathtt{srs} = \tilde{\mathtt{srs}} \ \wedge \ r = \tilde{r}] \cdot \varepsilon(\mathcal{A}(\tilde{\mathtt{srs}}, \tilde{r})) = \varepsilon(\mathcal{A})$.

Note that the event B corresponds to the event that the tree $\mathcal{T}$ computed in line 3 by Ext is a $\boldsymbol{\kappa}$-tree-of-accepting transcripts. Finally, we prove that

$$\Pr\left[\begin{array}{l} \mathsf{V}(\mathtt{srs}, \mathbb{x}, \pi) = 1 \\ \wedge (\mathtt{srs}, \mathbb{x}, \mathbb{w}) \in \mathcal{R} \end{array} \middle| \begin{array}{l} (\mathtt{srs}, \mathtt{td}_{\mathtt{srs}}) \leftarrow \mathsf{KGen}(\mathsf{p}); \\ r \leftarrow\!\!\$\ \mathsf{RAND} \\ (\mathbb{x}, \mathbb{w}, \pi) \leftarrow \mathsf{Ext}^{\mathcal{A}(\mathtt{srs},r)}(\mathtt{srs}); \end{array} \right] \geq \Pr[B](1 - \varepsilon_{ss}(\lambda))$$

for a negligible function ε_{ss}.

Let us consider the expected PPT CSS adversary $\mathcal{B}$, that runs $\mathsf{Ext}^{\mathcal{A}(\mathtt{srs};r)}$, until the line 3, to obtain $\mathcal{T}$, and then outputs $\mathcal{T}$. Conditioned on the event B happening. Thus, $\mathcal{T}$ being a tree of accepting transcripts, the probability that $\mathcal{B}$ wins the CSS game is equal to the probability that $\mathbb{w}$ computed by $\mathsf{Ext}^{\mathcal{A}(\mathtt{srs};r)}$ at line 1 is not a valid witness for $\mathbb{x}$. Therefore, we need to

show that there exists a negligible function ε_{ss} such that, for each expected PPT, we have $\mathsf{Adv}^{\mathrm{ss}}_{\mathsf{Pgen},\mathsf{PC},\mathsf{Ext}_{\mathsf{ss}},\mathcal{B},\boldsymbol{\kappa}}(\lambda) \leq \varepsilon_{ss}(\lambda)$. We recall a proposition from [LPS24], adapting the notation for the particular case of CSS we are interested in this context.

Lemma 8. *[Lemma 6 [LPS24]] Let Π be a $\boldsymbol{\kappa}$-CSS interactive argument. Thus for all PPT $\mathcal{A}$, we have $\mathsf{Adv}^{\mathrm{ss}}_{\mathsf{Pgen},\mathsf{PC},\mathsf{Ext}_{\mathsf{ss}},\mathcal{A},\boldsymbol{\kappa}}(\lambda) \approx_\lambda 0$. Then for any expected PPT $\mathcal{B}$, $\mathsf{Adv}^{\mathrm{ss}}_{\mathsf{Pgen},\mathsf{PC},\mathsf{Ext}_{\mathsf{ss}},\mathcal{B},\boldsymbol{\kappa}}(\lambda) \approx_\lambda 0$.*

The existence of a negligible function ε_{ss} follows directly from Π being $\boldsymbol{\kappa}$-CSS and Lemma 8.

To conclude the proof, we note that

$$\begin{aligned}
\Pr[B](1-\varepsilon_{ss}(\lambda)) &\geq \frac{\varepsilon(\mathcal{A}) - (\mathsf{Q}+1)\cdot \mathsf{Er}}{1-\mathsf{Er}}(1-\varepsilon_{ss}(\lambda)) \\
&\geq \frac{\varepsilon(\mathcal{A})(1-\varepsilon_{ss}(\lambda)) - (\mathsf{Q}+1)\mathsf{Er}(1-\varepsilon_{ss}(\lambda))}{1-\mathsf{Er}} \\
&\geq \frac{\varepsilon(\mathcal{A}) - \varepsilon_{ss}(\lambda) - (\mathsf{Q}+1)\mathsf{Er}}{1-\mathsf{Er}} \ ,
\end{aligned}$$

where the last line follows from $\varepsilon_{ss}(\lambda) \geq \varepsilon_{ss}(\lambda)\varepsilon(\mathcal{A})$ and $1-\varepsilon_{ss}(\lambda) \leq 1$. □

E.2 Knowledge-Soundness of Plonk in the ROM

This section summarizes our results so far to show the knowledge-soundness of (non-interactive) Plonk. For readability, we ignore the negligible loss, proportional to $1/p$, introduced by the verifier's batch of the two verification equations. From Plonk description Section 5.2 and Theorem 6, we get

$$\begin{aligned}
\mathsf{Er} =& 1 - \left(1-\frac{3n}{p}\right)\left(1-\frac{3n}{p}\right)\left(1-\frac{2}{p}\right)\left(1-\frac{12n+20}{p}\right)\left(1-\frac{5}{p}\right) \\
=& \frac{18n+27}{p} + \mathcal{O}\left(\frac{1}{p^2}\right) \ .
\end{aligned} \tag{26}$$

We can now state the following corollary.

Corollary 1 (Knowledge-Soundness of Plonk). *Suppose the ARSDH assumption and the SplitRSDH assumption hold; then Plonk is knowledge-sound in the ROM. In particular, there exists an extractor Ext, such that, for each PPT adversary $\mathcal{A}$ that can make up to Q queries to the RO, there exists an expected PPT adversary $\mathcal{B}$ against the ARSDH assumption and an expected PPT adversary $\mathcal{C}$ against the SplitRSDH assumption such that*

$$\begin{aligned}
&\Pr\left[\begin{array}{l|l} \mathsf{V}(\mathtt{srs},\mathbb{x},\pi)=1 & (\mathtt{srs},\mathtt{td}_{\mathtt{srs}}) \leftarrow \mathsf{KGen}(\mathsf{p}); r \leftarrow\!\!\$\, \mathsf{RAND} \\ \wedge (\mathtt{srs},\mathbb{x},\mathbb{w}) \in \mathcal{R} & (\mathbb{x},\mathbb{w},\pi) \leftarrow \mathsf{Ext}^{\mathcal{A}(\mathtt{srs},r)}(\mathtt{srs}); \end{array}\right] \\
&\geq \frac{\varepsilon(\mathcal{A}) - \mathsf{Adv}^{\mathrm{arsdh}}_{\mathsf{Pgen},n+5,\mathbb{G}_1,\mathcal{B}}(\lambda) - \mathsf{Adv}^{\mathrm{splitrsdh}}_{\mathsf{Pgen},n,\mathbb{G}_1,\mathcal{C}}(\lambda) - (\mathsf{Q}+1)\cdot\mathsf{Er}}{1-\mathsf{Er}} \ ,
\end{aligned}$$

where Er is defined in Eq. (26).

To show how our results improve on this, we write the knowledge-soundness theorem for Plonk, which was known prior to our work. Recall first the knowledge-soundness error of the interactive version of Plonk, proven in [GWC19]. Summing up, Lemma 4.7 and Corollary 7.2 of [GWC19] we get that the interactive Plonk is knowledge-sound in the AGM, under the $2(n+5)$-PDL assumption. Moreover, as stated in [AFK22], when applying the Fiat-Shamir transform to a $(2\mu+1)$ round knowledge-sound interactive argument, the resulting non-interactive argument is knowledge-sound in the ROM, with a loss of Q^μ. Here, Q is again the maximum number of RO queries allowed to the knowledge-soundness adversary. We state the following proposition about Plonk's knowledge-soundness guarantee before our work.

Fact 2 (Prior to our work.) *Suppose the PDL assumption holds, then Plonk is knowledge-sound in the ROM and the AGM. In particular, for each algebraic PPT adversary $\mathcal{A}$, there exists an expected PPT extractor $\mathsf{Ext}_{\mathcal{A}}$ that can make up to Q queries to the RO, and an adversary $\mathcal{B}$ to the PDL assumption such that*

$$\Pr\left[\begin{array}{l|l} \mathsf{V}(\mathtt{srs},\mathtt{x},\pi)=1 & (\mathtt{srs},\mathtt{td}_{\mathtt{srs}})\leftarrow\mathsf{KGen}(\mathsf{p});\\ \wedge(\mathtt{srs},\mathtt{x},\mathtt{w})\notin\mathcal{R} & (\mathtt{x},\mathtt{w},\pi)\leftarrow\mathsf{Ext}_{\mathcal{A}}(\mathtt{srs});\end{array}\right]\leq\mathsf{Adv}^{\mathrm{pdl}}_{\mathsf{Pgen},2n+10,\mathsf{p},\mathcal{B}}(\lambda)\mathcal{O}\big(\mathsf{Q}^4\big).$$

Since $\Pr[V=1\wedge(\mathtt{srs},\mathtt{x},\mathtt{w})\in\mathcal{R}]=1-\Pr(V=0)-\Pr[V=1\wedge(\mathtt{srs},\mathtt{x},\mathtt{w})\notin\mathcal{R}]$*, we get the following equation, which is easier to compare to our result from above.*

$$\Pr\left[\begin{array}{l|l} \mathsf{V}(\mathtt{srs},\mathtt{x},\pi)=1 & (\mathtt{srs},\mathtt{td}_{\mathtt{srs}})\leftarrow\mathsf{KGen}(\mathsf{p});\\ \wedge(\mathtt{srs},\mathtt{x},\mathtt{w})\in\mathcal{R} & (\mathtt{x},\mathtt{w},\pi)\leftarrow\mathsf{Ext}_{\mathcal{A}}(\mathtt{srs});\end{array}\right]$$
$$\geq\varepsilon(\mathcal{A})-\mathsf{Adv}^{\mathrm{pdl}}_{\mathsf{Pgen},2n+10,\mathsf{p},\mathcal{B}}(\lambda)\mathcal{O}\big(\mathsf{Q}^4\big)\ .$$

Note that the previous security bound is not optimally tight (in terms of Q) and relies on two different ideal models.

F Zero-Knowledge of SanPlonk

We recall the zero-knowledge property of a non-interactive argument.Below $\mathcal{UR}_{\mathsf{p},n}$ is a family of binary relations parameterized by the system parameters p and an integer n. For $\mathcal{R}\in\mathcal{UR}_{\mathsf{p},n}$ and $\mathtt{srs}\in\mathrm{range}(\mathsf{KGen}(\mathsf{p},n))$.

A non-interactive argument is (statistical) *zero-knowledge* if there exists a PPT simulator Sim, s.t. for all unbound $\mathcal{A}=(\mathcal{A}_1,\mathcal{A}_2)$, all $\mathsf{p}\in\mathrm{range}(\mathsf{Pgen})$, all $n\in\mathsf{poly}(\lambda)$,

$$\Pr\left[\begin{array}{c|c}\mathcal{A}_2(st,\pi)=1\wedge & (\mathtt{srs},\mathtt{td}_{\mathtt{srs}})\leftarrow\mathsf{KGen}(\mathsf{p},n);\\ \mathcal{R}(\mathtt{x},\mathtt{w})\wedge\mathcal{R}\in\mathcal{UR}_{\mathsf{p},n}; & (\mathcal{R},\mathtt{x},\mathtt{w},st)\leftarrow\mathcal{A}_1(\mathtt{srs});\\ & \pi\leftarrow\mathsf{P}(\mathcal{R},\mathtt{srs},\mathtt{x},\mathtt{w})\end{array}\right]\approx_s$$
$$\Pr\left[\begin{array}{c|c}\mathcal{A}_2(st,\pi)=1\wedge & (\mathtt{srs},\mathtt{td}_{\mathtt{srs}})\leftarrow\mathsf{KGen}(\mathsf{p},n);\\ \mathcal{R}(\mathtt{x},\mathtt{w})\wedge\mathcal{R}\in\mathcal{UR}_{\mathsf{p},n} & (\mathcal{R},\mathtt{x},\mathtt{w},st)\leftarrow\mathcal{A}_1(\mathtt{srs});\\ & \pi\leftarrow\mathsf{Sim}(\mathcal{R},\mathtt{srs},\mathtt{td}_{\mathtt{srs}},\mathtt{x})\end{array}\right].$$

Here, $\approx_s$ denotes the statistical distance as a function of λ. *Π is perfect zero-knowledge* if the above probabilities are equal.

Recently, Sefranek [Sef24] showed that an earlier version of Plonk did not satisfy statistical zero-knowledge since the polynomials $\mathsf{t}_{\mathsf{lo}}(X)$, $\mathsf{t}_{\mathsf{mid}}(X)$, $\mathsf{t}_{\mathsf{hi}}(X)$ were not randomized. He corrects this mistake and proves statistical zero-knowledge of the corrected Plonk. The correction is a part of the Plonk now, [GWC19]. Sefranek also mentions that SmallPlonk, which does not split $\mathsf{t}(X)$, is simulatable. Our paper contains the current (corrected) version of Plonk, so we will not repeat the zero-knowledge proofs of Plonk and SmallPlonk and instead refer the reader to [Sef24]. However, we provide a similar proof of zero-knowledge for SanPlonk.

We recall the following well-known lemma; see, e.g., [Sef24].

Lemma 9 ([Sef24]). *Let $\mathsf{f}(X)\in\mathbb{F}[X]$ and $x_1,\ldots,x_k$ be distinct values in $\mathbb{F}\setminus\mathbb{H}$. Assume $\widetilde{\mathsf{f}}(X)=\mathsf{f}(X)+\varrho(X)\mathsf{Z}_{\mathbb{H}}(X)$ for $\varrho(X)\leftarrow\!\!\$\,\mathbb{F}_{\leq k-1}[X]$. Then, $(\widetilde{\mathsf{f}}(x_1),\ldots,\widetilde{\mathsf{f}}(x_k))$ is distributed uniformly over $\mathbb{F}^k$,*

The same claim also holds for $\widetilde{\mathsf{f}}(X)=\mathsf{f}(X)+\varrho(X)$; moreover, then it is true even when $x_1,\ldots,x_k\in\mathbb{F}$, not just in $\mathbb{F}\setminus\mathbb{H}$ (in Lemma 9, we need that $\mathsf{Z}_{\mathbb{H}}(x_i)\neq 0$). We use both results to prove the statistical zero-knowledge of SanPlonk.

1. $a, b, c \leftarrow_\$ \mathbb{F}$.
2. $\beta \leftarrow \mathsf{H}(\mathtt{srs}, \mathbb{x}, [a, b, c]_1, 0)$; $\gamma \leftarrow \mathsf{H}(\mathtt{srs}, \mathbb{x}, [a, b, c]_1, 1)$.
3. $z \leftarrow_\$ \mathbb{F}$.
4. $\alpha \leftarrow \mathsf{H}(\mathtt{srs}, \mathbb{x}, [a, b, c, z]_1)$.
5. Sample $\bar{z}_{x\omega} \leftarrow_\$ \mathbb{F}$ (a candidate value for $\mathsf{z}(x\omega)$ used to compute $\mathsf{F}_1(x)$ in the next step).
6. For $\mathsf{F}_i(X)$ defined as in Eq. (7), set $[\mathsf{t}(x)]_1 \leftarrow \frac{1}{\mathsf{Z}_{\mathbb{H}}(x)}[\mathsf{F}_0(x) + \alpha\mathsf{F}_1(x) + \alpha^2\mathsf{F}_2(x)]_1$.
7. $t_{hi}, t_{mid} \leftarrow_\$ \mathbb{F}$; $[t_{lo}]_1 \leftarrow [\mathsf{t}(x) - t_{mid}x^n - t_{hi}x^{2n}]_1$. Abort if $\mathsf{Z}_{\mathbb{H}}(x) = 0$.
8. $\mathfrak{z} \leftarrow \mathsf{H}(\mathtt{srs}, \mathbb{x}, [a, b, c, z, t_{lo}, t_{mid}, t_{hi}]_1)$.
9. Abort if $\mathfrak{z} = x$ or $\mathfrak{z}\omega = x$.
10. $\bar{a}, \bar{b}, \bar{c}, \bar{z}_\omega \leftarrow_\$ \mathbb{F}$; $\bar{s}_{\sigma 1} \leftarrow \mathsf{S}_{\sigma 1}(\mathfrak{z})$; $\bar{s}_{\sigma 2} \leftarrow \mathsf{S}_{\sigma 2}(\mathfrak{z})$.
11. $\delta \leftarrow \mathsf{H}(\mathtt{srs}, \mathbb{x}, [a, b, c, z, t_{lo}, t_{mid}, t_{hi}]_1, \bar{a}, \bar{b}, \bar{c}, \bar{s}_{\sigma 1}, \bar{s}_{\sigma 2}, \bar{z}_\omega)$.
12. $\bar{t}_{\mathfrak{z}} \leftarrow_\$ \mathbb{F}$.
13. $v \leftarrow \mathsf{H}(\mathtt{srs}, \mathbb{x}, [a, b, c, z, t_{lo}, t_{mid}, t_{hi}]_1, \bar{a}, \bar{b}, \bar{c}, \bar{s}_{\sigma 1}, \bar{s}_{\sigma 2}, \bar{z}_\omega, \bar{t}_{\mathfrak{z}})$.
14. $\mathsf{W} \leftarrow r + v(a - \bar{a}) + v^2(b - \bar{b}) + v^3(c - \bar{c}) + v^4(s_{\sigma 1} - \bar{s}_{\sigma 1}) + v^5(s_{\sigma 2} - \bar{s}_{\sigma 2})$.
15. $[\mathsf{W}_{\mathfrak{z}}]_1 \leftarrow \frac{1}{x - \mathfrak{z}}[\mathsf{W} + v^6(t_{lo} + \delta t_{mid} + \delta^2 t_{hi} - \bar{t}_{\mathfrak{z}})]_1$.
16. $[\mathsf{W}_{\mathfrak{z}\omega}]_1 \leftarrow \frac{1}{x - \mathfrak{z}\omega}[z - \bar{z}_\omega]_1$.
17. Return $([a, b, c]_1; \beta, \gamma; [z]_1; \alpha; [t_{lo}, t_{mid}, t_{hi}]_1; \mathfrak{z}; \bar{a}, \bar{b}, \bar{c}, \bar{s}_{\sigma 1}, \bar{s}_{\sigma 2}, \bar{z}_\omega; \delta; \bar{t}_{\mathfrak{z}}; v; [\mathsf{W}_{\mathfrak{z}}, \mathsf{W}_{\mathfrak{z}\omega}]_1)$.

Fig. 20. SanPlonk's simulator.

Theorem 12. *SanPlonk has statistical zero-knowledge.*

Proof. Consider the following simulation strategy. Recall that SanPlonk's transcript is $([a, b, c]_1; \beta, \gamma; [z]_1; \alpha; [t_{lo}, t_{mid}, t_{hi}]_1; \mathfrak{z}; \bar{a}, \bar{b}, \bar{c}, \bar{s}_{\sigma 1}, \bar{s}_{\sigma 2}, \bar{z}_\omega; \delta; \bar{t}_{\mathfrak{z}}; v; [\mathsf{W}_{\mathfrak{z}}, \mathsf{W}_{\mathfrak{z}\omega}]_1)$. Similarly to Plonk's simulator in [Sef24], given the verifier's random challenges, SanPlonk's simulator works like an honest prover with four crucial differences. First, it creates the commitments a, b, c, z and their openings by sampling them randomly. Second, since $\mathsf{t}(X)$, $\mathsf{W}_{\mathfrak{z}}(X)$, $\mathsf{W}_{\mathfrak{z}\omega}(X)$ might not be polynomials, it computes $[\mathsf{t}(x), \mathsf{W}_{\mathfrak{z}}(x), \mathsf{W}_{\mathfrak{z}\omega}(x)]_1$ by using the trapdoor. Third, it computes $[t_{lo}, t_{mid}, t_{hi}]_1$ by sampling two of the values at random and setting the third one so that $[t_{lo}, t_{mid}, t_{hi}]_1$ agrees with the previously computed value of $[\mathsf{t}(x)]_1$. Fourth, the sanitization value $\bar{t}_{\mathfrak{z}}$ is sampled randomly. In Fig. 20, we present the full simulator for SanPlonk as a zk-SNARK, including the definition of the verifier's challenges as the outputs of the random oracle. Here, H is the random oracle. (For brevity, we refer to Eq. (7) for the formulas of $\mathsf{F}_i(X)$ and $\mathsf{F}(X)$.)

From Lemma 9 it follows that in the honest proof $\mathsf{a}(x)$, $\mathsf{b}(x)$, $\mathsf{c}(x)$, $\mathsf{z}(x)$, $\mathsf{a}(\mathfrak{z})$, $\mathsf{b}(\mathfrak{z})$, $\mathsf{c}(\mathfrak{z})$, $\mathsf{z}(\mathfrak{z}\omega)$, and $\mathsf{z}(x\omega)$ are distributed uniformly randomly and independently. Here, the last element $\bar{z}_{x\omega} = \mathsf{z}(x\omega)$ is not part of the proof transcript, but it is necessary for determining a unique $[\mathsf{t}(x)]_1$. Furthermore, in the honest protocol $\mathsf{t}_{\mathsf{hi}}(X) := \mathsf{t}'_{\mathsf{hi}}(X) - b_{11}$, $\mathsf{t}_{\mathsf{mid}}(X) := \mathsf{t}'_{\mathsf{mid}}(X) - b_{10} - b_{12}X + b_{11}X^n$, and $\mathsf{t}_{\mathsf{lo}}(X) := \mathsf{t}'_{\mathsf{lo}}(X) + b_{10}X^n + b_{12}X^{n+1}$ (as always, we hilight the changes compared to Plonk). Thus, $\mathsf{t}_{\mathsf{hi}}(x)$, $\mathsf{t}_{\mathsf{mid}}(x)$, and $\mathsf{t}_{\mathsf{lo}}(\mathfrak{z})$ are distributed uniformly at random and independently since (resp.) b_{11}, b_{10}, and b_{12} are sampled uniformly at random. In the case of $\mathsf{t}_{\mathsf{lo}}(\mathfrak{z})$, it holds under the assumption that $\mathfrak{z}^{n+1} \neq 0$ (otherwise $b_{12}\mathfrak{z}^{n+1} = 0$). Note that the proof does not reveal $t_{lo}(\mathfrak{z})$. However, $t_{lo}(\mathfrak{z})$ being uniformly random and independent of other elements guarantees that $\bar{t}_{\mathfrak{z}} = \mathsf{t}_{\mathsf{lo}}(\mathfrak{z}) + \delta\mathsf{t}_{\mathsf{mid}}(\mathfrak{z}) + \delta^2\mathsf{t}_{\mathsf{hi}}(\mathfrak{z})$ is uniformly random and independent. In the simulator, we also pick all the mentioned random elements uniformly at random and independently.

The remaining proof elements have only one possible satisfiable value. Namely, there is precisely one possible value of $t_{lo}(x)$ such that $\mathsf{t}_{\mathsf{lo}}(x) + x^n\mathsf{t}_{\mathsf{mid}}(x) + x^{2n}\mathsf{t}_{\mathsf{hi}}(x) = t(x)$ and only one possible value for opening proofs $[\mathsf{W}_{\mathfrak{z}}, \mathsf{W}_{\mathfrak{z}\omega}]_1$ such that the verification equation is satisfied. The simulator computes these elements accordingly. The public polynomials are evaluated honestly as $\bar{s}_{\sigma 1} \leftarrow \mathsf{S}_{\sigma 1}(\mathfrak{z})$ and $\bar{s}_{\sigma 2} \leftarrow \mathsf{S}_{\sigma 2}(\mathfrak{z})$. Also, the challenges $\beta, \gamma, \ldots$ are computed from the correct distribution. The simulator fails when either $x^{n+1} = 0$, $\mathfrak{z} = x$, $\mathfrak{z}\omega = x$, or $\mathsf{Z}_{\mathbb{H}}(x) = 0$ (the last

three conditions are needed to avoid division by 0 in the simulator); this happens only with a negligible probability. □